14 haftalık yol haritasında 11. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

# HAFTA 11

# Graf Algoritmalarına Giriş

## Temsil, Genişlik ve Derinlik Öncelikli Arama

MODÜL IV · GRAFLAR VE AÇGÖZLÜ STRATEJİ · H11–H12

> **HAFTANIN BÜYÜK SORUSU**
>
> *Bir labirentten çıkışın en kısa yolu nasıl bulunur ve bir navigasyon uygulaması, bir sosyal ağ ve bir ders önkoşul listesi neden aynı soyut yapıyla modellenebilir?*

**Dr. Öğr. Üyesi Mehmet Sevri**
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

# İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ..... 4
1.1 Öğrenme çıktıları ..... 5
2. Geçen Haftadan ve AP-1'den Bağlantı ..... 5
3. Kavramsal Anlatım: Graf, Model ve Temsil ..... 6
3.1 Temel kavramlar ..... 6
3.2 Modelleme: köşe ve kenar neyi temsil ediyor? ..... 6
3.3 Temsil: kenar listesi, komşuluk matrisi, komşuluk listesi ..... 7
3.4 Dosyadan graf okuma ve biçim hataları ..... 9
3.5 Seyrek mi yoğun mu? Bellek hesabı ve Kaputun Altında ..... 9
4. Algoritmalar ..... 10
4.1 Genişlik öncelikli arama (BFS): kuyrukla katman katman ..... 11
4.2 BFS ile en kısa yol: ebeveyn dizisi ve yolun geri çıkarılması ..... 13
4.3 Derinlik öncelikli arama (DFS): üç renk, keşif ve bitiş ..... 14
4.4 Açık yığınla DFS ve Kaputun Altında: derin özyinelemenin bedeli ..... 17
4.5 Bağlı bileşenler ..... 20
4.6 İki parçalılık: BFS ile 2-renklendirme ..... 22
4.7 Topolojik sıralama I: Kahn algoritması ..... 24
4.8 Topolojik sıralama II: DFS bitiş sırası ve yönlü döngü tespiti ..... 28
5. Çözümlü Örnekler ..... 30
5.1 Izgarada taşkın doldurma ve ada sayma (Örnek 9) ..... 30
5.2 Labirentte en kısa yol: BFS ve H3'ün geri izlemesi (Örnek 10) ..... 31
5.3 “Kaç adım uzakta?”: ortak yazarlık ağı ve Erdős sayısı (Örnek 13) ..... 32
6. Haftanın Deneyi: Temsil Seçimi ve Derin Özyineleme ..... 33
6.1 (a) Matris, liste ve CSR: süre ve bellek ..... 33
6.2 (b) Derin zincir: özyinelemeli ve açık yığınlı DFS ..... 34
6.3 (c) CSR: bitişik bellek ne kazandırır? ..... 35
6.4 Kestirimler, sınırlılıklar ve kendi makinenizde ..... 35
7. Hata Avcısı ..... 35
8. Dil Penceresi: C · C++ · Python ..... 37
9. Mühendis Gözüyle: Temsil Bir Modelleme Kararıdır ..... 38
10. Mini-OBS 2 v2.10: Ders Önkoşul Grafı ..... 38
10.1 Gereksinimler ..... 39
10.2 İpuçları ..... 39
10.3 Başlangıç noktası ..... 39

10.4 Kabul ölçütleri ve test senaryoları ..... 40
11. Alıştırmalar ve Atölyeler ..... 41
11.1 İzleme ve çıktı tahmini (A) ..... 41
11.2 Karmaşıklık (B) ..... 42
11.3 Değişmez yazma (C) ..... 42
11.4 Hata bulma (D) ..... 43
11.5 Parsons (E) ..... 43
11.6 Kod yazma (F) ..... 44
11.7 Diller arası okuma (G) ..... 44
11.8 Atölye 1 · İnsan BFS'si: tanışıklık ağında dalga (bütün sınıf, ≈ 20 dk) ..... 45
11.9 Atölye 2 · Labirent çözücü yarışması: BFS, DFS ve geri izleme (gruplar, ≈ 30 dk) ..... 45
11.10 Atölye 3 · Ders planlayıcı: önkoşul listesinden DAG ve topolojik sıra (eşli, ≈ 30 dk) ..... 46
12. Kendini Değerlendirme ..... 46
13. İleri Bakış ve Merak Kutusu ..... 46
14. Özet, Kavram Haritası ve Terim Sözlüğü ..... 47
15. Kaynaklar ..... 48
Ek A. Dil Karşılıkları ..... 49
A.1 Örnek 1 · Komşuluk matrisi ..... 49
A.2 Örnek 2 · Komşuluk listesi ve dosyadan okuma ..... 50
A.3 Örnek 3 · BFS katmanları ..... 51
A.4 Örnek 4 · En kısa yol ve ebeveyn ..... 51
A.5 Örnek 5 · Özyinelemeli DFS ..... 52
A.6 Örnek 6 · Açık yığınlı DFS ..... 53
A.7 Örnek 7 · Bağlı bileşenler ..... 54
A.8 Örnek 8 · İki parçalılık ..... 56
A.9 Örnek 9 · Taşkın doldurma ve ada sayma ..... 57
A.10 Örnek 10 · Labirentte BFS ..... 58
A.11 Örnek 11 · Kahn ..... 60
A.12 Örnek 12 · DFS topolojik sırası ..... 61
A.13 Örnek 13 · Ortak yazarlık ağı ..... 62

# 1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bir labirentten çıkışın en kısa yolu nasıl bulunur ve bir navigasyon uygulaması, bir sosyal ağ ve bir ders önkoşul listesi neden aynı soyut yapıyla modellenebilir?

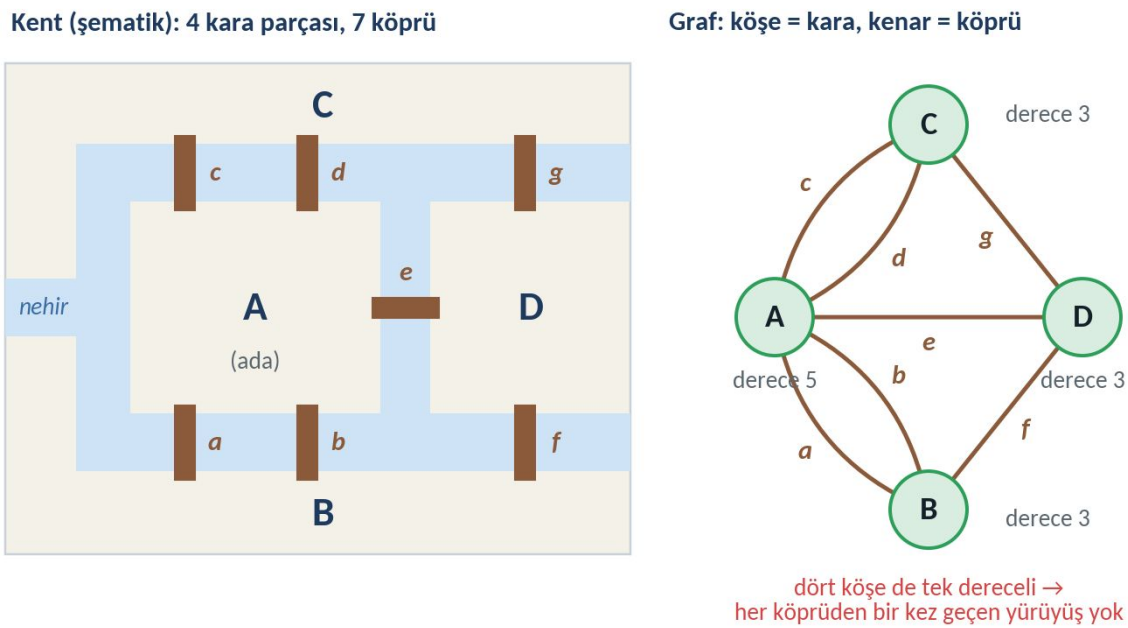
H3'te labirentten çıkışı **geri izlemeyle** (backtracking) aradık: ilk bulunan yol çıkışa götürür ama **en kısa** olmak zorunda değildir. Açık bir odada geri izleme 14 adımlık bir yol bulur, oysa çıkış iki hücre ötededir; genişlik öncelikli arama (BFS) 2 adımlık yolu 6 hücre işleyerek bulur (Örnek 10). H3'ün labirentinde ikisi de 22 adımlık yola ulaşır (BFS 61 hücre işler, geri izleme 44 çağrı yapar), ama yalnız BFS'nin yolu **her zaman** en kısadır. Farkı yaratan, labirenti bir **graf** olarak görmektir: açık hücreler köşe, yan yana iki açık hücre kenardır.

Navigasyonda kavşaklar köşe, yollar uzunluğu (ağırlığı) olan kenardır; sosyal ağda kişiler köşe, tanışıklıklar kenardır; önkoşul listesinde dersler köşe, "A, B'nin önkoşuludur" bağıntısı **yönlü** kenardır. Aynı soyut yapı aynı algoritmalarla çözülür: "kaç adım uzakta?" sorusunu BFS, "bütün parçaları dolaş, döngü var mı?" sorusunu derinlik öncelikli arama (DFS), "hangi sırayla?" sorusunu topolojik sıralama yanıtlar. H12 kenarlara ağırlık ekleyerek navigasyonun gerçek sorusuna (Dijkstra) geçecek.

> **GERÇEK DÜNYADAN · Königsberg'in yedi köprüsü: bir bulmacadan graf kuramına**
>
> Königsberg'de (bugün Kaliningrad) Pregel ırmağının kolları kenti dört kara parçasına ayırıyor, yedi köprü bunları bağlıyordu: her köprüden **tam bir kez** geçen bir yürüyüş var mıdır? Leonhard Euler "Solutio problematis ad geometriam situs pertinentis" (konum geometrisine ait bir problemin çözümü) başlıklı makalesinde kara parçalarını A, B, C, D, köprüleri a–g diye adlandırdı: A adasına beş köprü (a–e), B, C ve D'ye üçer köprü bağlanır (Euler, 1741; Şekil 11.1). Makale St. Petersburg Akademisi'ne 26 Ağustos 1735'te sunuldu; 1736 yılına ait 8. ciltte yer aldı, cilt 1741'de basıldı (Enestrom, 1910–1913).
>
> Akıl yürütme bugünkü dille bir **derece** sayımıdır: yürüyüş her geçişte bir kara parçasına bir köprüyle girer, bir köprüyle çıkar; başlangıç ve bitiş dışında her kara parçasının köprü sayısı çift olmalıdır. Königsberg'de dördü de tektir: böyle bir yürüyüş yoktur. Kentin biçimi önemsizdir, yalnız "hangi kara parçası hangisine kaç köprüyle bağlı" bilgisi kalır: bugün **graf** (çizge; graph) dediğimiz soyutlama. Makale graf kuramının başlangıcı sayılır.

> **GERÇEK DÜNYADAN · Labirentten devre kartına: Moore (1959) ve Lee (1961)**
>
> - Edward F. Moore "The shortest path through a maze" (labirentte en kısa yol) başlıklı çalışmasını 1959'da Harvard'daki bir anahtarlama kuramı sempozyumunun bildiriler kitabında yayımladı (Moore, 1959); genişlik öncelikli aramanın "kaynaktan dalga dalga yayılma" fikri bu labirent sorusuyla birlikte anılır.
> - Bell Laboratuvarları'ndan C. Y. Lee aynı fikri devre kartlarında bağlantı yolu bulmaya uyguladı: yöntemini "bir kaynaktan genişleyen dalgaların bilgisayar modeli" diye tanımladı, IBM 704'te programladı, labirent problemlerini de çözebileceğini belirtti ve en kısa yol örneklerinde Moore'un algoritmalarından birini kullandığını yazdı (Lee, 1961). Örnek 10'un ızgara BFS'si aynı dalga yayılımıdır.

**Şekil 11.1**
*Königsberg köprüleri (şematik, Euler'in harfleriyle) ve graf soyutlaması: köşe = kara parçası, kenar = köprü; dört köşe de tek dereceli*




## 1.1 Öğrenme çıktıları

- **ÖÇ 11.1** Gerçek dünya problemlerini graf olarak modeller (köşe, kenar, yön, ağırlık); komşuluk matrisi ve komşuluk listesi temsillerini karşılaştırır. (Bölüm 3, Örnek 1–2)
- **ÖÇ 11.2** Genişlik öncelikli aramayı (BFS) kuyrukla gerçekler; ağırlıksız graflarda en kısa yol uzaklıklarını ve yolun kendisini üretir. (Bölüm 4.1–4.2, Örnek 3–4, 13)
- **ÖÇ 11.3** Derinlik öncelikli aramayı (DFS) özyinelemeli ve açık yığınla gerçekler; bağlı bileşenleri ve döngüleri bulur. (Bölüm 4.3–4.6, Örnek 5–8)
- **ÖÇ 11.4** Yönlü döngüsüz graflarda (DAG) topolojik sıralamayı Kahn algoritması ve DFS ile yapar. (Bölüm 4.7–4.8, 10, Örnek 11–12)
- **ÖÇ 11.5** Izgara problemlerini (labirent, taşkın doldurma, ada sayma) örtük graf olarak çözer; BFS ve DFS'nin O(V + E) karmaşıklığını açıklar. (Bölüm 5.1–5.2, 6, Örnek 9–10)

# 2. Geçen Haftadan ve AP-1'den Bağlantı

Modül IV önceki modüllerin araçlarını buluşturur. **Modül I** (H1–H3): değişmez, sayaç, özyineleme; H3'ün labirent geri izlemesi Örnek 10'da BFS ile yarışır. **Modül II** (H4–H7): ikili arama (Örnek 13'te ad → köşe), sıralama (komşu listeleri), öbek (Kahn'ın en küçük öbeği). **Modül III** (H8–H10): H8'in **dinamik dizisi** komşuluk listesidir; H9'un **kuyruğu** BFS'yi, **yığını** DFS'yi yönetir; H10'un **karma tablosu** köşeler 0 … V − 1 değilse ziyaret kümesidir (Mini-OBS 2'de ders kodu → köşe). AP-1 H11'in **2B dizisi** komşuluk matrisidir; ızgara ve 4/8 komşuluk bir **örtük graftır**.

> TAHMİN ET · **Geri çağırma: dört soru (yanıtlar aşağıda)**
>
> - **S1 (H10).** Örnek 8 n = $10^5$ öğrenci numarasında yinelenenleri karma kümesiyle kaç hücre yoklamasıyla bulur? Bütün çiftleri karşılaştıran kaba kuvvet kaç karşılaştırma yapar?
> - **S2 (H10).** Python'da bir `list`'i `dict` anahtarı ya da `set` öğesi yaparsak ne olur? Bir ızgara hücresini (satır, sütun) ziyaret kümesine nasıl koymalıyız?

- **S3 (H9).** Kuyrukla "katman katman işleme" örneğinde 1'den başlayıp x'in ardılları 2x ve 2x + 1 (≤ 15) iken 3. düzeyde hangi sayılar işlenir?
- **S4 (AP-1 H11).** 5 × 5 bir ızgarada köşe hücresi (0, 0), kenar hücresi (0, 2) ve iç hücre (2, 2) kaç 4-komşuya ve kaç 8-komşuya sahiptir?

**Yanıtlar.** S1: 107 133 yoklama; kaba kuvvet n(n − 1)/2 = 4 999 950 000 karşılaştırma (H10 Örnek 8). Bu hafta aynı fikir "bu köşeyi gördüm mü?" sorusudur: her köşe bir kez işaretlenir, bir kez denetlenir. S2: `TypeError: unhashable type: 'list'` (H10 Hata Avcısı 6): `list` değiştirilebilir olduğu için karma edilemez; hücre `(i, j)` **demeti** olarak saklanır ya da tek bir indise çevrilir (`i * Y + j`; Örnek 9 böyle yapar). S3: 8, 9, 10, 11, 12, 13, 14, 15 (H9 Örnek 9): kuyruk, bir düzeyin bütün öğelerini sonraki düzeyden önce işler; BFS'nin katmanları aynı düzendir. S4: köşe 2 ve 3, kenar 3 ve 5, iç 4 ve 8 (AP-1 H11 Kavram 7): ızgarada bir hücrenin **derecesi** konumuna bağlıdır; sınır denetimi, komşuya erişmeden **önce** yapılmalıdır (Hata Avcısı 3).

# 3. Kavramsal Anlatım: Graf, Model ve Temsil

## 3.1 Temel kavramlar

Bir **graf** G = (V, E), **köşeler** (vertex) kümesi V ile köşeleri bağlayan **kenarlar** (edge) kümesi E'den oluşur; köşe sayısını V, kenar sayısını E ile gösterir, köşeleri 0 … V − 1 diye numaralarız (Cormen vd., 2022). Kenarın yönü yoksa graf **yönsüzdür** (undirected; u–v), varsa **yönlüdür** (directed; u → v). Kenara bir sayı (uzunluk, süre, maliyet) bağlanırsa graf **ağırlıklıdır** (weighted); ağırlıklar H12'nin konusudur.

- **Derece** (degree): yönsüzde bir köşeye değen kenar sayısı. Her kenar iki ucunun derecesine birer katkı yaptığı için Σ derece = 2E'dir (**el sıkışma lemması**; handshake lemma). Örnek 1'in yönsüz grafı G'de (V = 8, E = 10) `Σ derece = 20, 2E = 20`; en büyük derece 4 (köşe 4). Yönlüde **çıkış derecesi** (out-degree) ve **giriş derecesi** (in-degree) ayrıdır; Σ çıkış = Σ giriş = E (Örnek 1'in yönlü grafı G3'te çıkış 3 1 2 1 2 1 1 1, giriş 1 2 2 2 2 2 1 0).
- **Yol** (path): ardışık köşeleri kenarla bağlı bir köşe dizisi; uzunluğu kenar sayısıdır. **Döngü** (cycle): başladığı köşede biten, başka köşe ve kenar tekrarlamayan yol (yönsüzde en az üç kenarlı; yönlüde yönlü döngü). İki köşe arasındaki **uzaklık**, aralarındaki en kısa yolun kenar sayısıdır (yol yoksa ∞; programlarımızda −1).
- **Bağlılık** (connectivity): yönsüz grafta her köşe çiftini bir yol bağlıyorsa graf bağlıdır; değilse **bağlı bileşenlere** (connected component) ayrılır. **Ağaç** (tree): bağlı ve döngüsüz yönsüz graf; V köşeli bir ağacın tam V − 1 kenarı vardır.
- **DAG**: yönlü döngüsüz graf (yönlü dönüşsüz graf diye de anılır; directed acyclic graph). Önkoşul grafları DAG olmalıdır: A → B → C → A döngüsü "A'yı almak için önce A'yı almalısınız" demektir.
- **Seyrek ve yoğun** (sparse, dense): yönsüz bir grafın en çok V(V − 1)/2 kenarı olabilir. E, V ile aynı ölçekteyse (E = O(V); yol ağları, sosyal ağlar) graf seyrektir; E, V² ölçeğindeyse yoğundur. Temsil seçimi bu ayrıma dayanır (Bölüm 3.3).

## 3.2 Modelleme: köşe ve kenar neyi temsil ediyor?

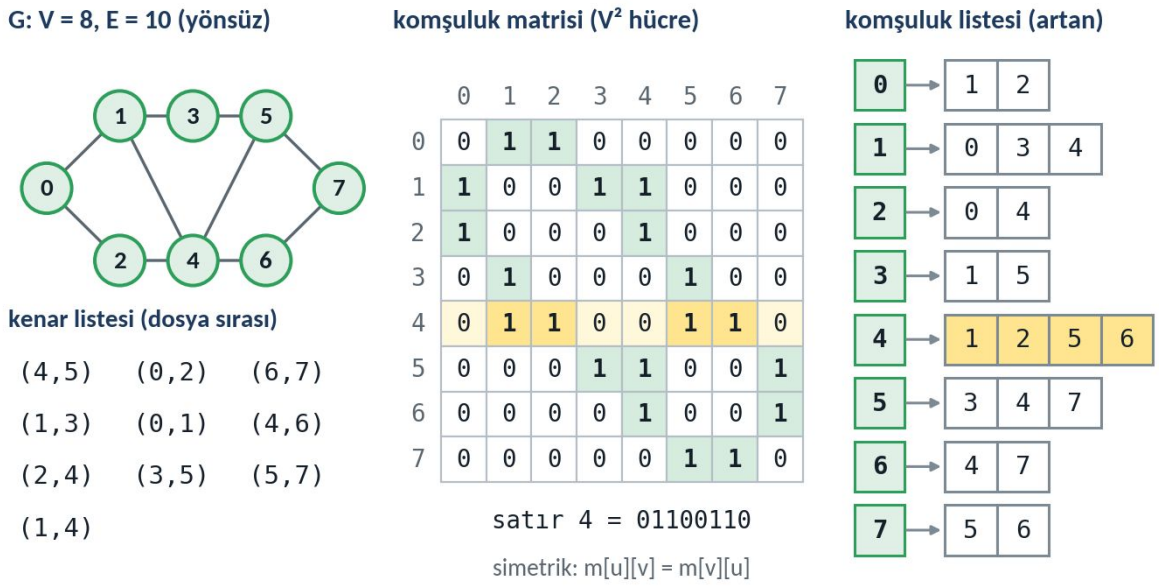
Modellemek üç karardır: **köşe** nedir, iki köşe ne zaman **kenarla** bağlıdır, kenarın **yönü** ve **ağırlığı** var mı? Yol ağında kavşaklar köşe, yol parçaları ağırlıklı kenardır (tek yönlü sokak yönlü kenar); sosyal ağda kişiler ve tanışıklıklar (takip ilişkisi yönlüdür); ders kataloğunda dersler ve "önkoşul" yönlü kenarı; web'de sayfalar ve bağlantılar (döngüler doğaldır); oyunlarda **durumlar** ve hamleler (durum uzayı; H3'ün N vezir araması). Sonra soru bir graf sorusudur: "en az kaç adım?" en kısa yol, "kim kime ulaşır?" bağlı bileşenler, "hangi sırayla?" topolojik sıra. **Örtük graf** (implicit graph): kenarlar saklanmaz,

hesaplanır. Izgarada her hücre köşedir; 4-komşulukta yukarı, aşağı, sol, sağ, 8-komşulukta çaprazlar da komşudur; S × Y ızgarada V = S·Y, E ≤ 2·S·Y (4-komşu): seyrektir ve komşular indis aritmetiğiyle bulunur (Örnek 9, 10).

## 3.3 Temsil: kenar listesi, komşuluk matrisi, komşuluk listesi

Aynı graf üç biçimde tutulabilir (Şekil 11.2). **Kenar listesi** (u, v) çiftlerinin dizisidir: saklaması kolay, ama "u'nun komşuları?" sorusu bütün listeyi tarar. **Komşuluk matrisi** (adjacency matrix) V × V'lik 2B dizidir: `matris[u][v] = 1` ⇔ u–v kenarı; yönsüzde simetriktir. **Komşuluk listesi** (adjacency list) her köşe için komşularının dizisidir (C'de H8'in dinamik dizilerinden oluşan dizi). G'de öğe sayısı: matris $V^2 = 64$, liste $V + 2E = 28$, kenar listesi $2E = 20$.

**Şekil 11.2**
*Aynı graf G (V = 8, E = 10) üç temsilde: kenar listesi (dosya sırası), komşuluk matrisi (satır 4 vurgulu) ve artan sıralı komşuluk listesi*




öğe sayısı: matris $V^2 = 64$ · liste $V + 2E = 28$ · kenar listesi $2E = 20$

Matriste u'nun komşularını bulmak, u satırının **tamamını** okumaktır: komşu sayısı ne olursa olsun V hücre (Örnek 1):

c/H11_01_komsuluk_matrisi.c · satır 75–86

```c
/* u'nun komşuları: satır u baştan sona taranır (V hücre) */
static int komsulari_bul(int V, int u, int komsu[])
{
    int k = 0;
    for (int v = 0; v < V; v++) {
        okunan_hucre++;
        if (matris[u][v] != 0) {
            komsu[k++] = v;
        }
    }
    return k;
}
```

Örnek 1 bütün komşuları bulurken $V^2 = 64$ hücre okur; bunların yalnız 20'si 1'dir (her kenar iki hücrede). Listede u'nun dizisi yalnız komşularını tutar: bütün köşeler için Σ derece = 2E adım. Örnek 2 kenarları dosya sırasıyla ekler; yönsüzde her kenar **iki** listeye girer:

c/H11_02_komsuluk_listesi.c · satır 208–221

```c
    for (int u = 0; u < g->V; u++) {
        g->komsu[u].veri = NULL;
        g->komsu[u].boyut = g->komsu[u].kapasite = 0;
    }
    for (int i = 0; i < kl->E; i++) {
        const Kenar *k = &kl->kenar[i];
        if (dd_ekle(&g->komsu[k->u], k->v) != 0 ||
            (!g->yonlu && dd_ekle(&g->komsu[k->v], k->u) != 0)) {
            graf_birak(g);
            return -1;
        }
    }
    return 0;
}
```

`dd_ekle` H8'in iki katına büyüyen dinamik dizisidir. 4'ün listesi önce dosya sırasıyla `5 6 2 1` olur, `qsort` ile `1 2 5 6`. **Bu haftanın bütün programlarında komşular artan köşe numarasıyla dolaşılır:** ziyaret sırası, keşif/bitiş zamanları ve topolojik sıra buna bağlıdır; kural üç dilde aynı çıktıyı ve elle izlenebilir sonuçları güvenceler. Örnek 2 temsilleri birbirine dönüştürür: matristen çıkarılan kenar listesinden yeniden kurulan liste, 3. adımdaki listeyle aynıdır (çıktıda "evet").

**Tablo 11.1**
*Üç temsilin bellek ve işlem maliyetleri (V köşe, E kenar)*

| İşlem | Kenar listesi | Komşuluk matrisi | Komşuluk listesi |
|---|---|---|---|
| Bellek | Θ(E) | Θ($V^2$) | Θ(V + E) |
| "u–v kenarı var mı?" | Θ(E) | Θ(1) | Θ(derece(u)) |
| u'nun komşularını dolaş | Θ(E) | Θ(V) | Θ(derece(u)) |
| Bütün köşeleri dolaş (BFS/DFS) | — | Θ($V^2$) | Θ(V + E) |
| Kenar ekle | Θ(1) amortize | Θ(1) | Θ(1) amortize |
| Uygun olduğu yer | dosya, giriş/çıkış, H12 Kruskal | yoğun, küçük V; O(1) komşuluk sorusu | seyrek ve büyük graflar (çoğu gerçek ağ) |

**Tablo 11.2**
*Komşuluk matrisiyle dolaşma kimlik kartı*

| ALGORİTMA KİMLİK KARTI **Komşuluk matrisiyle dolaşma** *(graph traversal with an adjacency matrix)* | |
|---|---|
| **Strateji** | Çıkan her u için matrisin u. satırının tamamı taranır (V hücre) |
| **Girdi önkoşulu** | V × V matris ($V^2$ hücre bellekte) |
| **Zaman (en iyi · ortalama · en kötü)** | Θ($V^2$) · Θ($V^2$) · Θ($V^2$) |
| **Ek bellek** | O($V^2$) |

| | |
|---|---|
| **Özellikler** | Okunan hücre $V^2$, kenar sayısından bağımsız; seyrek grafta listeden yüzlerce kat yavaş (Bölüm 6.1) |
| **Değişmez / doğruluk** | u satırında matris[u][v] = 1 olan her v, u'nun komşusudur |
| **Kütüphane karşılığı** | C: 2B dizi (elle) · C++: Boost.Graph adjacency_matrix · Python: networkx (yalnız kâhin) |
| **Ne zaman kullanılır** | Yoğun graf, küçük V, O(1) "komşu mu?" sorusu |
| **Ne zaman kullanılmaz** | Seyrek ve büyük graflar (yol ağları, sosyal ağlar) |

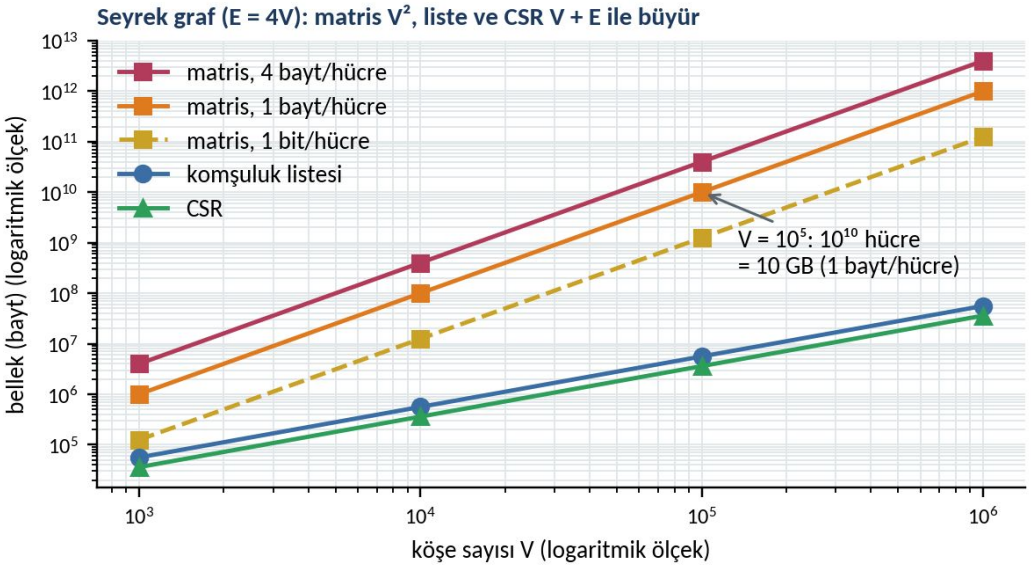
## 3.4 Dosyadan graf okuma ve biçim hataları

H11 ve H12'nin graf dosyaları aynı biçimdedir: ilk satır `V E [yonlu|yonsuz]`, ardından E satır `u v [w]` (w ağırlık; bu hafta yok); köşeler 0 … V − 1, `#` ile başlayan satır yorumdur. Okuyucu her satırı denetler, hatayı satır numarasıyla bildirir (stderr'e ileti, çıkış kodu 1; kullanım hatası 2). Örnek 2'nin ayrıntılı okuyucusu yedi bozuk dosyada üç dilde aynı iletiyi verir; örneğin `hatali_kose.txt: satır 5: köşe 9 aralık dışı (0..7)`, `hatali_eksik.txt: kenar eksik: E = 5, okunan 4`.

Örnek 3–8, 11 ve 12 aynı biçimi kısa bir okuyucuyla okur (tek ileti: "satır N: biçim hatası"). 255 ve daha uzun karakterli satırlar (yorumlar dahil) reddedilir: okuyucu sabit uzunluklu tamponla çalışır ve kesilen satırı sessizce kabul etmez; uzun açıklamaları birkaç yorum satırına bölün.

## 3.5 Seyrek mi yoğun mu? Bellek hesabı ve Kaputun Altında

**Şekil 11.3**
*Seyrek grafta (E = 4V) temsil belleği, hesap (Kavram 1): matris $V^2$ ile, liste ve CSR V + E ile büyür (log–log)*



KAPUTUN ALTINDA · **Komşuluk matrisinde bellek patlaması**

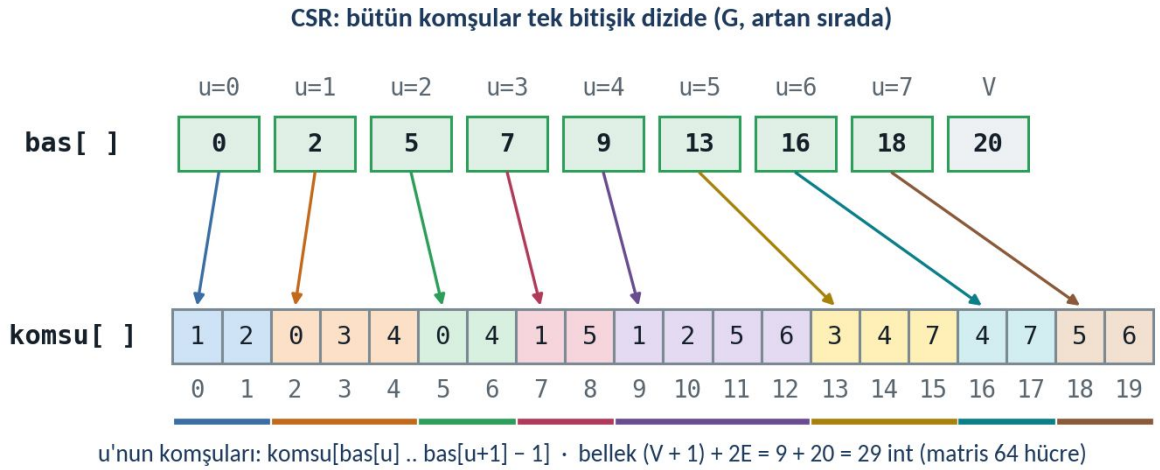
Seyrek bir grafta matris belleğin neredeyse tamamını sıfırlara harcar. Kavram 1 gereken belleği **hesaplar** (ölçüm değil; 1 GB = $10^9$ bayt). V = $10^5$ köşeli bir sosyal ağda matris $V^2 = 10^{10}$ hücredir: hücre başına 1 bayt bile 10 GB, `int` hücreyle 40 GB eder; bit başına bir hücreyle sıkıştırılsa da 1,25 GB. Aynı graf (E = 4V) listeyle 5,60 MB, CSR ile 3,60 MB tutar: matris listenin ≈ 1 786 katıdır. Matris $V^2$ ile, liste V + E ile büyür (Şekil 11.3); Haftanın Deneyi'nde matris seyrek grafta V = $2 \cdot 10^4$'ten sonra 1,5 GB'lık süreç bütçesini aştı (Bölüm 6). Seyrek graflarda liste bir tercih değil **zorunluluktur**; matris yoğun graflarda ve O(1) "komşu mu?" sorusunda değerlidir.

Python'da liste listesinin her hücresi 8 baytlık bir **başvurudur**: `[0] * 1000` satırı 8056 bayt (hücre başına ≈ 8,06), `bytearray(1000)` 1057 bayt (hücre başına ≈ 1,06) kaplar (CPython 3.11, bu çalıştırmada): V = $10^5$ için liste listesi yalnız başvurulara 80 GB ister.

MERAK KUTUSU · **Sıkıştırılmış seyrek satır (CSR): bütün komşular tek dizide**

Listede her köşenin dizisi ayrı bir `malloc` bloğudur ve bellekte dağınık durabilir. **CSR** (compressed sparse row) bütün komşuları **tek bitişik** `komsu[]` dizisine yazar: u'nun komşuları `komsu[bas[u] .. bas[u+1] − 1]` (Şekil 11.4). Kurulum üç geçiştir (Kavram 2): dereceleri say (G'de `2 3 2 2 4 3 2 2`), önek toplamıyla `bas`'ı kur (`0 2 5 7 9 13 16 18 20`), kenarları yerlerine yaz. Bellek: `(V + 1) + 2E = 9 + 20 = 29 int; matris V² = 64 hücre`. Komşular ardışık adreslerden okunur, önbellek satırı sonrakini de getirir: deneyde C'de V = $10^5$ seyrek grafta CSR listeden 1,58 kat hızlıydı (Bölüm 6.3). Mini-OBS 2 v2.10'un önkoşul grafı bu biçimdedir (v2.10 belgesi § 4.1). Soru: CSR'ye sonradan kenar eklemek neden pahalıdır?

**Şekil 11.4**
*CSR temsili (Kavram 2, graf G): bas[u], u'nun komşularının komsu[] içindeki başlangıcıdır; her köşenin komşuları aynı renkte*




# 4. Algoritmalar

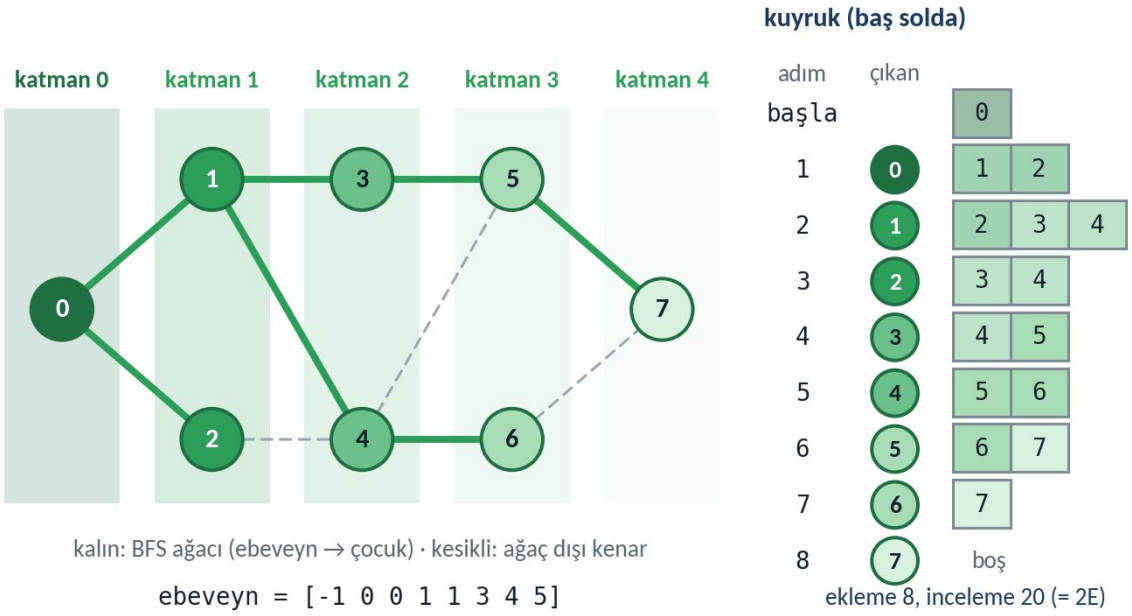
Sekiz algoritmayı on bir adımlı standart anlatımla (problem, fikir, elle izleme, sözde kod, doğruluk, analiz ve kimlik kartı, C, üç dil, test, deney, varyantlar) işliyoruz: BFS ve en kısa yol (4.1–4.2), DFS (4.3–4.4), bağlı bileşenler ve iki parçalılık (4.5–4.6), topolojik sıralama (4.7–4.8). Elle izlemeler çoğunlukla G üzerindedir (Şekil 11.2); üç dil adımında kısa kesitler, Ek A'da öteki fonksiyonlar verilir.

## 4.1 Genişlik öncelikli arama (BFS): kuyrukla katman katman

**1 · Problem.** Bir kaynak köşe s'den ulaşılabilen bütün köşeleri s'ye **uzaklık** (en az kenar sayısı) sırasıyla dolaşmak; her köşenin uzaklığını ve bir en kısa yolunu bulmak. **2 · Fikir.** Suya atılan taşın halkaları gibi: önce 1 kenar uzaktakiler, sonra 2 kenar uzaktakiler (**katmanlar**, layers). Bunu bir **kuyruk** (FIFO, H9) sağlar: baştan bir köşe çıkar, henüz görülmemiş komşuları **sona** eklenir; bir katmanın bütün köşeleri sonraki katmandan önce işlenir. Köşe **kuyruğa eklenirken** "görüldü" diye işaretlenir; böylece hiçbir köşe kuyruğa ikinci kez girmez (Hata Avcısı 1).

**3 · Elle izleme.** G'de BFS(0) (Örnek 3; Şekil 11.5, sağda adım adım kuyruk): kuyruk [0]; 0 çıkar, 1 ve 2 eklenir: [1 2]; 1 çıkar, komşuları 0 (görüldü), 3, 4: [2 3 4]; 2 çıkar, 0 ve 4 görülmüştür: [3 4]. **Tahmin edin:** kuyruk [4 5] iken sıradaki çıkan köşe ve kuyruğa eklenen hangisidir? (Yanıt: 4 çıkar; 1, 2, 5 görülmüştür, yalnız 6 eklenir: [5 6].) Ziyaret sırası 0 1 2 3 4 5 6 7, katmanlar [0] [1 2] [3 4] [5 6] [7], ebeveyn = [-1 0 0 1 1 3 4 5]. Kaynak değişince katmanlar değişir (BFS(7)'nin katmanları [7] [5 6] [3 4] [1 2] [0]); yönlü G3'te BFS(0) yalnız ulaşılabilenleri dolaşır, 7 köşesine ulaşılamaz.

**Şekil 11.5**
*BFS(0) dalgası, G (Örnek 3): katman renkleri, BFS ağacı (kalın) ve adım adım kuyruk (baş solda)*




SÖZDE KOD · 4 · Sözde kod: genişlik öncelikli arama

```
algoritma BFS(G, s)                          ▷ uzak[v] = −1: v henüz görülmedi
    her v için: uzak[v] ← −1;  ebeveyn[v] ← −1
    uzak[s] ← 0;  KUYRUĞA-EKLE(Q, s)         ▷ işaret EKLERKEN
    iken Q boş değil
        u ← KUYRUKTAN-ÇIKAR(Q)               ▷ baştan
        u'nun her v komşusu için (artan sırada)
            eğer uzak[v] = −1 ise
                uzak[v] ← uzak[u] + 1;  ebeveyn[v] ← u;  KUYRUĞA-EKLE(Q, v)
    döndür uzak, ebeveyn
```

**5 · Doğruluk. Katman değişmezi:** kuyruk her an [d, …, d, d + 1, …, d + 1] biçimindedir. *Başlatma:* [0]. *Koruma:* çıkan u kuyruğun başıdır, uzak[u] = d en küçüktür; eklenen her v'ye d + 1 verilir ve sona eklenir. Bu yüzden köşeler uzaklık sırasıyla çıkar. *Sonlanma:* kuyruk boşalınca ulaşılabilen her köşe bir kez çıkmıştır; v ilk kez u'dan görüldüğünde uzak[v] = uzak[u] + 1 gerçek uzaklıktır: daha kısa bir yol olsaydı, o

yolda v'den önceki w'nin uzaklığı uzak[u]'dan küçük olur, w u'dan önce çıkar ve v'yi önce o görürdü. Ulaşılamayan köşeler kuyruğa hiç girmez (uzak = −1). **6 · Analiz.** Her köşe kuyruğa **en çok bir kez** girer: V ekleme/çıkarma. Her çıkan köşenin listesi bir kez dolaşılır: yönsüzde Σ derece = 2E, yönlüde E inceleme. Toplam **Θ(V+E)**, ek bellek O(V). Örnek 3'ün G'deki sayaçları: `kuyruğa ekleme = 8, kenar inceleme = 20 (= 2E: her kenar iki uçtan), en uzun kuyruk = 3`. Komşuluk matrisiyle her çıkan köşe için V hücre okunur: Θ(V²) (Tablo 11.2).

**Tablo 11.3**
*Genişlik öncelikli arama (BFS) kimlik kartı*

| ALGORİTMA KİMLİK KARTI | **Genişlik öncelikli arama (BFS)** *(breadth-first search (BFS))* |
|---|---|
| **Strateji** | Kuyrukla katman katman; işaret kuyruğa EKLERKEN |
| **Girdi önkoşulu** | Kaynak köşe; en kısa yol yalnız ağırlıksız grafta |
| **Zaman (en iyi · ortalama · en kötü)** | Θ(V + E) · Θ(V + E) · Θ(V + E) |
| **Ek bellek** | O(V) |
| **Özellikler** | Ağırlıksız en kısa yol ve katmanlar; G'de 8 ekleme, 20 kenar inceleme |
| **Değişmez / doğruluk** | Kuyrukta d'lerin ardından yalnız d + 1'ler; giren v'nin uzak[v]'si kesin |
| **Kütüphane karşılığı** | C: elle (kuyruk H9) · C++: Boost.Graph · Python: networkx (yalnız kâhin) |
| **Ne zaman kullanılır** | Ağırlıksız en kısa yol, katmanlar, taşkın doldurma, iki parçalılık |
| **Ne zaman kullanılmaz** | Kenarlar ağırlıklıysa (Dijkstra, H12) |

**7 · C gerçekleştirimi.** Örnek 4'ün `bfs_uzaklik` döngüsü (kuyruk H9'un dizi kuyruğudur; her köşe en çok bir kez girdiği için V kapasite yeter; Örnek 3 aynı döngüye adım adım izleme satırları ekler):

C `c/H11_04_bfs_en_kisa_yol.c · satır 203–216`

```c
    q[son++] = s;
    while (bas < son) {
        int u = q[bas++];
        for (size_t i = 0; i < g->komsu[u].boyut; i++) {
            int v = g->komsu[u].veri[i];
            if (uzak[v] == -1) {
                uzak[v] = uzak[u] + 1;
                ebeveyn[v] = u;          /* v'ye ilk u'dan ulaşıldı */
                q[son++] = v;
            }
        }
    }
    free(q);
    return son;
```

**8 · Üç dil.** C++'ta kuyruk `std::queue<int>`, Python'da `collections.deque`'tir (`popleft` O(1); `list.pop(0)` O(n): Kavram 3, Bölüm 8):

C++ `H11_04_bfs_en_kisa_yol.cpp · satır 165–176`

```cpp
    while (!q.empty()) {
        int u = q.front();
        q.pop();
```

```
        ulasilan++;
        for (int v : g.komsu[u]) {
            if (uzak[v] == -1) {
                uzak[v] = uzak[u] + 1;
                ebeveyn[v] = u;          // v'ye ilk u'dan ulaşıldı
                q.push(v);
            }
        }
    }
```

Python **H11_04_bfs_en_kisa_yol.py · satır 121–128**

```
    while q:
        u = q.popleft()
        ulasilan += 1
        for v in g.komsu[u]:
            if uzak[v] == -1:
                uzak[v] = uzak[u] + 1
                ebeveyn[v] = u          # v'ye ilk u'dan ulaşıldı
                q.append(v)
```

**9 · Test.** G, yönlü G3, bileşenli graf, farklı kaynaklar; üç dilin çıktısı bayt bayt aynı; Kavram 4 networkx ile karşılaştırır (yalnız kâhin; Hagberg vd., 2008). **10 · Deney.** Listeyle süre V ile neredeyse doğrusal, matrisle karesel (Bölüm 6.1). **11 · Varyantlar.** Çok kaynaklı BFS (Alıştırma 11.12), katman katman işleme, iki yönlü BFS, ızgara BFS'si (Örnek 9, 10).

## 4.2 BFS ile en kısa yol: ebeveyn dizisi ve yolun geri çıkarılması

**1 · Problem.** Ağırlıksız bir grafta s'den t'ye bir en kısa yolun **kendisi**. **2 · Fikir.** BFS bir köşeyi ilk gördüğünde kimin gördüğünü yazar: `ebeveyn[v] = u`. Ebeveyn bağları s köklü **BFS ağacını** oluşturur (Şekil 11.5, kalın); t'den ebeveynler izlenerek s'ye çıkılır ve dizi ters çevrilir: yolun **geri çıkarılması** (reconstruction). **3 · Elle izleme.** 7'den: 7 → 5 → 3 → 1 → 0; ters çevrilince 0 → 1 → 3 → 5 → 7 (Örnek 4'ün tablosunda 7'nin satırı: uzaklık 4, ebeveyn 5). Ulaşılamayan köşenin uzaklığı ve ebeveyni −1 kalır: bileşenli grafta 0'dan BFS'de 1, 2, 4, 5, 7, 9 köşeleri için tablo “ulaşılamaz (yol yok)” yazar.

SÖZDE KOD · 4 · Sözde kod: yolun ebeveyn zincirinden geri çıkarılması

```
algoritma YOL-ÇIKAR(ebeveyn, s, t)
    eğer t ≠ s ve ebeveyn[t] = −1 ise döndür YOK   ▷ ulaşılamaz
    yol ← boş;  v ← t
    iken v ≠ −1:  yol'un sonuna v ekle;  v ← ebeveyn[v]
    yol'u ters çevir;  döndür yol                  ▷ |yol| − 1 = uzak[t]
```

**5 · Doğruluk.** Her ağaç kenarında uzak[v] = uzak[ebeveyn[v]] + 1; t'den köke inen zincirin uzunluğu tam uzak[t]'dir, 4.1 gereği en kısadır. (Birden çok en kısa yol olabilir; artan komşu sırası hep aynısını seçer.) **6 · Analiz.** Geri çıkarma O(V); BFS ile Θ(V + E). **7 · C gerçekleştirimi.**

C **c/H11_04_bfs_en_kisa_yol.c · satır 221–236**

```
static int yol_cikar(const int ebeveyn[], int s, int t, int yol[])
{
    int k = 0;
    if (t != s && ebeveyn[t] == -1) {
        return -1;                      /* ulaşılamaz */
```

```
    }
    for (int v = t; v != -1; v = ebeveyn[v]) {
        yol[k++] = v;                    /* t, ebeveyn[t], ..., s */
    }
    for (int i = 0, j = k - 1; i < j; i++, j--) {
        int gecici = yol[i];             /* ters çevir: s .. t */
        yol[i] = yol[j];
        yol[j] = gecici;
    }
    return k - 1;
}
```

**8 · Üç dil.** C yolu çağıranın dizisine yazar, kenar sayısını döndürür; C++ `std::reverse`, Python `list.reverse()` kullanır:

C++ **H11_04_bfs_en_kisa_yol.cpp · satır 185–193**

```
    std::vector<int> yol;
    if (t != s && ebeveyn[t] == -1) {
        return yol;                      // ulaşılamaz
    }
    for (int v = t; v != -1; v = ebeveyn[v]) {
        yol.push_back(v);                // t, ebeveyn[t], ..., s
    }
    std::reverse(yol.begin(), yol.end());   // s .. t
    return yol;
```

Python **H11_04_bfs_en_kisa_yol.py · satır 133–143**

```
    """s'den t'ye yolu ebeveyn zincirinden geri çıkarır: [s, .., t];
    yol yoksa boş liste."""
    yol = []
    if t != s and ebeveyn[t] == -1:
        return yol                       # ulaşılamaz
    v = t
    while v != -1:
        yol.append(v)                    # t, ebeveyn[t], ..., s
        v = ebeveyn[v]
    yol.reverse()                        # s .. t
    return yol
```

**9 · Test.** Program her çalıştırmada bir **kâhin** çalıştırır (`dogrula`): ulaşılan her u'nun her komşusu v için uzak[v] ≤ uzak[u] + 1 ve her v ≠ s için uzak[v] = uzak[ebeveyn[v]] + 1; iki koşul uzaklıkları **sonuçtan** denetler (iki grafta da çıktı "geçti" der). **10 · Deney.** Örnek 13 aynı BFS'yi ortak yazarlık ağında kullanır (Bölüm 5.3). **11 · Varyantlar.** Ağırlıklı graflarda katman güvencesi bozulur: H12'de Dijkstra kuyruğu en küçük öbekle değiştirir; ızgarada ebeveyn hücre indisi tutar (Örnek 10).

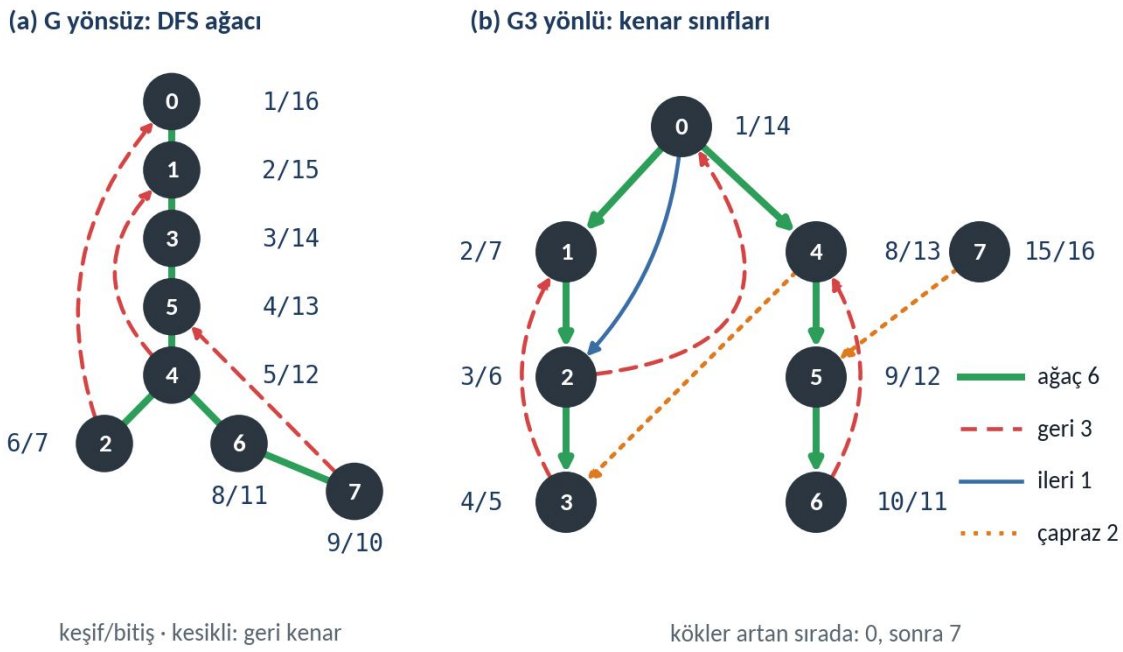
## 4.3 Derinlik öncelikli arama (DFS): üç renk, keşif ve bitiş

**1 · Problem.** Bütün köşeleri, bir daldan **olabildiğince derine inerek** dolaşmak ve yapıyı kaydetmek: hangi kenar döngü kapatır, hangi köşe hangisinin içinde biter? **2 · Fikir.** Labirentte çıkmaza kadar ilerleyip son kavşağa dönmek: H3'ün geri izlemesinin graf biçimi; özyineleme ya da **yığın** (LIFO) dönülecek yeri hatırlar. Köşeler **üç renkle** izlenir: **beyaz** (keşfedilmedi), **gri** (keşfedildi, komşuları

dolaşılıyor: özyineleme yığınında), **siyah** (bitti). Her köşeye bir **keşif** (griye dönüş) ve bir **bitiş** (siyaha dönüş) zamanı verilir; zaman her olayda bir artar (Cormen vd., 2022).

**3 · Elle izleme.** G'de DFS(0) (Örnek 5; Şekil 11.6a): 0 (keşif 1) → 1 (2) → 3 (3) → 5 (4) → 4 (5). 4'ün komşusu 1 gridir ve ebeveyni değildir: **geri kenar** 4–1. **Tahmin edin:** 4'ten sonra hangi köşeye inilir, 2'nin komşusu 0 ne tür bir kenardır? (Yanıt: 2'ye (6); 0 gridir: geri kenar 2–0; 2 biter (7).) Sonra 4 → 6 (8) → 7 (9); 7–5 geri kenardır; 7, 6, 4, 5, 3, 1, 0 sırayla 10…16'da biter. Keşif = [1 2 6 3 5 4 8 9], bitiş = [16 15 7 14 12 13 11 10] (köşe 0…7).

**Şekil 11.6**
*DFS ağaçları: (a) yönsüz G, keşif/bitiş zamanları ve geri kenarlar; (b) yönlü G3, dört kenar sınıfı (kökler artan sırada: 0, sonra 7)*




**SÖZDE KOD · 4 · Sözde kod: özyinelemeli DFS ve kenar sınıflaması**

```
algoritma DFS(G)                          ▷ kökler artan sırada: DFS ormanı
    her v için: renk[v] ← BEYAZ;  zaman ← 0
    her u için (artan): eğer renk[u] = BEYAZ ise ZİYARET(u)
algoritma ZİYARET(u)
    renk[u] ← GRİ;  keşif[u] ← ++zaman
    u'nun her v komşusu için (artan sırada)
        eğer renk[v] = BEYAZ ise  ebeveyn[v] ← u;  ZİYARET(v)   ▷ ağaç kenarı
        değilse eğer renk[v] = GRİ ise  geri kenar  ▷ yönsüzde ebeveyn hariç
        değilse  ileri (keşif[u] < keşif[v]) ya da çapraz kenar  ▷ yalnız yönlüde
    renk[u] ← SİYAH;  bitiş[u] ← ++zaman
```

**5 · Doğruluk. Gri köşe değişmezi:** gri köşeler her an kökten şimdiki köşeye giden yolu oluşturur (özyineleme yığını). **Parantez teoremi:** keşfi "(u", bitişi "u)" yazarsak iki köşenin aralıkları ya iç içedir ya ayrıktır. Sezgisel kanıt: v, u gri iken keşfedilirse v'nin çağrısı u'nunkinin **içinde** başlar ve yığın gereği önce biter: [keşif v, bitiş v] ⊂ [keşif u, bitiş u]; değilse biri bitmeden öbürü başlamaz. G'de parantez yapısı `(0 (1 (3 (5 (4 (2 2) (6 (7 7) 6) 4) 5) 3) 1) 0)`. **Kenar sınıfları:** u → v incelenirken v **beyazsa** ağaç kenarı; **griyse** v, u'nun atasıdır: **geri kenar**; **siyahsa** v'nin aralığı u'nunkinin içindeyse **ileri** (keşif[u] < keşif[v]), önce bitmişse **çapraz** kenar. Yönsüz grafta yalnız ağaç ve geri kenar olur: siyah bir v, u'dan

önce bitseydi u–v kenarını görüp u'yu kendi altında keşfederdi. G3'te kenar sayıları: ağaç 6, geri 3, ileri 1, çapraz 2 (toplam 12 = E).

**6 · Analiz.** Her köşe bir kez griye, bir kez siyaha döner, her liste bir kez dolaşılır: **Θ(V + E)**. Ek bellek O(V), ama özyineleme derinliği en kötü durumda V'dir (zincir; G'de en büyük derinlik 7): 4.4'ün konusu.

**Tablo 11.4**
*Derinlik öncelikli arama (DFS) kimlik kartı*

| ALGORİTMA KİMLİK KARTI | **Derinlik öncelikli arama (DFS)** *(depth-first search (DFS))* |
|---|---|
| **Strateji** | Derine in, çıkmazda dön; üç renk, keşif/bitiş zamanları |
| **Girdi önkoşulu** | Yönlü ya da yönsüz graf; komşular artan sırada |
| **Zaman (en iyi · ortalama · en kötü)** | Θ(V + E) · Θ(V + E) · Θ(V + E) |
| **Ek bellek** | O(V) |
| **Özellikler** | G3'te ağaç 6, geri 3, ileri 1, çapraz 2; özyineleme derinliği V'ye ulaşabilir |
| **Değişmez / doğruluk** | Gri köşeler kökten şimdiki köşeye yol; aralıklar iç içe ya da ayrık |
| **Kütüphane karşılığı** | C: elle · C++: Boost.Graph · Python: networkx (yalnız kâhin) |
| **Ne zaman kullanılır** | Erişilebilirlik, bileşenler, döngü, topolojik sıra, kenar sınıfları |
| **Ne zaman kullanılmaz** | En kısa yol (Hata Avcısı 4); derin graflarda özyinelemeli sürüm |

**7 · C gerçekleştirimi.** Örnek 5'in `dfs_ziyaret`'i; DFS durumu (renk, zamanlar, ebeveyn, kenar kaydı) bir yapıdadır:

c/H11_05_dfs_ozyinelemeli.c · satır 212–238

```c
static void dfs_ziyaret(const Graf *g, int u, Dfs *d)
{
    d->renk[u] = GRI;
    d->kesif[u] = ++d->zaman;
    d->sira[d->adim++] = u;
    d->olay[d->olay_sayisi++] = u;
    if (++d->derinlik > d->en_derin) {
        d->en_derin = d->derinlik;
    }
    for (size_t i = 0; i < g->komsu[u].boyut; i++) {
        int v = g->komsu[u].veri[i];
        if (d->renk[v] == BEYAZ) {
            kenar_kaydet(d, u, v, AGAC);
            d->ebeveyn[v] = u;
            dfs_ziyaret(g, v, d);
        } else if (g->yonlu) {
            kenar_kaydet(d, u, v, d->renk[v] == GRI ? GERI :
                                  d->kesif[u] < d->kesif[v] ? ILERI : CAPRAZ);
        } else if (d->renk[v] == GRI && v != d->ebeveyn[u]) {
            kenar_kaydet(d, u, v, GERI);  /* yönsüz: atasına kenar */
        }
    }
    d->derinlik--;
    d->renk[u] = SIYAH;
    d->bitis[u] = ++d->zaman;
```

```
    d->olay[d->olay_sayisi++] = -1 - u;
}
```

**8 · Üç dil.** Durum C'de `malloc`'lu dizileri olan bir yapı, C++'ta kurucusu dizileri dolduran bir `struct`, Python'da `__slots__`'lı bir sınıftır; Python'da `++` olmadığı için zaman iki adımda artırılır. Örnek 5'in `dfs_ziyaret`'inin komşu döngüsü ve bitişi:

C++ **H11_05_dfs_ozyinelemeli.cpp · satır 186–201**

```cpp
    for (int v : g.komsu[u]) {
        if (d.renk[v] == BEYAZ) {
            kenar_kaydet(d, u, v, AGAC);
            d.ebeveyn[v] = u;
            dfs_ziyaret(g, v, d);
        } else if (g.yonlu) {
            kenar_kaydet(d, u, v, d.renk[v] == GRI ? GERI :
                         d.kesif[u] < d.kesif[v] ? ILERI : CAPRAZ);
        } else if (d.renk[v] == GRI && v != d.ebeveyn[u]) {
            kenar_kaydet(d, u, v, GERI);  // yönsüz: atasına kenar
        }
    }
    d.derinlik--;
    d.renk[u] = SIYAH;
    d.bitis[u] = ++d.zaman;
    d.olay.push_back(-1 - u);
```

Python **H11_05_dfs_ozyinelemeli.py · satır 145–158**

```python
    for v in g.komsu[u]:
        if d.renk[v] == BEYAZ:
            kenar_kaydet(d, u, v, AGAC)
            d.ebeveyn[v] = u
            dfs_ziyaret(g, v, d)
        elif g.yonlu:
            kenar_kaydet(d, u, v, GERI if d.renk[v] == GRI else
                         ILERI if d.kesif[u] < d.kesif[v] else CAPRAZ)
        elif d.renk[v] == GRI and v != d.ebeveyn[u]:
            kenar_kaydet(d, u, v, GERI)     # yönsüz: atasına kenar
    d.derinlik -= 1
    d.renk[u] = SIYAH
    d.zaman += 1
    d.bitis[u] = d.zaman
```

**9 · Test.** Açık yığınlı sürüm aynı zamanları üretmelidir (Örnek 6); Kavram 4 sırayı networkx `dfs_preorder_nodes` ile karşılaştırır. **10 · Deney.** Çökme derinliği (Bölüm 6.2). **11 · Varyantlar.** Döngü tespiti, topolojik sıra (4.8), bileşenler (4.5); Tarjan (1972) bu yapısal bilgiyle kuvvetli bağlı ve iki bağlı bileşenleri doğrusal zamanda bulmuştur.

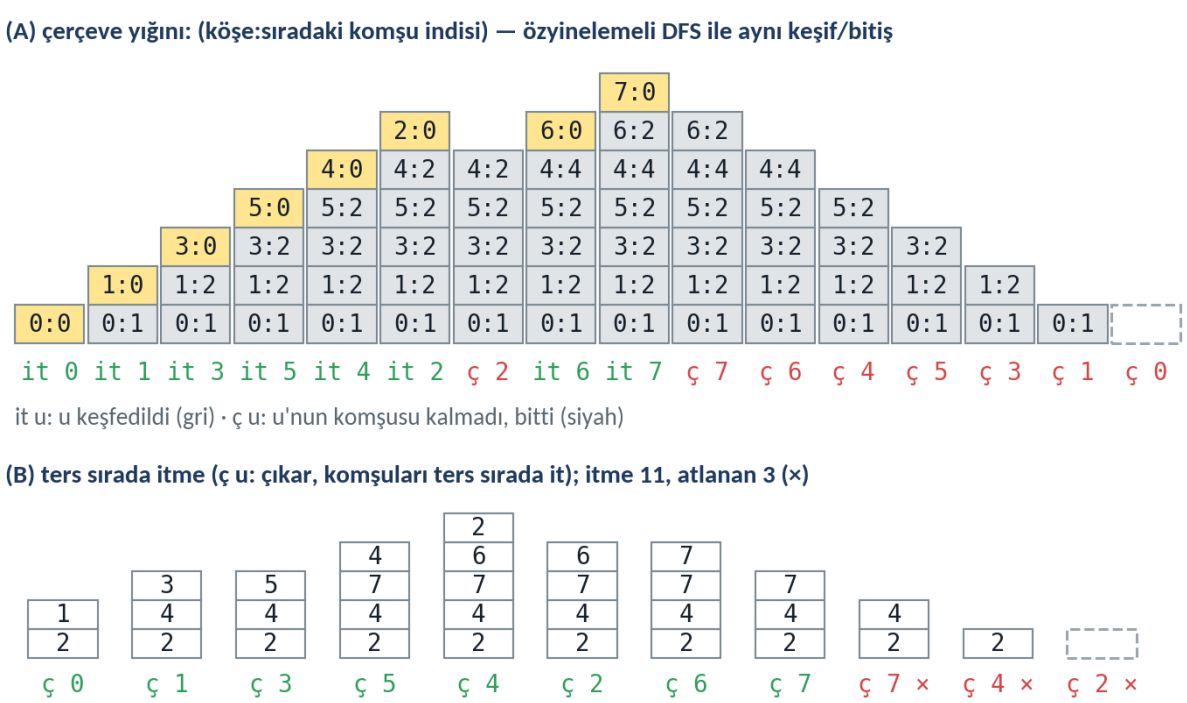
## 4.4 Açık yığınla DFS ve Kaputun Altında: derin özyinelemenin bedeli

**1 · Problem.** DFS'yi özyinelemesiz yazmak: özyinelemeli sürüm her köşe için bir çağrı çerçevesi açar, derin graflarda çağrı yığınını taşırır. **2 · Fikir.** Bir çağrının yerel durumu iki sayıdır: **köşe u** ve **sıradaki komşunun indisi i**. (A) **Çerçeve yığını:** tepedeki (u, i) u'nun i. komşusuyla sürer; komşu beyazsa onun

çerçevesi itilir; komşu kalmayınca u biter ve çerçeve çıkarılır: özyinelemenin **birebir** karşılığı. (B) **Ters sırada itme:** çıkan köşe ziyaret edilir, ziyaretsiz komşuları **ters** sırada itilir (en küçük komşu tepede); bir köşe yığına birden çok kez girebilir, ziyaretliyse çıkarken atlanır.

**3 · Elle izleme.** (A) G'de: `it 0` → [0:0], `it 1` → [0:1 1:0], … `it 2` → [0:1 1:2 3:2 5:2 4:2 2:0]; 2'nin komşuları bitince `çıkar 2 (7)`: bitiş 7, özyinelemeliyle aynı (Şekil 11.7; u:i = köşe:sıradaki komşu indisi). (B) 0 çıkar, 2, 1 itilir → [2 1]; 1 çıkar, 4, 3 itilir → [2 4 3]…; sonunda `yığına ekleme = 11`, `atlanan = 3`, sıra aynı. Artan sırada itilseydi `0 2 4 6 7 5 3 1` çıkardı.

**Şekil 11.7**
*Açık yığınlı DFS, G (Örnek 6): (A) çerçeve yığını, sarı = yeni itilen; (B) ters sırada itme, × = zaten ziyaretli, atlandı*




SÖZDE KOD · 4 · Sözde kod: açık yığınlı DFS (çerçeve yığını)

```
algoritma DFS-AÇIK(G, kök)                ▷ (A) çerçeve yığını
    renk[kök] ← GRİ;  keşif[kök] ← ++zaman;  İT(Y, (kök, 0))
    iken Y boş değil
        (u, i) ← TEPE(Y)                   ▷ başvuru: i burada ilerler
        eğer i < derece(u) ise
            v ← komşu(u, i);  i ← i + 1
            eğer renk[v] = BEYAZ ise
                renk[v] ← GRİ;  keşif[v] ← ++zaman;  İT(Y, (v, 0))
        değilse  renk[u] ← SİYAH;  bitiş[u] ← ++zaman;  ÇIKAR(Y)
```

**5 · Doğruluk.** Yığındaki çerçeveler, özyinelemeli sürümde o anda etkin çağrılarla (aynı u, aynı i) birebir aynıdır: her adım bir çağrının sonraki satırını yürütür. Zamanlar ve kenar sınıfları bu yüzden aynıdır (Örnek 6 sırayı, keşif ve bitiş zamanlarını özyinelemeli sürümle karşılaştırır: sıra evet, keşif evet, bitiş evet). **6 · Analiz.** (A) her köşe bir kez itilir, her komşu bir kez okunur: Θ(V + E), yığın ≤ V çerçeve. (B) her kenar en çok bir itme üretir: yığın O(E), süre yine Θ(V + E). **7 · C gerçekleştirimi.** (A) yönteminin döngüsü:

C `c/H11_06_dfs_acik_yigin.c · satır 294–308`

```c
    while (n > 0) {
        Cerceve *c = &y[n - 1];
        if (c->i < g->komsu[c->u].boyut) {
            int v = g->komsu[c->u].veri[c->i++];
            if (g->yonlu) {
                d->tur[d->tur_sayisi++] = sinif(d, c->u, v);
            }
            if (d->renk[v] == BEYAZ) {
                kesfet(d, v);
                y[n++] = (Cerceve){v, 0};
                if (iz) {
                    printf("   it %d          ", v);
                    cerceve_yaz(y, n);
                }
            }
```

Komşu kalmayınca (`else` dalı, satır 309–312) u siyaha boyanır, bitiş zamanı verilir ve çerçeve çıkarılır (`n--`); `iz` satırları yalnız izlemeyi yazdırır (Şekil 11.7).

**8 · Üç dil.** Yığın C'de dizi, C++'ta `std::vector`, Python'da listedir. C++'ta `y.back()` başvurusu `push_back`'ten sonra geçersiz olabilir (vektör taşınabilir): kod başvuruyu itmeden önce kullanır:

C++ `H11_06_dfs_acik_yigin.cpp · satır 227–241`

```cpp
    while (!y.empty()) {
        Cerceve& c = y.back();
        if (c.i < g.komsu[c.u].size()) {
            int v = g.komsu[c.u][c.i++];
            if (g.yonlu) {
                d.tur.push_back(sinif(d, c.u, v));
            }
            if (d.renk[v] == BEYAZ) {
                kesfet(d, v);
                y.push_back({v, 0});     // c bundan sonra geçersiz olabilir
                if (iz) {
                    std::cout << "   it " << v << "          ";
                    cerceve_yaz(y);
                }
            }
```

Python `H11_06_dfs_acik_yigin.py · satır 170–182`

```python
    while y:
        c = y[-1]
        u = c[0]
        if c[1] < len(g.komsu[u]):
            v = g.komsu[u][c[1]]
```

```
            c[1] += 1
            if g.yonlu:
                d.tur.append(sinif(d, u, v))
            if d.renk[v] == BEYAZ:
                kesfet(d, v)
                y.append([v, 0])
                if iz:
                    print(f"   it {v}          " + cerceve_metni(y))
```

**KAPUTUN ALTINDA · Derin DFS'de yığın taşması ve açık yığın çözümü**

Her özyinelemeli çağrı **çağrı yığınında** (stack) bir çerçeve kaplar (dönüş adresi, yazmaçlar, yerel değişkenler); Kavram 6 ardışık iki çağrının yerel değişken adresleri farkından çerçeve boyunu ölçer: bu derleyicide (-O0) 48 bayt. Linux'ta varsayılan yığın sınırı `ulimit -s` = 8192 KB: kestirim 8 388 608 / 48 ≈ 174 762 çağrı. Deneyde (Bölüm 6.2) n köşelik zincirde C'nin özyinelemeli DFS'si (-O0) 8 MiB'ta 174 061 derinliği bitirdi, 174 062'de SIGSEGV ile çöktü; 64 MiB'ta eşik 1 397 384 (≈ 8 kat); C++ -O2'de çerçeve büyüdü, eşik 104 432. Hata Avcısı 7 $10^6$ köşede: `(süreç SIGSEGV sinyaliyle sonlandı; kabukta çıkış kodu 139)`; Python'da sınır yorumlayıcının korumasıdır (`sys.getrecursionlimit()` = 1000, `RecursionError`). (Bu makinede, bu derleyici ve bayraklarla.)

Açık yığınlı DFS çerçeveleri **öbekte** (heap; `malloc`'lu dizi) tutar: derinlik yalnız bellekle sınırlıdır. C'nin açık yığınlı DFS'si $10^6$ köşelik zinciri 9,28 ms'de çökmeden bitirdi (Şekil 11.16). Python'da `sys.setrecursionlimit` sınırı büyütür ama C yığınını taşırıp yorumlayıcıyı çökertebilir (Kavram 5). Derinliği girdiye bağlı her özyineleme bir kararlılık riskidir (Bölüm 9).

**9 · Test.** Kâhin özyinelemeli DFS'dir: sıra, keşif, bitiş, yönlüde kenar sınıfları. **10 · Deney.** Bölüm 6.2. **11 · Varyantlar.** (B) daha kısadır ama doğru bitiş zamanı vermez; topolojik sıra ve kenar sınıfları gerekiyorsa (A) kullanılır.

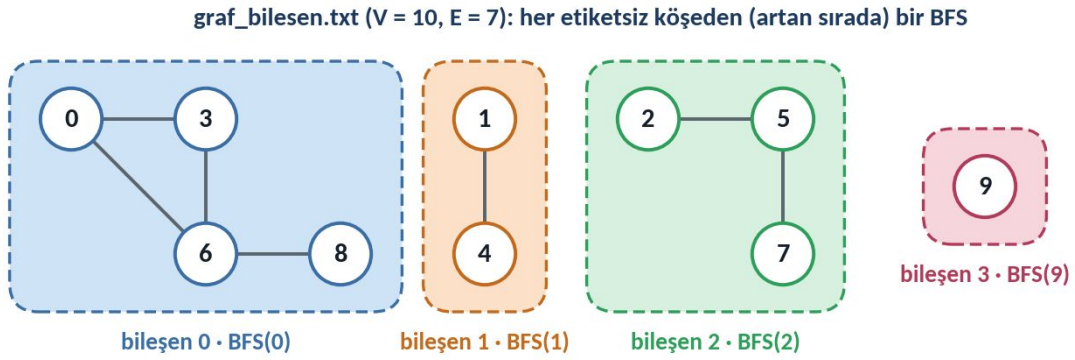
## 4.5 Bağlı bileşenler

**1 · Problem.** Yönsüz bir grafı **bağlı bileşenlerine** ayırmak. **2 · Fikir.** Köşeleri artan sırada tara; etiketsiz bir köşeden BFS (ya da DFS) başlat, ulaşılan her köşeye yeni bir numara ver. **3 · Elle izleme.** graf_bilesen.txt (V = 10, E = 7): BFS(0) → 0 = {0 3 6 8}; BFS(1) → 1 = {1 4}; BFS(2) → 2 = {2 5 7}; BFS(9) → 3 = {9} (Şekil 11.8).

**SÖZDE KOD · 4 · Sözde kod: bağlı bileşenler**

```
algoritma BİLEŞENLER(G)
    her v için: etiket[v] ← −1;  no ← 0
    her s için (artan): eğer etiket[s] = −1 ise
        BFS(G, s) ile ulaşılan her v için etiket[v] ← no;  no ← no + 1
    döndür no, etiket                ▷ aynı bileşen ⇔ etiket[u] = etiket[v]
```

**Şekil 11.8**
*Bağlı bileşenler (Örnek 7): her etiketsiz köşeden artan sırada bir BFS; dört bileşen*




```
etiket = [0 1 2 0 1 2 0 2 0 3] · bileşen sayısı = 4
```

**5 · Doğruluk.** Etiketi k olan her köşe k. dolaşmanın kökünden ulaşılabilirdir; BFS ulaşılabilen **bütün** köşeleri bulduğu için bileşen tamdır, başka bileşenden kenar gelemeyeceği için yanlış köşe etiketlenmez. **6 · Analiz.** Dolaşmaların toplamı **Θ(V + E)** (Örnek 7'nin sayaçları: BFS çağrısı = 4, kuyruğa ekleme = 10 (= V), kenar inceleme = 14 (= 2E)). Sonra "aynı bileşende mi?" tek karşılaştırmadır (Örnek 7: 0 ve 8: evet; 1 ve 9: hayır). Kenar sayısı ≥ köşe sayısı olan bileşende döngü vardır (ağaçta kenar = köşe − 1).

**Tablo 11.5**
*Bağlı bileşenler kimlik kartı*

| ALGORİTMA KİMLİK KARTI **Bağlı bileşenler** *(connected components)* | |
|---|---|
| **Strateji** | Her etiketsiz köşeden (artan) bir BFS/DFS; ulaşılanlara aynı numara |
| **Girdi önkoşulu** | Yönsüz graf (yönlüde: kuvvetli bağlı bileşenler) |
| **Zaman (en iyi · ortalama · en kötü)** | Θ(V + E) · Θ(V + E) · Θ(V + E) |
| **Ek bellek** | O(V) |
| **Özellikler** | Örnek 7: 4 bileşen; kenar ≥ köşe ise döngü; "aynı bileşende mi?" O(1) |
| **Değişmez / doğruluk** | Etiketi k olan her köşe k. dolaşmanın kökünden ulaşılabilir |
| **Kütüphane karşılığı** | C: elle · C++: Boost.Graph · Python: networkx (yalnız kâhin) |
| **Ne zaman kullanılır** | Parça sayısı, ada sayma, erişilebilirlik sorguları |
| **Ne zaman kullanılmaz** | Kenarlar tek tek eklenip sık sorgulanıyorsa (birleşim–bul) |

**7 · C gerçekleştirimi.** Bir bileşeni etiketleyen BFS; ebeveyn olmayan etiketli komşu (ağaç dışı kenar) döngünün kanıtıdır:

C `c/H11_07_bagli_bilesenler.c · satır 206–221`

```c
    while (bas < son) {
        int u = q[bas++];
        for (size_t i = 0; i < g->komsu[u].boyut; i++) {
            int v = g->komsu[u].veri[i];
            say->inceleme++;
```

```
            if (etiket[v] == -1) {
                etiket[v] = no;
                ebeveyn[v] = u;
                q[son++] = v;
                say->ekleme++;
            } else if (v != ebeveyn[u] && ilk_dongu[0] == -1) {
                ilk_dongu[0] = u;        /* ağaç dışı kenar: döngü */
                ilk_dongu[1] = v;
            }
        }
    }
```

**8 · Üç dil.** C bileşenin köşelerini çağıranın kuyruk dizisinde bırakır; C++ vektör döndürür ve kenarı `std::pair` başvurusuyla verir; Python ikisini demetle döndürür:

C++ **H11_07_bagli_bilesenler.cpp · satır 179–190**

```
        for (int v : g.komsu[u]) {
            say.inceleme++;
            if (etiket[v] == -1) {
                etiket[v] = no;
                ebeveyn[v] = u;
                q.push(v);
                say.ekleme++;
            } else if (v != ebeveyn[u] && ilk_dongu.first == -1) {
                ilk_dongu = {u, v};      // ağaç dışı kenar: döngü
            }
        }
    }
```

Python **H11_07_bagli_bilesenler.py · satır 135–143**

```
        for v in g.komsu[u]:
            say.inceleme += 1
            if etiket[v] == -1:
                etiket[v] = no
                ebeveyn[v] = u
                q.append(v)
                say.ekleme += 1
            elif v != ebeveyn[u] and ilk_dongu[0] == -1:
                ilk_dongu = (u, v)      # ağaç dışı kenar: döngü
```

**9 · Test.** networkx `connected_components` ile aynı (Kavram 4); bağlı G'de `bileşen sayısı = 1`. **10 · Deney.** Ada sayma aynı algoritmadır (Bölüm 5.1). **11 · Varyantlar.** Kenarlar tek tek eklenirken sorgu sıksa birleşim–bul (H12); yönlü graflarda **kuvvetli bağlı bileşenler** (Bölüm 13).

## 4.6 İki parçalılık: BFS ile 2-renklendirme

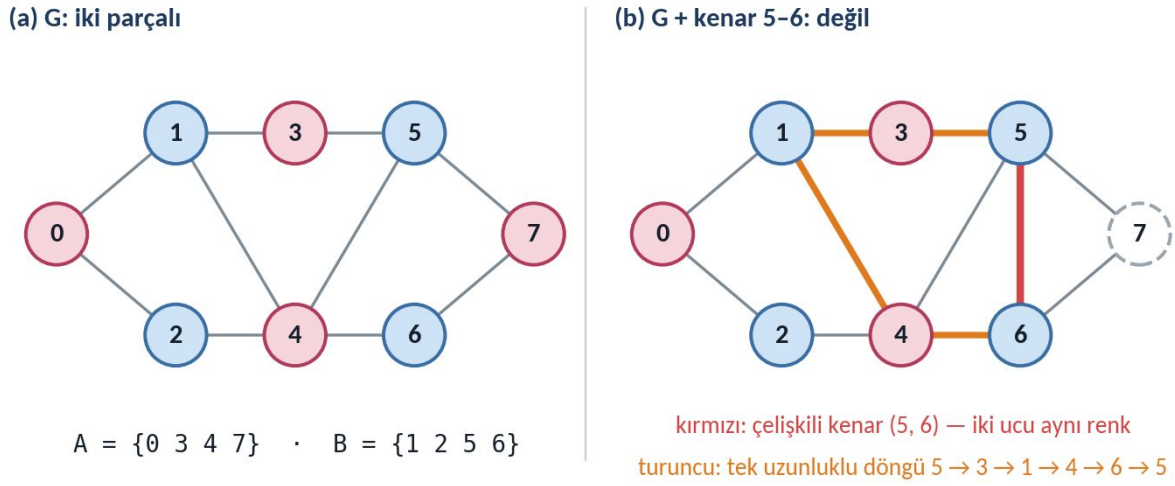
**1 · Problem.** Köşeler, her kenar bir A köşesini bir B köşesine bağlayacak biçimde iki kümeye ayrılabilir mi (**iki parçalı**, bipartite; öğrenci–ders, iş–işçi eşlemeleri)? **2 · Fikir.** BFS sırasında her köşeyi ebeveyninin **karşı rengine** boya; renkli bir komşu kendi rengindeyse çelişki vardır. Katmanlarla: renk = uzaklığın paritesi; aynı katmandaki iki köşeyi bağlayan kenar **tek uzunluklu bir döngü** demektir (Kleinberg & Tardos, 2006). **3 · Elle izleme.** G iki parçalıdır: A = {0 3 4 7}, B = {1 2 5 6}. 5–6 kenarı

eklenince 5 çıktığında komşusu 6 da renk 1'dedir (program: “çelişki: kenar (5, 6), iki ucu da renk 1”); döngü iki uçtan ortak ataya çıkan iki ağaç yolunun birleşimidir: 5 → 3 → 1 → 4 → 6 → 5 (uzunluk 5 = 2 + 2 + 1; ortak ata 1) (Şekil 11.9).

**SÖZDE KOD · 4 · Sözde kod: iki parçalılık sınaması (her bileşenden ayrı başlanır)**

```
algoritma İKİ-RENK(G, s)                    ▷ renk[v] = −1: boyanmadı
    renk[s] ← 0;  KUYRUĞA-EKLE(Q, s)
    iken Q boş değil
        u ← KUYRUKTAN-ÇIKAR(Q)
        u'nun her v komşusu için
            eğer renk[v] = −1 ise  renk[v] ← 1 − renk[u];  KUYRUĞA-EKLE(Q, v)
            değilse eğer renk[v] = renk[u] ise  döndür HAYIR (u, v)   ▷ çelişki
    döndür EVET                             ▷ A = {renk 0}, B = {renk 1}
```

**Şekil 11.9**
*İki parçalılık (Örnek 8): (a) G'nin 2-renklendirmesi; (b) 5–6 kenarı eklenince çelişki ve tek uzunluklu döngü 5 → 3 → 1 → 4 → 6 → 5*




**5 · Doğruluk.** renk[v] = uzak[v] mod 2. Bir kenarın iki ucu aynı renkteyse uçlardan ortak ataya çıkan a ve b uzunluklu yollar aynı paritededir ve döngünün uzunluğu a + b + 1 **tektir**; tek döngü 2 renkle boyanamaz (renkler döngü boyunca dönüşümlüdür), öyleyse HAYIR doğrudur. Çelişki yoksa her kenar farklı renkleri bağlar (Örnek 8'in denetimi: 10 kenarın 10'u de farklı renkli uçludur). **6 · Analiz.** BFS ile aynı: Θ(V + E). **7 · C gerçekleştirimi.** BFS döngüsü (izleme satırlarıyla):

**c/H11_08_iki_parcali.c · satır 202–224**

```
202     while (bas < son) {
203         int u = b->q[bas++];
204         printf("   çıkan %d (renk %d):", u, b->renk[u]);
205         for (size_t i = 0; i < g->komsu[u].boyut; i++) {
206             int v = g->komsu[u].veri[i];
207             if (b->renk[v] == -1) {
208                 b->renk[v] = 1 - b->renk[u];      /* karşı renk */
209                 b->uzak[v] = b->uzak[u] + 1;
210                 b->ebeveyn[v] = u;
211                 b->q[son++] = v;
212                 printf(" %d -> %d", v, b->renk[v]);
213             } else if (b->renk[v] == b->renk[u]) {
214                 printf(" %d ÇELİŞKİ\n", v);
```

```
                b->cu = u;
                b->cv = v;
                return 0;
            } else {
                printf(" %d uygun", v);
            }
            printf(i + 1 < g->komsu[u].boyut ? "," : "");
        }
        printf("\n");
    }
```

**8 · Üç dil.** Python sürümü izlemeyi bir parça listesinde toplar; döngü yazımında `ya + yb[-2::-1]` dilimi ikinci yolu ata hariç ters çevirip ekler (Ek A.8):

C++ **H11_08_iki_parcali.cpp · satır 173–185**

```
        for (std::size_t i = 0; i < g.komsu[u].size(); i++) {
            int v = g.komsu[u][i];
            if (b.renk[v] == -1) {
                b.renk[v] = 1 - b.renk[u];        // karşı renk
                b.uzak[v] = b.uzak[u] + 1;
                b.ebeveyn[v] = u;
                q.push(v);
                std::cout << ' ' << v << " -> " << b.renk[v];
            } else if (b.renk[v] == b.renk[u]) {
                std::cout << ' ' << v << " ÇELİŞKİ\n";
                b.cu = u;
                b.cv = v;
                return false;
```

Python **H11_08_iki_parcali.py · satır 133–145**

```
        for v in g.komsu[u]:
            if b.renk[v] == -1:
                b.renk[v] = 1 - b.renk[u]       # karşı renk
                b.uzak[v] = b.uzak[u] + 1
                b.ebeveyn[v] = u
                q.append(v)
                parcalar.append(f" {v} -> {b.renk[v]}")
            elif b.renk[v] == b.renk[u]:
                parcalar.append(f" {v} ÇELİŞKİ")
                print(f"   çıkan {u} (renk {b.renk[u]}):"
                      + ",".join(parcalar))
                b.cu, b.cv = u, v
                return False
```

**9 · Test.** networkx `is_bipartite` iki grafta da aynı yanıtı verir (Kavram 4). **10 · Deney.** Ayrı bir ölçüm yok: maliyet BFS'ninkidir (Bölüm 6.1). **11 · Varyantlar.** DFS ile de boyanabilir; çizelgeleme ve eşleme problemleri iki parçalı graflarda tanımlanır.

## 4.7 Topolojik sıralama I: Kahn algoritması

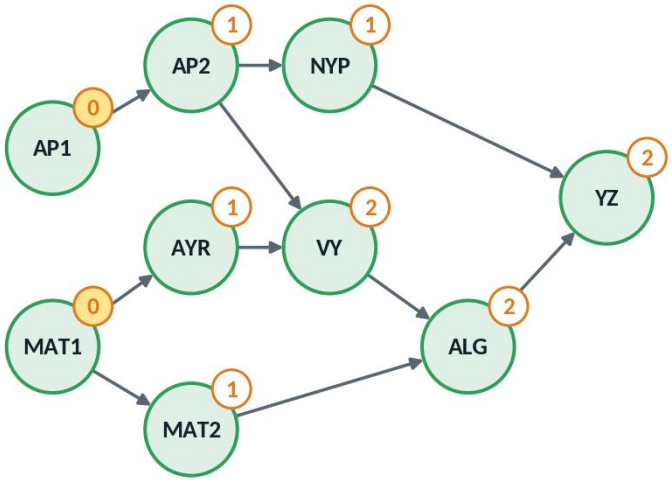
**1 · Problem.** Yönlü bir grafın köşelerini, her u → v kenarı için u, v'den **önce** gelecek biçimde sıralamak (**topolojik sıralama**, topological sort). Önkoşul grafında bu bir **ders planıdır**. **2 · Fikir.** Önkoşulu

kalmamış (giriş derecesi 0) bir dersi seç, plana yaz, ardıllarının giriş derecelerini bir azalt; derecesi 0'a inen ders "alınabilir" olur (Kahn, 1962). Birden çok aday varsa seçim kuralı sırayı belirler: Örnek 11 **en küçük numaralıyı** seçer (en küçük öbek, H7), böylece üç dil aynı planı verir.

**3 · Elle izleme.** Kurgusal ders DAG'ı (dersler_dag.txt; 0 ALG, 1 AP1, 2 AP2, 3 AYR, 4 MAT1, 5 MAT2, 6 NYP, 7 VY, 8 YZ): giriş dereceleri ALG 2, AP1 0, AP2 1, AYR 1, MAT1 0, MAT2 1, NYP 1, VY 2, YZ 2. Öbek {AP1, MAT1}; AP1 çıkar, AP2 1 → 0: {AP2, MAT1}; AP2 çıkar, NYP 1 → 0, VY 2 → 1: {MAT1, NYP}; MAT1 çıkar, AYR ve MAT2 0'a iner: {AYR, MAT2, NYP} (Şekil 11.10). **Tahmin edin:** sıradaki ders hangisi, VY'nin derecesi ne zaman 0 olur? (Yanıt: AYR (3); AYR çıkınca VY'nin iki önkoşulu da biter.) Sıra: 1 2 4 3 5 6 7 0 8 = AP1 AP2 MAT1 AYR MAT2 NYP VY ALG YZ.

**Şekil 11.10**
*Kahn algoritması (Örnek 11): ders DAG'ı ve başlangıç giriş dereceleri (rozet); adım adım çıkan ders ve en küçük öbek*

**Ders DAG'ı (dersler_dag.txt)**




turuncu rozet: başlangıçtaki giriş derecesi

`sıra: AP1 AP2 MAT1 AYR MAT2 NYP VY ALG YZ`

**Kahn: en küçük öbekten çıkar**

| adım | çıkan | öbek (sonra) |
|---|---|---|
| 0 | — | AP1 MAT1 |
| 1 | AP1 | AP2 MAT1 |
| 2 | AP2 | MAT1 NYP |
| 3 | MAT1 | AYR MAT2 NYP |
| 4 | AYR | MAT2 NYP VY |
| 5 | MAT2 | NYP VY |
| 6 | NYP | VY |
| 7 | VY | ALG |
| 8 | ALG | YZ |
| 9 | YZ | boş |

SÖZDE KOD · 4 · Sözde kod: Kahn algoritması (en erken dönem ile)

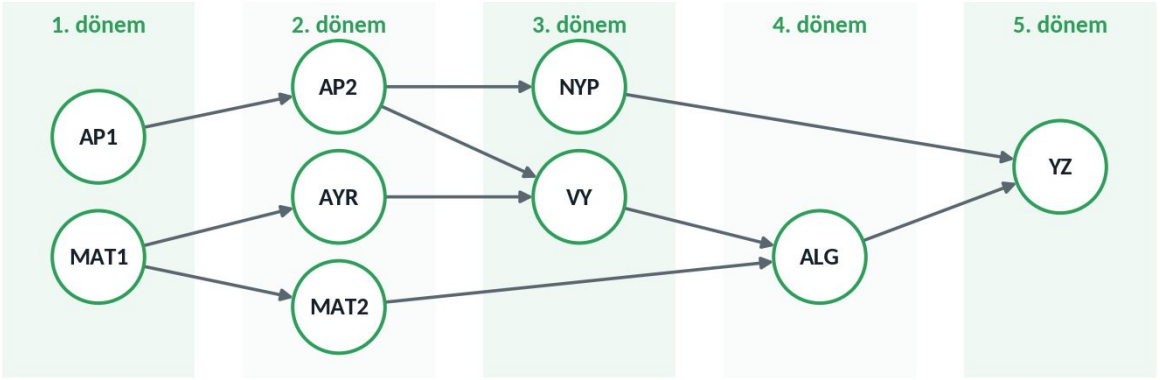
```
algoritma KAHN(G)
    her v için: derece[v] ← giriş derecesi;  dönem[v] ← 1
    H ← { v : derece[v] = 0 }              ▷ en küçük öbek (ya da FIFO kuyruk)
    iken H boş değil
        u ← EN-KÜÇÜĞÜ-ÇIKAR(H);  sıra'ya u ekle
        u'nun her v ardılı için
            dönem[v] ← max(dönem[v], dönem[u] + 1)
            derece[v] ← derece[v] − 1;  eğer derece[v] = 0 ise EKLE(H, v)
    eğer |sıra| < V ise döndür DÖNGÜ VAR   ▷ kalan köşeler sıralanamaz
    döndür sıra, dönem
```

**5 · Doğruluk. Varlık koşulu: topolojik sıra ancak ve ancak graf bir DAG ise vardır.** Döngü varsa sıra olamaz: döngünün her kenarı sırada ileri gitmeli, ama döngü başladığı köşeye döner. Tersine, her DAG'da giriş derecesi 0 olan bir köşe vardır: yoksa her köşeden geriye bir önkoşul kenarıyla yürüyebilir, V + 1 adımda bir köşeyi tekrarlar, döngü bulurduk. O köşeyi yazıp silmek kalan grafı DAG bırakır; tümevarımla Kahn bütün köşeleri sıralar. Değişmez: **sıraya giren her köşenin bütün önkoşulları ondan önce girmiştir.** Döngüdeki köşelerin derecesi hiç 0'a inmez: |sıra| < V döngünün kanıtıdır: döngülü

katalogda (`dersler_dongulu.txt`) 4 / 9 köşe sıralanır: 1 4 3 5; giriş derecesi 0'a inmeyenler (köşe:kalan derece) 0:1 2:1 6:1 7:1 8:2.

**En erken dönem.** Önkoşulsuz ders 1. dönemde, öbürleri bütün önkoşullarından sonra: dönem(v) = 1 + max{dönem(u) : u → v}; u çıktığında dönem(u) kesindir ve ardıllarına iletilir (dönem 1: 1 4; dönem 2: 2 3 5; dönem 3: 6 7; dönem 4: 0; dönem 5: 8). Bu v'ye giden **en uzun** önkoşul zinciridir; BFS katmanı (en **kısa** zincir) daha küçük olabilir: Mini-OBS 2 v2.10'un kavramsal düğümü (Bölüm 10). **6 · Analiz.** Dereceleri saymak Θ(V + E); her köşe bir kez öbeğe girip çıkar (O(log V)), her kenar bir kez azaltılır: en küçük öbekle **O(V log V + E)**, FIFO kuyrukla **Θ(V + E)**. Kural değişince plan değişir ama geçerli kalır: FIFO kuyrukla Kahn 1 4 2 3 5 6 7 0 8 verir (geçerli, ama en küçük öbek sırasından farklı); bu küçük DAG'ın bile 108 geçerli topolojik sırası vardır (Örnek 11 geri izlemeyle sayar, H3; Şekil 11.11).

**Şekil 11.11**
*Ders önkoşul DAG'ı en erken dönemlere göre (sütun = dönem) ve üç geçerli topolojik sıra: en küçük öbekli Kahn, FIFO'lu Kahn, DFS bitişinin tersi*




Kahn, en küçük öbek: AP1 → AP2 → MAT1 → AYR → MAT2 → NYP → VY → ALG → YZ

Kahn, FIFO kuyruk: AP1 → MAT1 → AP2 → AYR → MAT2 → NYP → VY → ALG → YZ

DFS bitişinin tersi: MAT1 → MAT2 → AYR → AP1 → AP2 → VY → NYP → ALG → YZ

**Bu DAG'ın geçerli topolojik sıra sayısı: 108 (üçü yukarıda)**

**Tablo 11.6**
*Topolojik sıralama (Kahn / DFS) kimlik kartı*

| ALGORİTMA KİMLİK KARTI **Topolojik sıralama (Kahn / DFS)** *(topological sort (Kahn's algorithm / DFS))* | |
|---|---|
| **Strateji** | Kahn: derece 0'ı çıkar, ardılları azalt · DFS: bitiş sırasının tersi |
| **Girdi önkoşulu** | Yönlü graf; sıra yalnız DAG'da, döngü raporlanır |
| **Zaman (en iyi · ortalama · en kötü)** | Θ(V + E) · Θ(V + E) · Θ(V + E) |
| **Ek bellek** | O(V) |
| **Özellikler** | Ders DAG'ında 108 geçerli sıra; en küçük öbekle O(V log V + E), FIFO ile Θ(V + E) |
| **Değişmez / doğruluk** | Kahn: önkoşullar önce çıkar · DFS: u → v için bitiş[v] < bitiş[u] |
| **Kütüphane karşılığı** | C: elle · C++: Boost.Graph · Python: graphlib (3.9+), networkx (kâhin) |
| **Ne zaman kullanılır** | Önkoşullu planlama, derleme sırası, DAG üzerinde DP (H13) |
| **Ne zaman kullanılmaz** | Döngü denetlenmiyorsa (Hata Avcısı 5); yönsüz graflar |

**7 · C gerçekleştirimi.** `kahn` öbeği elle tutar (H7), `fifo` FIFO'yu seçer. Ana döngü (izleme satırlarıyla):

C `c/H11_11_kahn.c · satır 291–314`

```c
    while (bas < h.n) {
        int u = fifo ? h.a[bas++] : obek_cikar(&h);
        sira[n++] = u;
        if (iz) {
            printf("   adım %d: çıkan %d (dönem %d);", n, u, donem[u]);
        }
        for (size_t i = 0; i < g->komsu[u].boyut; i++) {
            int v = g->komsu[u].veri[i];
            if (donem[u] + 1 > donem[v]) {
                donem[v] = donem[u] + 1;     /* en uzun önkoşul zinciri */
            }
            derece[v]--;
            if (iz) {
                printf("%s %d: %d->%d%s", i ? "," : "", v, derece[v] + 1,
                       derece[v], derece[v] == 0 ? " (öbeğe)" : "");
            }
            if (derece[v] == 0) {
                if (fifo) {
                    h.a[h.n++] = v;
                } else {
                    obek_ekle(&h, v);
                }
            }
        }
```

**8 · Üç dil.** En küçük öbek C'de elle, C++'ta `std::priority_queue<int, std::vector<int>, std::greater<int>>` (varsayılanı en **büyük** öbektir: Alıştırma 11.14), Python'da `heapq`:

C++ `H11_11_kahn.cpp · satır 220–236`

```cpp
        for (std::size_t i = 0; i < g.komsu[u].size(); i++) {
            int v = g.komsu[u][i];
            donem[v] = std::max(donem[v], donem[u] + 1);  // en uzun zincir
            derece[v]--;
            if (iz) {
                std::cout << (i ? "," : "") << ' ' << v << ": "
                          << derece[v] + 1 << "->" << derece[v]
                          << (derece[v] == 0 ? " (öbeğe)" : "");
            }
            if (derece[v] == 0) {
                if (fifo) {
                    q.push(v);
                } else {
                    h.push(v);
                }
            }
        }
```

Python `H11_11_kahn.py · satır 146–155`

```python
        for v in g.komsu[u]:
            donem[v] = max(donem[v], donem[u] + 1)  # en uzun zincir
```

```
            derece[v] -= 1
            parcalar.append(f" {v}: {derece[v] + 1}->{derece[v]}"
                            + (" (öbeğe)" if derece[v] == 0 else ""))
            if derece[v] == 0:
                if fifo:
                    q.append(v)
                else:
                    heapq.heappush(h, v)
```

**9 · Test.** Kâhin: her u → v için konum[u] < konum[v] (10 kenarın hepsinde sağlanır); networkx `lexicographical_topological_sort` aynı sırayı verir, `all_topological_sorts` 108 sıra sayar (Kavram 4). **10 · Deney.** Mini-OBS 2 v2.10 öbekli Kahn'ı $10^6$ köşelik DAG'da ölçer (Bölüm 10). **11 · Varyantlar.** FIFO kuyruk, dönem başına ders sınırıyla planlama (Atölye 3), DFS ile sıra (4.8).

## 4.8 Topolojik sıralama II: DFS bitiş sırası ve yönlü döngü tespiti

**1 · Problem.** Aynı sıra DFS ile; döngü varsa döngünün **kendisi** (hangi önkoşul yanlış girildi?). **2 · Fikir.** DFS'de bir köşe bütün ardılları bittikten sonra biter: köşeleri **bitiş sırasıyla** listeleyip **ters çevirirsek** her köşe ardıllarından önce gelir. İncelenen kenar **gri** bir köşeye gidiyorsa geri kenardır; gri köşeler kökten şimdiki köşeye giden yol olduğu için döngü bu yolun bir parçasıdır. **3 · Elle izleme.** Ders DAG'ında kökler artan sırada: `(0 (8 8) 0) (1 (2 (6 6) (7 7) 2) 1) (3 3) (4 (5 5) 4)`; bitiş sırası YZ ALG NYP VY AP2 AP1 AYR MAT2 MAT1, tersi MAT1 MAT2 AYR AP1 AP2 VY NYP ALG YZ (Kahn'ınkinden farklı ama geçerli). Hatalı önkoşul ALG → AP2 eklenince: (0 (2 (6 (8 8) 6) (7 … VY gri iken komşusu ALG da gridir: o anda gri yol (özyineleme yığını) ALG AP2 VY, döngü ALG → AP2 → VY → ALG (Şekil 11.12).

SÖZDE KOD · 4 · Sözde kod: DFS ile topolojik sıra ve döngü tespiti

```
algoritma DFS-TOPO(u)                    ▷ gri_yol: özyineleme yığınındaki köşeler
    renk[u] ← GRİ;  gri_yol'a u ekle
    u'nun her v ardılı için
        eğer renk[v] = GRİ ise  döndür DÖNGÜ (gri_yol'da v'den u'ya, + v)
        eğer renk[v] = BEYAZ ise  DFS-TOPO(v)        ▷ döngü bulunduysa yay
    gri_yol'dan u'yu çıkar;  renk[u] ← SİYAH;  bitiş_sırası'na u ekle
▷ sıra = bitiş_sırası'nın tersi (kökler artan sırada beyazsa DFS-TOPO)
```

**5 · Doğruluk.** Her u → v kenarı için **bitiş[v] < bitiş[u]**: v beyazsa u'nun altında keşfedilip önce biter (parantez teoremi), siyahsa zaten bitmiştir; gri olamaz, çünkü gri v u'nun atasıdır ve u → v ile döngü kapanırdı. Tersine çevrilmiş bitiş sırasında v bu yüzden u'dan sonradır. Gri komşu görülürse gri yolun o köşeden sonraki parçası döngüdür. İki renkle (yalnız "ziyaret edildi mi?") geri kenar siyah köşeye giden kenardan ayrılamaz: Hata Avcısı 5'in Python sürümü döngülü grafta önkoşul çiğneyen bir "plan" üretir. **6 · Analiz. Θ(V + E)**; derinlik en uzun önkoşul zinciri kadardır. **7 · C gerçekleştirimi.**

C c/H11_12_dfs_topolojik.c · satır 198–221

```
198 static int dfs_topo(const Graf *g, int u, Topo *t)
199 {
200     t->renk[u] = GRI;
201     t->kesif[u] = ++t->zaman;
202     t->gri_yol[t->ng++] = u;
203     printf(" (%d", u);
204     for (size_t i = 0; i < g->komsu[u].boyut; i++) {
205         int v = g->komsu[u].veri[i];
206         if (t->renk[v] == GRI) {
```

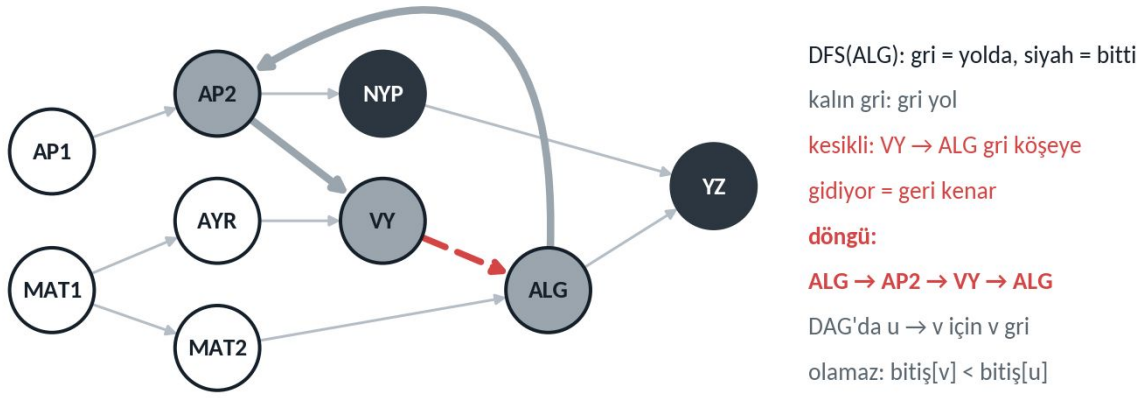
```
            t->dongu_u = u;              /* geri kenar: döngü */
            t->dongu_v = v;
            return 1;
        }
        if (t->renk[v] == BEYAZ && dfs_topo(g, v, t)) {
            return 1;
        }
    }
    t->ng--;                             /* u gri yoldan çıkar */
    t->renk[u] = SIYAH;
    t->bitis[u] = ++t->zaman;
    t->bitis_sirasi[t->nb++] = u;        /* bütün ardılları bitti */
    printf(" %d)", u);
    return 0;
}
```

**Şekil 11.12**
*DFS ile topolojik sıra (Örnek 12): (a) bitiş sırası ve tersi; (b) hatalı önkoşul ALG → AP2 eklenince gri yol ve geri kenar VY → ALG*

**(a) DAG: DFS bitiş sırası ve tersi**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| bitiş (siyaha dönüş): | YZ | ALG | NYP | VY | AP2 | AP1 | AYR | MAT2 | MAT1 |
| tersi = topolojik sıra: | MAT1 | MAT2 | AYR | AP1 | AP2 | VY | NYP | ALG | YZ |

**(b) Hatalı önkoşul ALG → AP2 eklenince**




**8 · Üç dil.** Gri yol C'de dizi + sayaç, C++'ta vektör, Python'da listedir; bitiş sırasının tersi C'de indisle, C++'ta `rbegin`/`rend`, Python'da `[::-1]` ile alınır (Ek A.12):

C++ **H11_12_dfs_topolojik.cpp · satır 170–183**

```
    for (int v : g.komsu[u]) {
        if (t.renk[v] == GRI) {
            t.dongu_u = u;               // geri kenar: döngü
            t.dongu_v = v;
            return true;
        }
        if (t.renk[v] == BEYAZ && dfs_topo(g, v, t)) {
            return true;
```

```
        }
    }
    t.gri_yol.pop_back();                // u gri yoldan çıkar
    t.renk[u] = SIYAH;
    t.bitis[u] = ++t.zaman;
    t.bitis_sirasi.push_back(u);         // bütün ardılları bitti
```

Python **H11_12_dfs_topolojik.py · satır 135–145**

```python
    for v in g.komsu[u]:
        if t.renk[v] == GRI:
            t.dongu_u, t.dongu_v = u, v     # geri kenar: döngü
            return True
        if t.renk[v] == BEYAZ and dfs_topo(g, v, t):
            return True
    t.gri_yol.pop()                      # u gri yoldan çıkar
    t.renk[u] = SIYAH
    t.zaman += 1
    t.bitis[u] = t.zaman
    t.bitis_sirasi.append(u)             # bütün ardılları bitti
```

**9 • Test.** Kâhin: her u → v için konum[u] < konum[v] (`dersler_dag.txt`'te 10 kenarın hepsinde sağlanır); networkx `find_cycle` aynı döngüyü bulur (Kavram 4). **10 • Deney.** Mini-OBS 2 v2.10 aynı DFS'yi açık yığınla $10^6$ derslik zincirde çalıştırır. **11 • Varyantlar.** Bütün döngüler için kuvvetli bağlı bileşenler (Tarjan, 1972); DAG üzerinde dinamik programlama (H13: en uzun yol, yol sayısı).

# 5. Çözümlü Örnekler

Örnek merdiveninin on üç basamağının onu Bölüm 3 ve 4'te işlendi (Örnek 1–2: Bölüm 3.3–3.4; Örnek 3–8: 4.1–4.6; Örnek 11–12: 4.7–4.8); bu bölüm ızgara örneklerini (9, 10) ve gerçeğe yakın bir ağı (13) çözer. Bütün örneklerin C++ ve Python sürümleri Ek A'dadır; çıktıları üç dilde bayt bayt aynıdır.

## 5.1 Izgarada taşkın doldurma ve ada sayma (Örnek 9)

**Problem.** 8 × 14'lük bir haritada '#' kara, '.' su hücresidir. (0, 0)'daki sudan başlayan bir taşkın hangi su hücrelerine ulaşır, kaç dalgada? Kaç kara parçası (ada) vardır? **Model.** Izgara örtük bir graftır: hücre (i, j) köşedir, indisi i·Y + j'dir (AP-1 H11'in satır öncelikli düzeni); komşular 4-komşulukta dört, 8-komşulukta sekiz yönde indis aritmetiğiyle bulunur. Taşkın doldurma (flood fill) bu grafta bir BFS'dir: "ziyaret işareti" hücrenin karakterini değiştirmektir ('.' → '~'). **Kritik kural:** komşunun ızgarada olup olmadığı **erişmeden önce** denetlenir (`icinde`): önce okuyup sonra denetlemek dizi dışına erişimdir (Hata Avcısı 3).

C **c/H11_09_tasma_doldurma.c · satır 45–62**

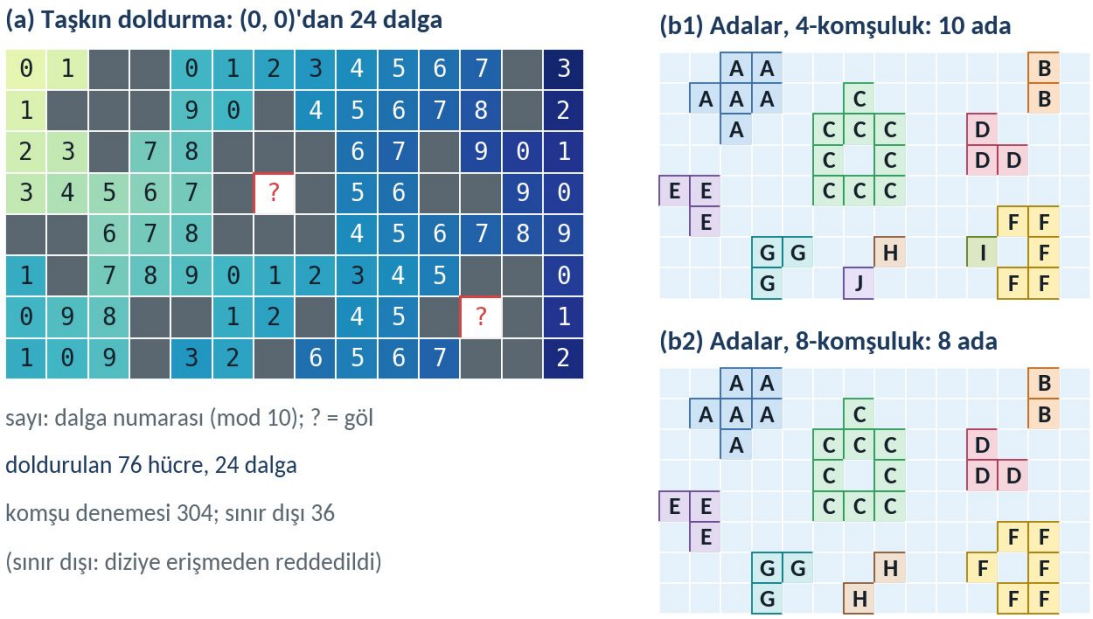
```c
    while (bas < son) {
        int h = q[bas++], i = h / Y, j = h % Y;
        for (int k = 0; k < yon; k++) {
            int ni = i + di[k], nj = j + dj[k];
            deneme++;
            if (!icinde(ni, nj)) {
                sinir_disi++;              /* erişilmeden reddedildi */
                continue;
            }
            if (iz[ni * Y + nj] == eski) {
```

```
                iz[ni * Y + nj] = yeni;
                q[son++] = ni * Y + nj;
                if (katman != NULL) {
                    katman[ni * Y + nj] = katman[h] + 1;
                }
            }
        }
    }
```

**Sonuç.** Taşkın 76 su hücresini 24 dalgada doldurur (en uzak hücre 23 adımda); her hücre 4 komşu dener ($304 = 4 \cdot 76$), bunların 36'sı ızgara dışındadır ve **erişilmeden** reddedilir; çevresi kara olan 2 su hücresine (göl) ulaşılamaz (Şekil 11.13a). Dalga boyutları 1, 2, 1, 2, 1, 1, 2, 4 … BFS katmanlarıdır. **Ada sayma** bağlı bileşen sayımıdır (Bölüm 4.5): her yeni kara hücresinden bir taşkın başlatılır. Komşuluk tanımı sonucu değiştirir: 4-komşulukta 10, 8-komşulukta 8 ada (çapraz değen F ile I ve H ile J birleşir; Şekil 11.13b).

**Karmaşıklık.** S × Y ızgarada her hücre en çok bir kez kuyruğa girer ve 4 (ya da 8) komşu dener: $\Theta(S \cdot Y)$; ada sayma da $\Theta(S \cdot Y)$. Çizim programlarının "boya kovası" aynı algoritmadır.

**Şekil 11.13**
*Örtük graf (Örnek 9): (a) (0, 0)'dan taşkın doldurmanın dalgaları (mod 10; ? = göl); (b) ada sayma, 4-komşulukta ve 8-komşulukta*




## 5.2 Labirentte en kısa yol: BFS ve H3'ün geri izlemesi (Örnek 10)

**Problem.** H3 Örnek 11'in 11 × 17 labirentinde S'den E'ye en kısa yol. **Yöntem.** BFS hücreleri kuyruğa eklerken `ebeveyn`'e kaydeder (−2: görülmedi, −1: kaynak); E kuyruktan çıkınca durur ve yolu ebeveyn zincirinden işaretler. Aynı dosyadaki `yol_bul` H3'ün geri izleme fonksiyonudur (H3 Örnek 11): hücreyi yola ekler ('*'), dört yönü sırayla dener, çıkmazda işareti 'x' yapıp geri döner. BFS döngüsü:

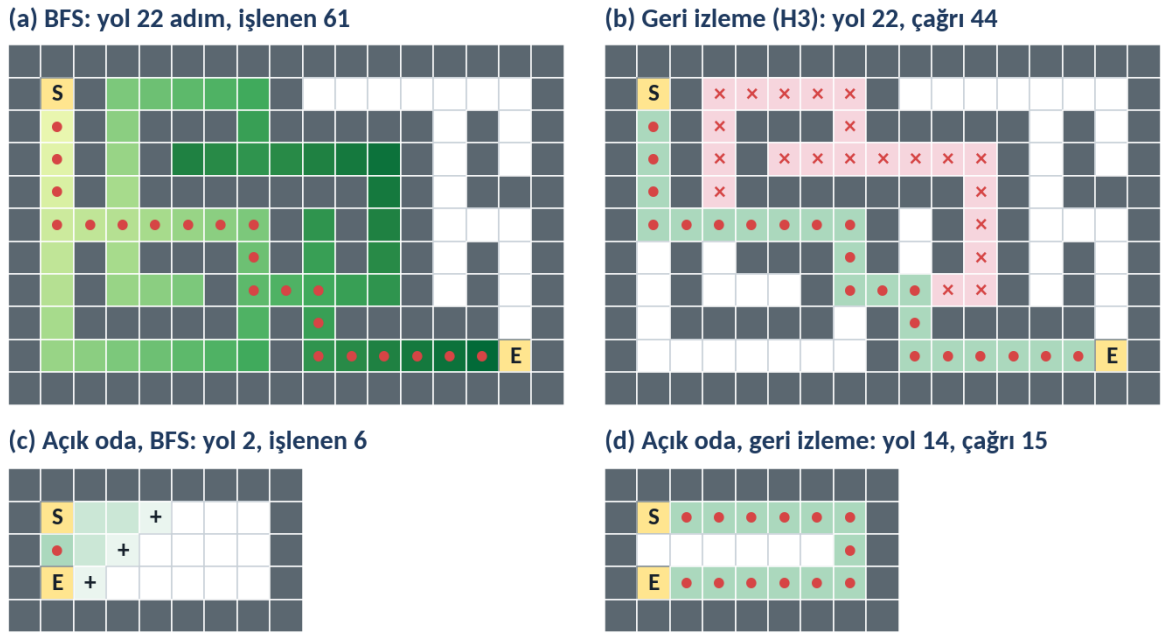
`c/H11_10_labirent_bfs.c · satır 68–80`

```
    while (bas < son) {
        int h = q[bas++], r = h / L->Y, c = h % L->Y;
        if (h == hedef) {
            break;                          /* E kuyruktan çıktı */
```

```
72          }
73          for (int d = 0; d < 4; d++) {
74              int yr = r + DR[d], yc = c + DC[d];
75              if (acik_mi(L, yr, yc) && ebeveyn[yr * L->Y + yc] == -2) {
76                  ebeveyn[yr * L->Y + yc] = h;
77                  q[son++] = yr * L->Y + yc;
78              }
79          }
80      }
```

**Neden?** H3'ün geri izlemesi, işaretlediği hücreyi ('x') bir daha denemez: bu, ızgara grafında **DFS**'nin ta kendisidir (her hücre en çok bir kez çağrılır; 44 çağrı ≤ 81 açık hücre). DFS bir dalda olabildiğince derine iner ve **ilk** bulduğu yolda durur: yol uzunluğu yön sırasına bağlıdır. H3 labirentinde koridorlar dar olduğu için ilk yol tesadüfen en kısadır; açık odada sağa giden ilk yön 14 adımlık bir dolambaç üretir. BFS ise E'yi ancak bütün daha yakın hücreleri işledikten sonra görür: **her zaman** en kısa yolu verir, karşılığında daha çok hücre işleyebilir (labirentte 61 ↔ 44). Lee'nin (1961) devre kartı yönlendiricisi bu dalga yayılımıdır. Atölye 2 (Bölüm 11.9) üç yöntemi yarıştırır (Şekil 11.14).

**Şekil 11.14**
*Labirent (Örnek 10): (a) BFS katmanları ve 22 adımlık yol; (b) H3 geri izlemesi (× geri dönülen hücre); (c–d) açık odada BFS 2, geri izleme 14 adım*




renk: BFS katmanı (koyu = uzak) · ● yol · × çıkmazdan geri dönülen · + kuyrukta bekleyen
BFS en kısa yolu her zaman bulur; geri izleme (DFS) ilk bulduğu yolda durur

## 5.3 “Kaç adım uzakta?”: ortak yazarlık ağı ve Erdős sayısı (Örnek 13)

**Problem.** 16 makalelik **kurgusal** bir yayın listesinden (`ortak_yazarlik.txt`; adlar uydurmadır) ortak yazarlık ağı kurmak: aynı makalede yer alan iki yazar arasında kenar vardır. Erdem'den her yazara kaç ortak yazarlık adımı vardır? **Yöntem.** Adlar sıralı bir diziye konur ve ad → köşe numarası **ikili aramayla** bulunur (H4); kenarlar komşuluk listesine eklenir, yinelenen çiftler atılır; ardından Erdem'den BFS.

Sonuç: en uzaktaki Lale'dir, 5 adım: Erdem → Ada → Deniz → Hakan → Jale → Lale; 15 / 19 yazara ulaşılır, 36 kenar ucu incelenir (ulaşılan köşelerin dereceleri toplamı). Adlar **bayt** sırasıyla sıralanır (C

`strcmp`, C++ `std::string::compare`, Python `bytes`): "Ömer" ve "İlker" UTF-8'de çok baytlı harflerle başladığı için ASCII adlardan sonra gelir: Türkçe alfabe sırası değildir, ama üç dilde aynıdır (Bölüm 8). Programın ikinci bölümü ortak üreteçle (tohum 2026) 2000 yazarlı, 3000 makaleli yapay bir ağ kurar; merkezden uzaklık dağılımı `0:1 1:581 2:1222 3:155 4:1 ∞:40`.

> **GERÇEK DÜNYADAN · Erdős sayısı: ortak yazarlık grafında BFS**
>
> Matematikçi Paul Erdős (1913–1996) çok sayıda ortak yazarla çalıştı. Bir araştırmacının **Erdős sayısı**, ortak yazarlık grafında Erdős'e olan BFS uzaklığıdır: Erdős'ün kendisi 0, onunla ortak makalesi olanlar 1, onların ortak yazarları 2... Oakland Üniversitesi'nden Jerry Grossman'ın yürüttüğü Erdős Sayısı Projesi, Mathematical Reviews'un Temmuz 2004 verisine göre dağılımı yayımlar: 0 sayılı 1 kişi, 1 sayılı 504, 2 sayılı 6 593, 3 sayılı 33 605, 4 sayılı 83 642, 5 sayılı 87 760 kişi; en büyük sonlu değer 13. Sonlu Erdős sayısı olan ≈ 268 000 kişinin ortanca değeri 5, ortalaması 4,65'tir (Grossman, 2025). Sonsuz olanlar Erdős'ün bileşeninde olmayanlardır; bu kadar büyük bir ağda uzaklıkların küçüklüğü "küçük dünya" olgusudur (Bölüm 13).

# 6. Haftanın Deneyi: Temsil Seçimi ve Derin Özyineleme

**Sorular.** (a) Seyrek ve yoğun graflarda komşuluk matrisi, komşuluk listesi ve CSR ile BFS ne kadar sürer, ne kadar bellek ister? Kuramdaki $\Theta(V^2) \leftrightarrow \Theta(V + E)$ farkı ölçümde görünür mü? (b) Derin bir zincirde özyinelemeli ve açık yığınlı DFS nasıl davranır, özyinelemeli sürüm hangi derinlikte çöker? (c) CSR'nin bitişik belleği listeye göre ne kazandırır? **Düzen.** Seyrek graf: rastgele bir kapsayan ağaç + 3V rastgele kenar (yinelenenler atılır; $E \approx 4V$); yoğun graf: her $u < v$ çifti 1/2 olasılıkla ($E \approx V^2/4$); üç dilde aynı graf (ortak LCG, tohum 42). Ölçüt BFS(0)'ın işlemci zamanıdır: 1 ısınma + 5 tekrar, ortanca; büyüme hızı log–log eğimle ve oranlarla okunur (Sedgewick & Wayne, 2011). Makine: Intel(R) Xeon(R) Processor @ 2.80GHz, 2 çekirdek (sanal makine); gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; Python 3.11.15; bayraklar `-O2`. Ölçüm sırasında makinede başka işler de çalışıyordu: süreler "bu makinede"dir, katların yönü güvenilir, ikinci ondalığı değildir.

## 6.1 (a) Matris, liste ve CSR: süre ve bellek

**Tablo 11.7**

*Seyrek graf ($E \approx 4V$), C, -O2: BFS işlemci süresi (ortanca) ve temsil belleği. * ölçülmedi (bellek bütçesini aşar): süre $c \cdot V^2$ kestirimi (taban $V = 20\,000$), bellek hesap*

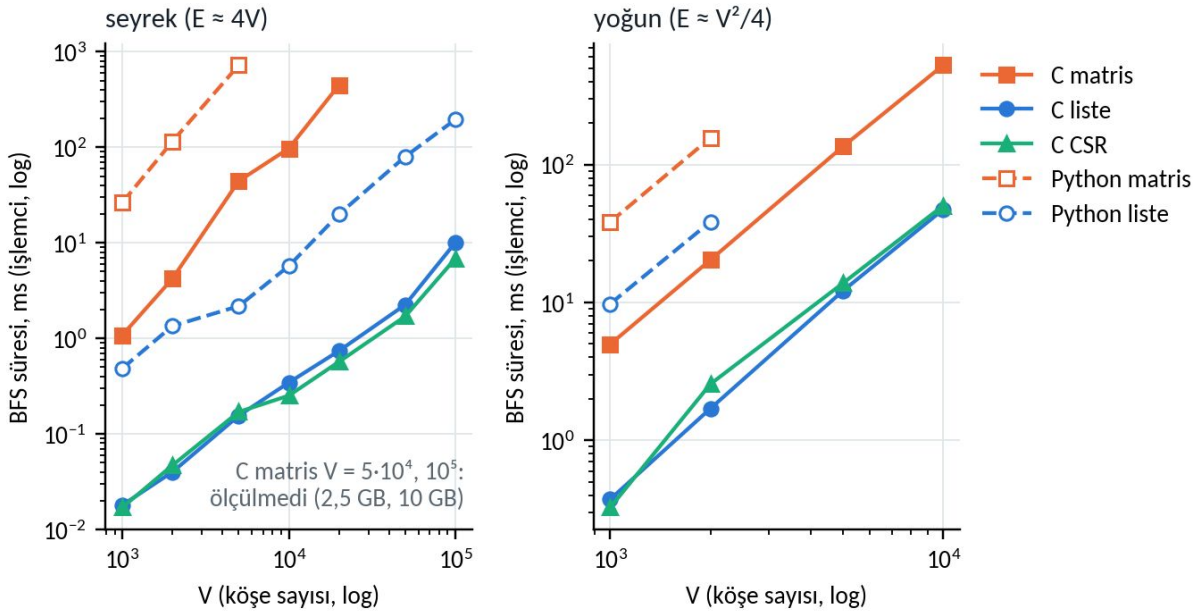
| V | E | matris (ms) | liste (ms) | CSR (ms) | matris / liste | matris bellek | liste bellek |
|---|---|---|---|---|---|---|---|
| 1 000 | 3 976 | 1,08 | 0,018 | 0,017 | 60 | 1,00 MB | 66,7 kB |
| 2 000 | 7 974 | 4,22 | 0,040 | 0,047 | 106 | 4,00 MB | 135,0 kB |
| 5 000 | 19 975 | 44,3 | 0,155 | 0,170 | 286 | 25,00 MB | 337,3 kB |
| 10 000 | 39 980 | 97,4 | 0,345 | 0,252 | 282 | 100,00 MB | 673,9 kB |
| 20 000 | 79 986 | 447,4 | 0,740 | 0,569 | 605 | 400,00 MB | 1,35 MB |
| 50 000 | 199 980 | 2 797* | 2,251 | 1,703 | — | 2,50 GB* | 3,38 MB |
| 100 000 | 399 983 | 11 186* | 10,068 | 6,848 | — | 10,00 GB* | 6,75 MB |

Log–log eğimler kuramla uyumludur: seyrek grafta matris 2,006 ($V = 1\,000$–$20\,000$; kuram 2), liste 1,345 ve CSR 1,227 ($V = 5\,000$–$100\,000$; kuram 1). Listenin 1'den büyük eğiminin iki olası nedeni vardır (ayrıca ölçülmedi): önbellek etkisi (V büyüdükçe diziler önbelleğe sığmaz ve rastgele komşu erişimi bellekten okunur; Mini-OBS 2 v2.10'un ölçümünde de aynı olgu görülür, v2.10 belgesi § 8.1) ve ölçüm gürültüsü: tablodaki $V \geq 10^5$ satırı ilk serinin ölçümüdür, üç serinin liste süreleri $V = 10^5$'te 4,80–

10,07 ms arasında değişti (Bölüm 6.3). V = 2·10⁴'te matris BFS listeden 605 kat yavaştır ve 400,00 MB ister; V = 10⁵'te 10 GB gerekirdi (Şekil 11.15).

**Yoğun grafta tablo değişir.** E ≈ V²/4 olunca liste de Θ(V²) olur (eğim 2,107) ve bellek üstünlüğü tersine döner: V = 10⁴'te matris 100,00 MB (hücre başına 1 bayt), liste 327,92 MB (kenar başına 13,1 bayt; her kenar iki listede 4 baytlık `int`, artı büyüme payı). Süre yine listeden yanadır (matris 525,1 ms, liste 47,2 ms): liste yalnız 1'leri okur, matris her hücre için bir karşılaştırma yapar; rastgele 0/1 hücrelerde dallanma tahmininin sık yanılması olası bir ek nedendir (ölçülmedi). Python'da aynı oranlar daha büyük sabitlerle görülür (Python belleği `tracemalloc` ile ölçüldü, diller arası bellek karşılaştırması yapılmamalıdır).

**Şekil 11.15**
*BFS süresi ↔ V, seyrek ve yoğun graf (C ve Python; log–log). Seyrekte matris V², liste ≈ V + E; yoğunda hepsi V² ölçeğinde*



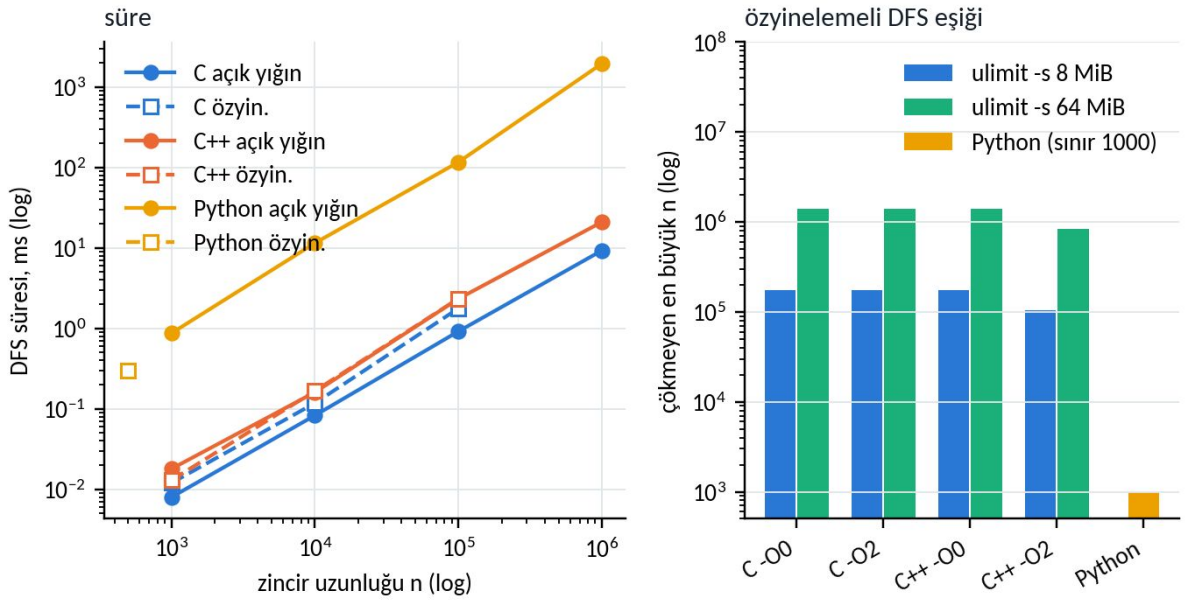

## 6.2 (b) Derin zincir: özyinelemeli ve açık yığınlı DFS

Açık yığınlı DFS'nin süresi n ile doğrusaldır (log–log eğim C 1,024, C++ 1,036, Python 1,105); 10⁶ köşede C 9,28 ms, Python 1,95 s sürer. Özyinelemeli sürüm çalıştığı sürece benzer hızdadır (C'de 10⁵'te 1,77 ms), ama bir derinlikten sonra çöker; Python'un özyinelemeli DFS'si yalnız n = 500'de ölçüldü (0,300 ms): sınır 996'da `RecursionError` verir. Çökme eşiği ikili aramayla bulundu (Şekil 11.16, sağda).

En derin çökmeyen n (1. deneme; ikinci denemeyle fark birkaç on köşe): 8 MiB yığında C -O0 174 061, C -O2 174 094, C++ -O0 174 063, C++ -O2 104 432; 64 MiB'ta C -O0 1 397 384, C -O2 1 397 423, C++ -O0 1 397 484, C++ -O2 838 452; Python 995. Çerçeve kestirimi (yığın sınırı / eşik) C'de 48,2, C++ -O2'de 80,3 bayttır.

Yığın sınırı 8 kat büyüyünce eşik ≈ 8 kat büyür: çerçeve boyu sabittir (Kavram 6'nın ölçtüğü 48 bayt C'de deneyle uyumlu). C++ -O2'de derleyici fonksiyonu farklı düzenlediği için çerçeve büyüdü ve eşik düştü: aynı kaynak kod, farklı bayrak, farklı sınır. Ders: **özyineleme derinliği girdiye bağlıysa** (zincir biçimli bir graf, uzun bir bağlı liste) özyinelemeli kod bir girdide çalışıp ötekinde çöker; açık yığın derinliği belleğe taşır.

**Şekil 11.16**
*Derin zincirde DFS: süre ↔ n (log–log) ve özyinelemeli DFS'nin çökme eşiği (8 ve 64 MiB yığın)*




## 6.3 (c) CSR: bitişik bellek ne kazandırır?

Liste / CSR süre katı (> 1: CSR hızlı; V ≥ $10^5$'te liste ve CSR için üçer ayrı seri ölçüldü, kat iki ortanca sürenin oranıdır): C'de V = $10^5$'te 1,58 (tek tek serilerin katları 1,47, 1,58, 1,25), $10^6$'da 1,26; C++'ta $10^5$'te 2,20. Küçük V'de bütün yapılar önbelleğe sığar, fark gürültüde kaybolur. Python'da CSR çoğunlukla **yavaştır** ($10^5$'te kat 0,82): Python listesi tamsayı nesnelerine başvuru tutar, bitişik tamsayı dizisi değildir; indis hesabının yorumlanma maliyeti önbellek kazancını aşar. CSR'nin belleği seyrek grafta listeninkinden ≈ 1,87 kat küçüktür.

## 6.4 Kestirimler, sınırlılıklar ve kendi makinenizde

- **Ölçülmeyenler** (41 satır, `h11_deney_olculmeyen.csv`): seyrek matris V ≥ 5·$10^4$ (2,5–10 GB), yoğun graflar V ≥ 2·$10^4$ (1,5 GB'lık süreç bütçesi), Python matris V ≥ $10^4$ (süre bütçesi). * işaretli süreler c·$V^2$ kestirimidir.
- Bellek C'de ayrılan bayt sayacı, C++'ta vektör kapasitesi, Python'da `tracemalloc` ile ölçüldü: yöntemler farklıdır. Çökme eşikleri makineye, derleyiciye ve bayraklara özgüdür.
- Kendi makinenizde: `cd kodlar/deney && python3 h11_olcum_calistir.py && python3 h11_cozumle.py` (≈ 3 dk). Eğimler, katların yönü ve eşiğin yığın sınırıyla orantısı aynı kalmalıdır; Windows'ta (MinGW) varsayılan yığın 1 MiB'tır, eşik ≈ 8 kat düşer.

# 7. Hata Avcısı

Hatalı ve düzeltilmiş programlar `kodlar/hata_avcisi/` altındadır (uyarısız derlenir; başlıklarında "BU DOSYA KASITLI HATA İÇERİR" yazar). Belirtiler bu makinede, bu çalıştırmada gözlendi; ASan raporlarında adres, süreç numarası ve yol sadeleştirildi. Hata Avcısı 3 ve 7'nin belirtileri "bu derleyicide, bu çalıştırmada" geçerlidir.

**Tablo 11.8**
*Hata Avcısı: hata, belirti, neden ve çözüm*

| # | Hata | Belirti (bu makinede) | Neden | Çözüm |
|---|---|---|---|---|
| 1 | Ziyaret işaretini kuyruktan **çıkarırken** koymak | G'de 16 ekleme (V = 8), en uzun kuyruk 5; 8 × 8 ızgarada 12 869 ekleme | Henüz işaretsiz köşe birden çok komşudan kuyruğa girer | İşaret kuyruğa **eklerken**; ekleme = V |
| 2 | Yönsüz grafta kenarı tek yönde eklemek (C++) | Σ derece = 10 (2E = 20 olmalı); BFS(7)'den hiçbir köşeye ulaşılamaz | v → u unutuldu; 0'dan BFS doğru görünür | Her kenar iki listeye; Σ derece = 2E denetimi |
| 3 | Izgarada sınırı komşuya **eriştikten sonra** denetlemek (C, Python) | ASan: heap-buffer-overflow (ada_sil, satır 38); Python: IndexError | Okumanın kendisi dizi dışıdır (tanımsız davranış); Python'da negatif indis sondan okur | Önce `icinde(ni, nj)`, sonra erişim (kısa devre) |
| 4 | Ağırlıksız en kısa yolu DFS ile aramak | 2'ye uzaklık 5 (BFS: 1) | DFS'nin ilk ulaştığı yol en kısa değil | BFS: uzaklık sırasıyla keşif |
| 5 | Topolojik sıralamada döngüyü denetlememek (C, Python) | C: eksik plan "AP1 MAT1 MAT2 AYR"; Python: VY -> ALG çiğnenir | Sıralanan sayısı V ile karşılaştırılmıyor; gri ve siyah ayrılmıyor | \|sıra\| < V ise döngü; üç renk |
| 6 | Python: `def dfs(g, u, ziyaret=[])` | 3. çağrı [0, 1, 3, 2, 0, 4, 5] döndürür | Varsayılan liste tanımda **bir kez** oluşturulur, paylaşılır | `ziyaret=None`, içeride yeni liste |
| 7 | Derin zincirde özyinelemeli DFS | C, n = $10^6$: SIGSEGV (çıkış 139); Python, n = 2000: RecursionError | Derinlik n; sınır 8 MiB (C) ya da 1000 çağrı (Python) | Açık yığınlı DFS (Örnek 6) |

**Hata Avcısı 1'e yakından bakış.** İşaret `u` kuyruktan çıkınca konur; `v` eklenirken yalnız "işlendi mi?" sorulur, "kuyrukta mı?" sorulmaz. G'de 7 köşesi 5 kez işlenir: 0'dan 7'ye 5 ayrı en kısa yol vardır ve her biri 7'yi bir kez kuyruğa ekler. Izgarada köşe hücreden başlayan BFS'de bir hücreye varan **en kısa yolların sayısı** kadar ekleme olur: k × k ızgarada C(2k, k) − 1; k = 12'de (V = 144) 2 704 155 ekleme. Düzeltilmiş sürüm her ızgarada tam V ekleme yapar.

**Hata Avcısı 3.** 38. satırda komşu hücre önce okunur, sınır sonra denetlenir:

C `hata_avcisi/H11_HA3_sinir_sonra.c · satır 37–39`

```c
            int ni = i + di[k], nj = j + dj[k];
            if (iz[ni * Y + nj] == '#' && icinde(ni, nj)) {  /* HATA */
                iz[ni * Y + nj] = '.';
```

`&&` sonucu "kurtarır" gibi görünür (sınır dışı değer kullanılmaz), ama **okumanın kendisi** dizi dışına erişimdir. ASan'sız derlemede çıktı doğru görünür (`ada sayısı (4-komşuluk) = 4`): tanımsız davranış sessizdir. AddressSanitizer okumayı yakalar (sadeleştirilmiş):

ASAN (SADELEŞTİRİLMİŞ)

```
$ gcc -std=c17 -O1 -g -fsanitize=address,undefined H11_HA3_sinir_sonra.c && ./a.out
==PID==ERROR: AddressSanitizer: heap-buffer-overflow
READ of size 1
#0 0x… in ada_sil H11_HA3_sinir_sonra.c:38
0x… is located 6 bytes before 24-byte region [0x…,0x…)
SUMMARY: AddressSanitizer: heap-buffer-overflow H11_HA3_sinir_sonra.c:38 in ada_sil
```

Python sürümünde negatif indis **hata vermez**, listenin sonundan okur: `ada 2: (0, 4) hücresinden, 3 hücre (şimdiye dek negatif indisle okuma: 4)`; alt kenarda ise `IndexError`. Düzeltme her iki dilde işlenenlerin yerini değiştirmektir: `icinde(ni, nj) && iz[ni * Y + nj] == '#'`.

**Hata Avcısı 6** Python'a özgüdür:

Python `hata_avcisi/H11_HA6_degisken_varsayilan.py · satır 16–21`

```python
def dfs(g, u, ziyaret=[]):                  # HATA: değiştirilebilir
    ziyaret.append(u)                       # varsayılan argüman
    for v in g[u]:
        if v not in ziyaret:
            dfs(g, v, ziyaret)
    return ziyaret
```

İkinci çağrı G2'de 0'dan başlar ama liste ilk çağrıdan kalmıştır: `2. çağrı, G2, kaynak 0: [0, 1, 3, 2, 0]`. Ayrıca `v not in ziyaret` listede doğrusal aramadır: DFS'nin maliyeti O(V·(V + E)) olur. Düzeltilmiş sürüm `ziyaret=None` ile başlar ve `[False] * len(g)` kullanır. **Hata Avcısı 5**'in C sürümü Kahn'ın sonucunu denetlemeden yazar: döngülü önkoşul dosyasında 9 dersten 4'ü sıraya girer ve program bunu "ders planı" diye sunar; Python sürümü iki renkli DFS ile bütün dersleri içeren ama `VY -> ALG` önkoşulunu çiğneyen bir sıra üretir.

# 8. Dil Penceresi: C · C++ · Python

**Tablo 11.9**
*Dil karşılıkları: graf temsilleri, kuyruk ve yığın, özyineleme sınırı ve kütüphaneler*

| Öğe | C | C++ | Python |
|---|---|---|---|
| Komşuluk listesi | dinamik dizilerden oluşan dizi: `KomsuDizisi *komsu` (≈ `int **` + boyut dizisi), `realloc` ile büyür (H8) | `std::vector<std::vector<int>>`; `push_back` | liste listesi `[[] for _ in range(V)]`; adlı köşelerde `dict` (Kavram 7) |
| Komşuluk matrisi | `int m[MAKS][MAKS]` ya da `calloc(V * V, 1)` + `m[u * V + v]` | `std::vector<std::vector<int>>(V, std::vector<int>(V))` | `[[0] * V for _ in range(V)]`; `[[0] * V] * V` aynı satırı paylaşır |
| Kuyruk (BFS) | dizi kuyruğu (H9): `q[son++]`, `q[bas++]` | `std::queue<int>`: `push`, `front`, `pop` | `collections.deque`: `append`, `popleft` (O(1)) |
| Yığın (DFS) | dizi yığın (H9) ya da çerçeve dizisi (Örnek 6) | `std::vector` (`push_back`/`back`/`pop_back`) ya da `std::stack` | liste: `append`, `pop` |
| Özyineleme sınırı | çağrı yığını, `ulimit -s` (Linux'ta 8 MiB); aşılınca SIGSEGV | aynı; çerçeve boyu bayraklara bağlı | `sys.getrecursionlimit()` = 1000; `RecursionError` |
| En küçük öbek (Kahn) | elle (H7) | `std::priority_queue<int, std::vector<int>, std::greater<int>>` | `heapq` (en küçük öbek) |
| Graf kütüphanesi | yok | standartta yok; Boost.Graph (Siek vd., 2002) | standartta yok (`graphlib.TopologicalSorter`, 3.9+); networkx (Hagberg vd., 2008) |
| Ad karşılaştırma | `strcmp` (bayt) | `std::string::compare` (bayt) | `bytes` karşılaştırması (str: kod noktası) |

**Kuyruk için Python listesi kullanmayın.** `list.pop(0)` kalan bütün başvuruları sola kaydırır: O(n). Kavram 3'te n ekleme + n baştan çıkarma, n = 10 000'de `list` ile 269,4 ms, `deque` ile 0,55 ms; n = 80 000'de 11 380 ms ↔ 5,39 ms sürdü (bu çalıştırmada): n 8 katına çıkınca `list` süresi ≈ 42 katına çıkar (karesel büyümede 64), `deque` süresi ≈ 9,8 katına (doğrusal büyümede 8; milisaniye düzeyindeki süreler gürültülüdür). BFS'ye çevrilirse `list` kuyruğu $\Theta(V + E)$'yi $\Theta(V^2 + E)$'ye çevirir.

**Kütüphaneler yalnız doğrulama içindir** (Tablo 11.9; `graphlib` için Python Software Foundation, t.y.): bu derste kütüphaneler **kâhindir** (Kavram 4: 13 denetim); ödevde algoritma yazılır.

**Adlı köşeler.** Python'da komşuluk "listesi" bir `dict` olabilir (anahtar ad, değer komşu adları; ziyaret `set`, kuyruk `deque`): Kavram 7 kurgusal bir mahalle ağında BFS yapar (`Yayla 3 adım`). C'de ad → numara eşlemesi gerekir (Örnek 13: ikili arama; Mini-OBS 2: karma tablosu). Üç dil de adları bayt sırasıyla karşılaştırır; Türkçe alfabe sırası için ayrı karşılaştırıcı gerekir.

# 9. Mühendis Gözüyle: Temsil Bir Modelleme Kararıdır

- **Temsil seçimi ölçülebilir bir karardır.** Seyrek grafta (yol ağı, sosyal ağ, önkoşul grafı) liste ya da CSR; matris yalnız yoğun grafta ve O(1) "komşu mu?" gerektiğinde. $V = 2 \cdot 10^4$ seyrek grafta matris 400,00 MB, liste 1,35 MB; BFS 447,4 ms ↔ 0,74 ms. Mini-OBS 2 v2.10'da matrisle Kahn V = 16 000'de listeyle Kahn'dan 249 kat yavaştır (v2.10 belgesi § 8.2, Tablo 4). CSR salt okunur, çok kez dolaşılan graflar içindir; sık kenar ekleme/silmede dinamik listeler gerekir.
- **Üretim kodunda derin özyinelemeden kaçının.** Derinliği girdiye bağlı özyineleme bir hizmet dışı bırakma riskidir: uzun bir zincir süreci çökertir. Bu makinede özyinelemeli DFS 8 MiB yığında C'de (-O0) 174 061, C++ -O2'de 104 432, Python'da 995 derinlikte durdu; açık yığın $10^6$'yı işledi (Bölüm 6.2). Yığını büyütmek (`ulimit -s`, `sys.setrecursionlimit`) sorunu ertelemektir.
- **Belirlilik test edilebilirliktir.** Artan komşu sırası, Kahn'da en küçük numara ve bayt sıralı adlar üç dilin aynı çıktıyı vermesini sağlar; karma tablosu sırasıyla çalışan bir planlayıcının "doğru" planı çalıştırmadan çalıştırmaya değişir.
- **Kâhinle ve bellek denetimiyle test edin.** BFS uzaklıkları kenar koşuluyla, topolojik sıra "her kenar ileri gider" koşuluyla, bileşenler ve iki parçalılık bağımsız bir kütüphaneyle **sonuçtan** denetlenir; okuyucular 28 bozuk ve sınır dosyasında üç dilde aynı ileti ve çıkış kodunu verir. İndis hesabıyla dolu graf kodu (`i * Y + j`, `bas[u + 1]`) AddressSanitizer ve Valgrind altında temizdir (`kodlar/BELLEK_DENETIMI.txt`).

# 10. Mini-OBS 2 v2.10: Ders Önkoşul Grafı

**Bu hafta:** Mini-OBS'ye **ders önkoşul grafı** ekliyoruz: önkoşulları dosyadan okuyup yönlü bir graf (CSR) kuruyoruz; **Kahn** algoritmasıyla geçerli bir ders planı (eşitlikte küçük ders kodu), **DFS'nin üç rengiyle** hatalı önkoşul döngüsünü (döngüdeki derslerle) buluyoruz; her dersin **en erken dönemini** (en uzun önkoşul zinciri) hesaplıyor ve bunun **BFS katmanından** (en kısa zincir) neden farklı olduğunu gösteriyoruz; öğrencinin aldığı derslere göre **şimdi alınabilecek dersleri** listeliyoruz. (v2.10 belgesi § 10)

Bu haftanın bütün algoritmaları burada birleşir: önkoşul grafı CSR'dir (Bölüm 3.5), plan en küçük öbekli Kahn'dır (4.7), hatalı önkoşul üç renkli DFS'nin geri kenarıdır (4.8; DFS açık yığınla yazılır, 4.4), en erken dönem Kahn sırasında tek geçiştir. Kavramsal düğüm: **en erken dönem en uzun önkoşul zinciridir, BFS katmanı en kısa zinciri verir**; kurgusal örnek katalogda 48 dersin 15'inde BFS katmanı dönemden küçüktür (v2.10 belgesi § 4.4). Ölçekte: rastgele DAG'da V = 1 000 000, E = 4 000 000 için bütün plan 1,03 s sürer ve okunan kenar sayısı kuramla birebir 5E'dir (v2.10 belgesi § 8.1).

## 10.1 Gereksinimler

- `graf.h/.c`: kenar dizisi; CSR kurma (komşular artan); ortak graf dosyası biçimini (Bölüm 3.4) okuma (biçim hatasında satır numarası, çıkış 1); çok kaynaklı BFS; **açık yığınlı** DFS (beyaz/gri/siyah; ilk geri kenarın kapattığı döngü; bitiş sırası); Kahn (giriş derecesi 0 olanlardan **en küçük numaralı**: en küçük öbek); en uzun yol katmanı (topolojik sırada tek geçiş).
- `onkosul.h/.c`: ders kodlu katalog (`V E yonlu`; `KOD AKTS DÖNEM Ad`; `ÖNKOŞUL DERS`); köşeler kod sırasında; kod → köşe **v2.9 karma tablosuyla**; bilinmeyen kod ve yinelenen önkoşul biçim hatası.
- `ders_plani`: geçerli plan ve doğrulaması (her önkoşul dersinden önce); döngü varsa döngüdeki dersler ve planlanamayan dersler, çıkış 1; dönem tablosu ve en uzun zincir; BFS katmanının dönemden küçük olduğu dersler (kısa ve uzun zincirleriyle); `--alinan` / `--ogrenci` ile alınabilir dersler; `--uzaklik KOD` ile önkoşul uzaklığı.
- Testler: CSR ve dolaşmalar ↔ kenar listesi ve geçişli kapanış (Floyd–Warshall); Kahn ↔ kaba "en küçük hazır köşe" seçimi; dönem ↔ gevşetme; katalog okuma ↔ `qsort`'lu kod sırası.
- Ölçüm: V = $10^3$…$10^5$ (isteğe bağlı $10^6$) rastgele DAG'da süre ↔ V + E; kenar sayacı = kuramın değeri; komşuluk matrisiyle kıyas.

## 10.2 İpuçları

- Komşu listelerini **artan** tutun (CSR'yi iki kararlı sayma geçişiyle kurmak bunu bedavaya verir); BFS/DFS sırası, Kahn'ın eşitlik kuralı ve testlerin beklenen çıktıları buna bağlıdır.
- Kahn'da "giriş derecesi 0" adaylarını en küçük öbekte tutun: v2.6'nın öbeği "en sonda geleni" köke koyuyorsa karşılaştırıcıyı **ters** çevirin. Sıraya V'den az köşe girdiyse graf döngülüdür.
- DFS'de **iki** değil **üç** renk kullanın: siyah köşeye giden kenar döngü değildir (v2.10 belgesi Şekil 2c). Döngüyü bulmak için ayrıca arama yapmayın: gri köşeden yığının tepesine kadar olan parça döngüdür (`konum[v]`: v'nin yığındaki yeri).
- Özyinelemeli DFS yerine açık yığın: her köşe için "sıradaki komşu" göstergesi (`ilerle[u]`) tutun.
- En erken dönem için BFS kullanmayın: dönem(v) = 1 + max(önkoşulların dönemi); Kahn'ın verdiği sırada tek geçiş yeter. Kendinizi `graf_kucuk.txt` ile sınayın (K2: BFS 2, dönem 3).
- Ders kodunu karma tablosuna anahtar yapmak için birebir bir sayıya çevirin (37 tabanı, boş konum 0) ya da v2.9'un tablosunu dizgi anahtarlı yapın; iki farklı kodun aynı anahtarı almadığını test edin.
- Alınabilir derslerde her ders için önkoşulları tek tek aramayın: alınmış derslerin ardıllarını sayın, sayaç = giriş derecesi ise ders alınabilir.

## 10.3 Başlangıç noktası

- **Verilir:** kendi v2.9'unuz (ya da referans v2.9), `graf.h` ve `onkosul.h` (bildirimler, sözleşmeler, sahiplik), `ornek_veri/onkosul.txt` ve üç test dosyası, `graf_uret` ve `olcer_graf`'ın ölçüm düzeneği (DAG üretimi, sayaç ve süre), v2.6'nın öbeği (öncelik kuyruğu işlemlerini siz ekleyeceksiniz), v2.10 belgesi Şekil 1 ve 2'nin örnekleri.
- **Yazılır:** `graf.c` (CSR, dosya, BFS, DFS, Kahn, katman), `onkosul.c` (katalog, plan, sorgular), `ders_plani.c`; testler (birim + kâhin); ölçüm ve yorum.
- **Değiştirilmez:** v1.0 kuralları ve menü, v2.9'un karma tablosu, ortak graf dosyası biçimi (Bölüm 3.4), ortak modüller.

- **Meydan okuma:** (a) Tarjan ya da Kosaraju ile kuvvetli bağlı bileşenleri bulup **bütün** döngüleri bildirin (yalnız ilki değil); (b) geçişli indirgeme ile gereksiz önkoşulları (A → C varken A → B → C) bulun; (c) dönem başına AKTS üst sınırıyla dönem ataması — liste çizelgeleme sezgiseliyle — ve en erken dönemle farkı; (d) DFS'yi özyinelemeli yazıp yığın sınırını ölçerek bulun (yalnız isteğe bağlı argümanla): C'de çerçeve küçük olduğundan $10^5$ köşelik zincirde `ulimit -s` sınırı (varsayılan 8 MiB) aşılmayabilir — H11'in ölçümünde (-O0) eşik ≈ 174 000 köşeydi; taşmayı $10^6$ köşelik zincirde ya da `ulimit -s` değerini düşürerek gözleyin.

## 10.4 Kabul ölçütleri ve test senaryoları

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri (414) ve `test_kahin` (300 deneme) geçer.
2. `./ders_plani` şu satırları basar: `Döngü denetimi (DFS, üç renk, açık yığın): YOK (geri kenar 0) — graf yönlü döngüsüz (DAG)` ve `Doğrulama: 52 önkoşulun hepsinde önkoşul dersten önce: EVET`; plan `ATA202 BLM101 BLM102 BLM103` ile başlar.
3. Aynı çıktıda `En uzun önkoşul zinciri: 8 ders (FIZ101 → FIZ102 → BLM203 → BLM204 → BLM301 → BLM310 → BLM405 → BLM406)`, BFS katmanı en erken dönemden küçük 15 ders ve `BLM202  BFS katmanı 3, en erken dönem 4` vardır.
4. `./ders_plani testler/onkosul_dongu.txt` döngüyü `BLM301 → BLM302 → BLM303 → BLM301` olarak yazar, `Kahn: 3/7 ders sıraya girdi` der ve 1 ile çıkar; `./ders_plani testler/onkosul_bozuk.txt` `satır 7: bilinmeyen ders kodu: BLM999` iletisiyle 1 ile çıkar.
5. `./ders_plani --ozet --alinan BLM101,MAT101,FIZ101,BLM102` `Şimdi alınabilecek dersler (12 ders, 57 AKTS; okunan kenar 8)` basar; `--uzaklik BLM401` 1 adımda BLM304 ve MAT203'ü, toplam 10 önkoşulu, en erken dönem 6 ve BFS katmanı 3'ü verir.
6. `./ders_plani testler/graf_kucuk.txt --uzaklik K2`: K2'nin en erken dönemi 3, BFS katmanı 2.
7. `./test_kahin --hata-goster gri` ve `--hata-goster katman` uyuşmazlık ve en küçük karşı örnek basar, 1 ile çıkar; `make asan` ve `make valgrind` temizdir.
8. Ölçüm raporunda: V = $10^3$…$10^5$ (en az dört nokta) süre tablosu ve log–log grafik; kenar sayacının kuramla (Kahn 2E, DFS/BFS/katman E) eşitliği; (V + E) başına süre ve yorumu; matrisle Kahn'ın en az üç V'de kıyası.

**Tablo 11.10**
*Mini-OBS 2 v2.10 test senaryoları (v2.10 belgesi § 10)*

| # | Girdi | Beklenen çıktı / davranış |
|---|---|---|
| 1 | 0 → 1 → 2, 0 → 2 (`graf_kucuk.txt`) | Kahn 0 1 2; K2: dönem 3, BFS katmanı 2 |
| 2 | 0 → 3, 1 → 2 | öbekli Kahn 0 1 2 3; FIFO 0 1 3 2 (ikisi de geçerli) |
| 3 | elmas 0 → 1, 0 → 2, 1 → 3, 2 → 3 | DFS döngü YOK; bitiş 3 1 2 0; 2 → 3 çapraz kenar |
| 4 | 0 → 1 → 2 → 0, 2 → 3, 3 → 3 | döngü 0 1 2; 2 geri kenar (öz döngü dahil); Kahn 0 köşe |
| 5 | 1 → 0 (iki köşe) | döngü YOK (0 siyah) — iki renkli DFS burada yanılır |
| 6 | `onkosul_dongu.txt` | döngü BLM301 BLM302 BLM303; planlanamayan 4 ders (BLM401 dahil); çıkış 1 |

| # | Girdi | Beklenen çıktı / davranış |
|---|---|---|
| 7 | kenar satırında `BLM999` | biçim hatası, satır 7, çıkış 1 |
| 8 | aynı önkoşul iki kez | `yinelenen önkoşul: A B`, satır numarası |
| 9 | `--alinan BLM101,MAT101,FIZ101,BLM102` | 12 alınabilir ders; BLM201 var, BLM202 yok |
| 10 | `--alinan BLM101,BLM202` | UYARI: önkoşulu alınmadan alınmış 1 ders (BLM202) |
| 11 | hiç ders alınmamış | kalan en uzun zincir 8 (mezuniyete en az 8 dönem) |
| 12 | $10^5$ köşelik zincir | DFS taşmaz; dönem(son) = $10^5$ |

# 11. Alıştırmalar ve Atölyeler

Türler: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez, D) hata bulma, E) Parsons (Parsons & Haden, 2006), F) kod yazma, G) diller arası okuma. Dosyalar `kodlar/alistirmalar/` altındadır: önce **çalıştırmadan** tahmin edin, sonra çalıştırın. Kodunuzu bir kâhinle sınayın, C/C++'ı AddressSanitizer ile çalıştırın. Çözümler öğretim üyesindedir.

## 11.1 İzleme ve çıktı tahmini (A)

**Alıştırma 11.1** ★☆☆ *Çıktı tahmini*

Program 7 köşeli yönsüz bir grafı komşuluk matrisinde kurar, BFS(0) ve özyinelemeli DFS(0) yapar (komşular artan sırada; `dfs` fonksiyonu dosyanın 21–30. satırlarında, aynı matrisi tarar). Üç çıktı satırını çalıştırmadan yazın. Grafı çizin: BFS ağacı ile DFS ağacı aynı mı? 6'ya en kısa yol hangisi?

**alistirmalar/H11_A01_cikti_tahmini.c · satır 34–64**

```
34      int q[V], uzak[V], bas = 0, son = 0;
35      kenar(0, 1);
36      kenar(0, 3);
37      kenar(1, 2);
38      kenar(1, 4);
39      kenar(3, 4);
40      kenar(4, 5);
41      kenar(2, 6);
42      kenar(5, 6);
43      for (int v = 0; v < V; v++) {
44          uzak[v] = -1;
45      }
46      uzak[0] = 0;
47      q[son++] = 0;
48      printf("BFS(0) sırası:");
49      while (bas < son) {
50          int u = q[bas++];
51          printf(" %d", u);
52          for (int v = 0; v < V; v++) {
53              if (m[u][v] && uzak[v] == -1) {
54                  uzak[v] = uzak[u] + 1;
55                  q[son++] = v;
56              }
57          }
58      }
59      printf("\nuzak:");
60      for (int v = 0; v < V; v++) {
61          printf(" %d", uzak[v]);
```

```
62      }
63      printf("\nDFS(0) sırası:");
64      dfs(0);
```

**Alıştırma 11.2** ★☆☆ *İzleme*

G'de (Şekil 11.2) **BFS(3)** ve **DFS(3)** yapın. BFS: her adımda çıkan köşe ve kuyruk (Şekil 11.5 biçiminde), katmanlar, ebeveyn dizisi. DFS: keşif/bitiş zamanları, parantez dizisi, geri kenarlar. 3'ten 6'ya en kısa yol? Örnek 3 ve 5'in kaynağını değiştirerek denetleyin.

**Alıştırma 11.3** ★★☆ *İzleme*

Yönlü grafın kenarları: 0 → 2, 1 → 2, 1 → 3, 2 → 4, 3 → 4, 3 → 5, 4 → 5. (a) Giriş derecelerini yazın. (b) Kahn'ı **en küçük öbekle** izleyin: her adımda çıkan köşe ve öbek. (c) **FIFO kuyrukla** sıra nedir? (d) DFS (kökler artan sırada) bitiş sırası ve tersi nedir? (e) Her köşenin en erken dönemi? (f) 5 → 1 kenarı eklenirse Kahn kaç köşeyi sıralar, DFS hangi geri kenarı bulur?

## 11.2 Karmaşıklık (B)

**Alıştırma 11.4** ★★☆ *Karmaşıklık*

$V = 10^4$ köşeli grafta BFS'nin "komşu okuma" sayısı (a) seyrek, $E = 4V$, (b) yoğun, $E = V^2/4$ iken matrisle ve listeyle kaçtır? Belleği (matris hücre başına 1 bayt, liste kenar başına 2 × 4 bayt) karşılaştırın. Bölüm 6.1'in ölçümleri aynı yönde mi; matris hangi durumda bellekte kazanır, süre neden yine listeden yanadır?

**Alıştırma 11.5** ★★☆ *Karmaşıklık*

$V = 10^6$, $E = 4 \cdot 10^6$ bir DAG'da Kahn (a) FIFO kuyrukla, (b) en küçük öbekle kaç kenar azaltması ve kaç kuyruk/öbek işlemi yapar; öbeğin ek maliyeti hangi terimdir? Dönem hesabı ne ekler? Hata Avcısı 1'de ızgarada ekleme sayısı neden üsteldir (k × k ızgarada bir hücreye varan en kısa yol sayısı)?

## 11.3 Değişmez yazma (C)

**Alıştırma 11.6** ★★☆ *Değişmez yaz*

BFS'nin **katman değişmezini** (kuyruk [d, ..., d, d + 1, ..., d + 1] biçimindedir) başlatma, koruma, sonlanma adımlarıyla yazın; (a) köşelerin uzaklık sırasıyla çıktığını, (b) uzak[v]'nin gerçek uzaklık olduğunu çıkarın. İşaret çıkarırken konsaydı (Hata Avcısı 1) hangi parça bozulurdu?

**Alıştırma 11.7** ★★☆ *Değişmez yaz*

Örnek 6'nın (A) yöntemi için kanıtlayın: "yığındaki çerçevelerin köşeleri tam olarak gri köşelerdir ve dipten tepeye kökten şimdiki köşeye giden yolu oluşturur". Buradan, çerçeve çıkarılırken u'nun altağacının bittiğini ve bitiş zamanlarının özyinelemeliyle aynı olduğunu çıkarın.

## 11.4 Hata bulma (D)

**Alıştırma 11.8** ★★☆ *Hata bul*

G'de BFS(0) ile uzaklık ve ebeveyn hesaplanıyor; program sonlanıyor ama bir değer yanlış. Çalıştırmadan: kuyruğa kaç ekleme olur, hangi uzaklık yanlış, neden? Düzeltin. Kuyruk neden V değil 64 öğelik; V'lik dizide bu hata ne yapardı?

**alistirmalar/H11_A08_hata_bul_bfs.c · satır 31–42**

```
    uzak[0] = 0;
    q[son++] = 0;
    while (bas < son) {
        int u = q[bas++];
        for (int v = 0; v < V; v++) {
            if (m[u][v] && ebeveyn[v] == -1) {
                uzak[v] = uzak[u] + 1;
                ebeveyn[v] = u;
                q[son++] = v;
            }
        }
    }
```

**Alıştırma 11.9** ★★☆ *Hata bul*

Python'da ızgara BFS'si (0, 0)'dan ulaşılabilen su hücrelerinin bir kısmını bulamıyor. İki çıktı satırını tahmin edin, hatalı satırı bulun (`gorulen[0][0] = True` hangi hücreleri işaretler?). C'de aynı hata yapılabilir mi?

**alistirmalar/H11_A09_hata_bul_izgara.py · satır 16–29**

```
    S, Y = len(IZGARA), len(IZGARA[0])
    gorulen = [[False] * Y] * S
    q = deque([(0, 0)])
    gorulen[0][0] = True
    doldurulan = 0
    while q:
        i, j = q.popleft()
        doldurulan += 1
        for di, dj in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            ni, nj = i + di, j + dj
            if 0 <= ni < S and 0 <= nj < Y and IZGARA[ni][nj] == "." \
                    and not gorulen[ni][nj]:
                gorulen[ni][nj] = True
                q.append((ni, nj))
```

## 11.5 Parsons (E)

**Alıştırma 11.10** ★★☆ *Parsons*

FIFO kuyruklu Kahn'ın ana döngüsünün karıştırılmış satırları; iki satır fazladır (dosya bu hâliyle derlenmez). `derece[]` giriş dereceleriyle dolu, derecesi 0 olanlar `q[0..son-1]`'dedir. Doğru sıra ve girinti? Fazla satırlar neden yanlış? Ders DAG'ında sıranız ne olur?

**alistirmalar/H11_A10_parsons_kahn.c · satır 8–21**

```
            if (derece[v] == 0) {
        sira[n++] = u;
    return n;
```

```
            derece[u]--;
        int u = q[bas++];
            q[son++] = v;
    while (bas < son) {
        for (size_t i = 0; i < g->komsu[u].boyut; i++) {
            derece[v]--;
            if (derece[v] == 1) {
            int v = g->komsu[u].veri[i];
            }
        }
    }
```

## 11.6 Kod yazma (F)

**Alıştırma 11.11** ★☆☆ *Kod yaz*

**Ters graf** (transpose): yönlü grafın kenarlarını ters çeviren `ters_graf`'ı Θ(V + E)'de yazın (G3 ile). Her köşenin giriş/çıkış derecesini ters grafla karşılaştıran tablo yazdırın, Σ giriş = Σ çıkış = E'yi doğrulayın. Ters graf hangi sorulara yarar?

**Alıştırma 11.12** ★★☆ *Kod yaz*

**Çok kaynaklı BFS:** birden çok çıkışlı ('E') bir ızgarada her açık hücrenin **en yakın** çıkışa uzaklığını tek BFS ile bulun (bütün çıkışlar başta kuyruğa). Kaynak başına ayrı BFS'yi kâhin yapın; kuyruğa ekleme sayılarını karşılaştırın; ulaşılamayan hücreleri sayın.

**Alıştırma 11.13** ★★★ *Kod yaz*

**Çap, yarıçap, merkez:** dış merkezlik (eccentricity) e(v), v'den en uzak köşeye uzaklıktır; çap max e(v), yarıçap min e(v), merkez e(v) = yarıçap olanlar. Her köşeden bir BFS ile Θ(V·(V + E))'de hesaplayın; G için tablo ve okunan kenar sayısı (V·2E ile karşılaştırın). Bağlı olmayan graf_bilesen.txt'de ne yapmalı?

## 11.7 Diller arası okuma (G)

**Alıştırma 11.14** ★★☆ *Diller arası okuma*

C++ programı ve Python eşi (`H11_A14_oncelik_kuyrugu.py`, `heapq`) ders DAG'ında Kahn'ı öncelik kuyruğuyla çalıştırır. Çıktılar aynı mı, hangisi Örnek 11'inkiyle aynı, neden? İkisi de geçerli mi? C++'ı Örnek 11'in sırasını verecek biçimde düzeltin.

**alistirmalar/H11_A14_oncelik_kuyrugu.cpp · satır 24–40**

```
24      std::priority_queue<int> h;
25      for (int v = 0; v < V; v++) {
26          if (derece[v] == 0) {
27              h.push(v);
28          }
29      }
30      std::printf("sıra:");
31      while (!h.empty()) {
32          int u = h.top();
33          h.pop();
```

```
34          std::printf(" %d", u);
35          for (int v : komsu[u]) {
36              if (--derece[v] == 0) {
37                  h.push(v);
38              }
39          }
40      }
```

## 11.8 Atölye 1 · İnsan BFS'si: tanışıklık ağında dalga (bütün sınıf, ≈ 20 dk)

ALGORİTMA ATÖLYESİ · **Sınıf bir graf olsun**

BFS'yi bir **kuyruk disipliniyle** yaşamak (etkin katılım; Naps vd., 2003). Her öğrenci bir köşe, "adını biliyorum" tanışıklıkları kenardır; tahta kuyruktur (baş solda).

- **Hazırlık (5 dk).** Herkes kâğıda en çok üç tanıdığını yazar; öğretim üyesi kaynağı seçip tahtaya yazar (uzaklık 0).
- **Dalga (10 dk).** Baştaki öğrenci kalkar, **işaretlenmemiş** tanıdıklarını söyler; her ad uzaklık(çıkan) + 1 ile **sona** yazılır ve hemen işaretlenir (işaret EKLERKEN). Kuyruk boşalana dek sürer. Her üç adımda **tahmin kapısı:** "Sıradaki kim, kimler girecek, uzaklığı kaç?"
- **Tartışma (5 dk).** Katmanları boyayın: en uzak öğrenci kaç adımda, ulaşılamayan var mı (ayrı bileşen)? İşaret çıkarırken konsaydı kim kaç kez girerdi? Sayaçları (ekleme, en uzun kuyruk) Örnek 3'ünkilerle (Şekil 11.5) karşılaştırın.

## 11.9 Atölye 2 · Labirent çözücü yarışması: BFS, DFS ve geri izleme (gruplar, ≈ 30 dk)

ALGORİTMA ATÖLYESİ · **Aynı labirentte üç strateji: kim daha az hücre ziyaret eder, kimin yolu en kısa?**

Şablon `alistirmalar/H11_A17_labirent_yarismasi.c` (ve `.py`) labirenti standart girdiden okur; üç yöntemin yol uzunluğunu ve ziyaret ettiği hücre sayısını yazar: BFS (Örnek 10), geri izleme (H3) ve **açık yığınlı DFS** (`dfs_labirent`: siz yazacaksınız). Yön sırası: sağ, aşağı, sol, yukarı.

ATÖLYE 2 · ŞABLONUN ÇIKTISI

```
$ ./H11_A17_labirent_yarismasi < ../girdiler/H11_10_labirent_bfs.txt
labirent 11 x 17, S = (1, 1), E = (9, 15), açık hücre = 81
yöntem               yol  ziyaret
BFS                   22       61
geri izleme (H3)      22       44
DFS (açık yığın)       -        -   (henüz yazılmadı)
```

- **Tahmin (5 dk).** Her grup açık yığınlı DFS'nin H3 labirentinde ve açık odada (`girdiler/H11_A17_oda.txt`) yolunu ve ziyaret sayısını tahmin edip tahtaya yazar.
- **Yazma (15 dk).** `dfs_labirent`'i Örnek 6'nın (A) yöntemiyle yazın (çerçeve: hücre + sıradaki yön; ziyaret = itilen hücre; çıkışta yığındaki çerçeveler yoldur). ASan ile çalıştırın.
- **Yarış (10 dk).** Geri izlemeyle karşılaştırın: neden aynı ya da farklı? Hangisi en kısa yolu **güvenceyle** verir, hangisi az bellek kullanır? DFS'nin BFS'den **az** hücre ziyaret edip **uzun** yol bulduğu bir labirent ve tersini tasarlayın.

### 11.10 Atölye 3 · Ders planlayıcı: önkoşul listesinden DAG ve topolojik sıra (eşli, ≈ 30 dk)

ALGORİTMA ATÖLYESİ · **Kendi bölümünüzün ders planı**

Araç `alistirmalar/H11_A18_ders_planlayici.py` "ÖNKOŞUL DERS" ya da tek "DERS" satırlarından Kahn ile bir plan (eşitlikte alfabede önce gelen), en erken dönemleri ve varsa döngüyü yazar. Örnek liste `girdiler/H11_A18_onkosul_ornek.txt` **kurgusaldır** (13 ders, 12 önkoşul).

- **Elle (10 dk; eşler).** Örnek listeden DAG'ı çizin, giriş derecelerini yazın, Kahn'ı alfabetik kuralla izleyin, en erken dönemleri bulun; sonra aracı çalıştırıp karşılaştırın (`python3 H11_A18_ders_planlayici.py ../girdiler/H11_A18_onkosul_ornek.txt`).
- **Gerçek liste (15 dk).** Bölümünüzün önkoşullarını (öğretim üyesinin dağıttığı liste) aynı biçimde yazın. Mezuniyet en az kaç dönem (en uzun zincir)? Hangi dersin BFS katmanı en erken döneminden küçük (Bölüm 4.7, 10)?
- **Hata girişi (5 dk).** Bilerek bir döngü ekleyin: araç hangi döngüyü bildiriyor, hangi dersler plana giremiyor? Dönem başına 6 ders sınırı olsaydı plan nasıl değişirdi?

## 12. Kendini Değerlendirme

- ☐ Bir problemi graf olarak modelleyip dört temsili bellek ve işlem maliyetiyle karşılaştırabiliyorum.
- ☐ BFS'yi kuyrukla (işaret eklerken) yazıp uzaklıkları ve en kısa yolu çıkarabiliyorum.
- ☐ DFS'yi özyinelemeli ve açık yığınla yazıp keşif/bitiş zamanlarını açıklayabiliyorum.
- ☐ Bileşen, iki parçalılık ve döngü bulabiliyor; ızgarada sınırı erişmeden önce denetliyorum.
- ☐ Bir DAG'ı Kahn ve DFS ile sıralayıp döngüyü raporlayabiliyorum; en erken dönemi bulabiliyorum.
- ☐ Yığın taşmasını açık yığınla önleyebiliyor; $\Theta(V + E)$ ile $\Theta(V^2)$'yi ölçümle karşılaştırabiliyorum.

## 13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Graflar nerede derinleşecek?**

- **H12 (açgözlü algoritmalar):** ağırlıklı graflarda BFS'nin "katman = uzaklık" güvencesi bozulur: Dijkstra kuyruğu (uzaklık, köşe) öbeğiyle değiştirir; en küçük kapsayan ağaçta Kruskal kenar listesi + birleşim–bul, Prim öbek kullanır.
- **Veri Yapıları ve Algoritma Analizi dersleri:** kuvvetli bağlı bileşenler (Tarjan, 1972), köprüler ve eklem noktaları, en kısa yol ailesi (Bellman–Ford, Floyd–Warshall, A*), ağ akışı ve eşleme.
- **Ağ bilimi:** sosyal ağ analizi, PageRank, graf veritabanları; milyarlarca kenarda CSR ve açık yığın zorunludur.

MERAK KUTUSU · **Königsberg'in genel yanıtı: Euler yolu**

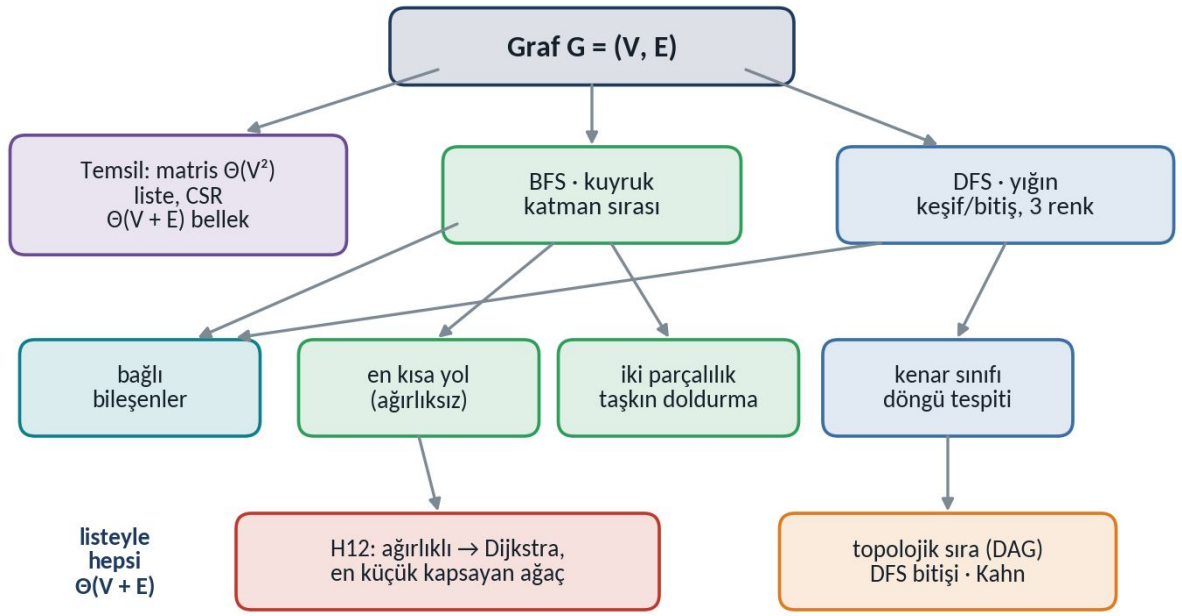
Euler'in akıl yürütmesi genelleşir: bağlı bir grafta **her kenardan tam bir kez geçen bir yürüyüş** (Euler yolu) ancak ve ancak tek dereceli köşe sayısı 0 ya da 2 ise vardır; 0 ise yürüyüş başladığı köşede biter (Euler turu), 2 ise o iki köşe başlangıç ve bitiştir. Gerekliliği Bölüm 1'de gördük; yeterliliği için bkz. Cormen vd. (2022).

G'nin dereceleri 2 3 2 2 4 3 2 2 (Örnek 1): tek dereceli köşeler 1 ve 5; öyleyse G'de 1'den 5'e bir Euler yolu vardır, Euler turu yoktur. Bu yolu bulun. Königsberg'de tur için kaç köprü eklemek gerekir?

# 14. Özet, Kavram Haritası ve Terim Sözlüğü

- Graf, köşe ve kenarlardan oluşan soyut bir modeldir: labirent, yol ağı, sosyal ağ ve önkoşul listesi aynı yapıdır. Seyrek graflarda liste ya da CSR $\Theta(V + E)$, matris $\Theta(V^2)$ bellek ister.
- BFS kuyrukla katman katman ilerler (işaret eklerken): ağırlıksız en kısa yol ve ebeveyn dizisinden yolun kendisi. DFS derine iner: üç renk, keşif/bitiş zamanları, kenar sınıfları; geri kenar döngüdür; derin graflarda açık yığın gerekir.
- Bileşenler, iki parçalılık, taşkın doldurma ve ada sayma birer dolaşmadır. Topolojik sıra yalnız DAG'da vardır (Kahn ya da DFS bitişinin tersi); ders DAG'ının 108 geçerli sırası vardır, en erken dönem en uzun önkoşul zinciridir.

**Şekil 11.17**
*Hafta 11 kavram haritası: temsil, iki dolaşma ve uygulamaları; H12'ye köprü*




**Tablo 11.11**
*Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)*

| Türkçe | İngilizce | Not |
|---|---|---|
| graf (çizge); köşe; kenar; derece | graph; vertex; edge; degree | yönlüde giriş/çıkış |
| yönlü / yönsüz; ağırlıklı; seyrek / yoğun | directed / undirected; weighted; sparse / dense | $E \approx V \leftrightarrow E \approx V^2$ |
| yol; döngü; bağlı bileşen; ağaç; ebeveyn | path; cycle; connected component; tree; parent | ağaç: $V - 1$ kenar |
| yönlü döngüsüz graf (DAG); topolojik sıralama | directed acyclic graph; topological sort | "yönlü dönüşsüz" da denir |
| komşuluk matrisi; komşuluk listesi; kenar listesi | adjacency matrix; adjacency list; edge list | CSR: compressed sparse row |
| genişlik / derinlik öncelikli arama; katman | breadth- / depth-first search; layer | BFS, DFS özgün kalır |
| keşif/bitiş zamanı; ağaç/geri/ileri/çapraz kenar | discovery/finish time; tree/back/forward/cross edge | parantez teoremi |
| iki parçalı graf; örtük graf; taşkın doldurma | bipartite graph; implicit graph; flood fill | ızgara: 4/8 komşuluk |
| yığın; öbek; açık yığın; çağrı yığını | stack; heap; explicit stack; call stack | yığın = stack, öbek = heap |

# 15. Kaynaklar


Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Eneström, G. (1910–1913). *Verzeichnis der Schriften Leonhard Eulers*. B. G. Teubner.

Euler, L. (1741). Solutio problematis ad geometriam situs pertinentis. *Commentarii Academiae Scientiarum Petropolitanae*, *8*, 128–140.

Grossman, J. W. (2025, August 12). *Facts about Erdős numbers and the collaboration graph*. The Erdős Number Project, Oakland University. https://sites.google.com/oakland.edu/grossman/home/the-erdoes-number-project/facts-about-erdoes-numbers-and-the-collaboration-graph

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15).

Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, *5*(11), 558–562. https://doi.org/10.1145/368996.369025

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson/Addison-Wesley.

Lee, C. Y. (1961). An algorithm for path connections and its applications. *IRE Transactions on Electronic Computers*, *EC-10*(3), 346–365. https://doi.org/10.1109/TEC.1961.5219222

Moore, E. F. (1959). The shortest path through a maze. In *Proceedings of an International Symposium on the Theory of Switching*, *Part II* (pp. 285–292). Harvard University Press.

Naps, T. L., Rößling, G., Almstrum, V., Dann, W., Fleischer, R., Hundhausen, C., Korhonen, A., Malmi, L., McNally, M., Rodger, S., & Velázquez-Iturbide, J. Á. (2003). Exploring the role of visualization and engagement in computer science education. *ACM SIGCSE Bulletin*, *35*(2), 131–152. https://doi.org/10.1145/782941.782998

Parsons, D., & Haden, P. (2006). Parson's programming puzzles: A fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education* (Vol. 52, pp. 157–163). Australian Computer Society.

Python Software Foundation. (t.y.). *The Python standard library*. Erişim tarihi 22 Ekim 2026, https://docs.python.org/3/library/

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Siek, J. G., Lee, L.-Q., & Lumsdaine, A. (2002). *The Boost Graph Library: User guide and reference manual*. Addison-Wesley.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, *1*(2), 146–160. https://doi.org/10.1137/0201010

# Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on üç basamağının C++ ve Python sürümlerinden çekirdek fonksiyonlar, dosyadaki satır numaralarıyla verilir; tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır. Bölüm 4'te üç dilde gösterilen çekirdekler (Örnek 4 `bfs_uzaklik` döngüsü, Örnek 5 `dfs_ziyaret`, Örnek 6 `dfs_cerceve`, Örnek 11 ana döngü, Örnek 12 `dfs_topo`) burada tekrarlanmaz; aynı dosyaların öteki fonksiyonları verilir. Örnek 3'ün BFS döngüsü Örnek 4'tekinin izleme satırlı biçimidir; burada dillerin ayrıştığı yer, kuyruğun yazdırılması gösterilir. Örnek 6'da (B) yönteminin döngüsü verilir; kenar sınıflandırması (`sinif`) sunumdaki "Üç dilde" slaytındadır. Bütün örneklerin çıktısı üç dilde aynıdır (=).

**Tablo 11.12**
*Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H11_03_bfs_katmanlar)*

| Dil | Derleme / çalıştırma | Not |
|---|---|---|
| C | `gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c -o X · ./X ../girdiler/graf_ornek.txt` | Örnek 3–8, 11, 12 graf dosyası okur |
| C++ | `g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X ../girdiler/graf_ornek.txt` | aynı argümanlar |
| Python | `python3 X.py ../girdiler/graf_ornek.txt (Windows: py X.py …)` | Python 3.10 ve üstü; yalnız standart kütüphane |
| Girdi | `./H11_09_tasma_doldurma < ../girdiler/H11_09_tasma_doldurma.txt (Örnek 10 da standart girdiden)` | dosyalar kodlar/girdiler/ altında; argümanlar KOD_DIZINI.md'de |

## A.1 Örnek 1 · Komşuluk matrisi

C++ **cpp/H11_01_komsuluk_matrisi.cpp · satır 76–87**

```cpp
76 static std::vector<int> komsulari_bul(const Matris& matris, int u)
77 {
78     std::vector<int> komsu;
79     int V = static_cast<int>(matris.size());
80     for (int v = 0; v < V; v++) {
81         okunan_hucre++;
82         if (matris[u][v] != 0) {
83             komsu.push_back(v);
84         }
85     }
86     return komsu;
87 }
```

Python **python/H11_01_komsuluk_matrisi.py · satır 51–59**

```python
51 def komsulari_bul(matris, u):
52     """u'nun komşuları: satır u baştan sona taranır (V hücre)"""
53     global okunan_hucre
54     komsu = []
55     for v in range(len(matris)):
```

```
        okunan_hucre += 1
        if matris[u][v] != 0:
            komsu.append(v)
    return komsu
```

**DİL PENCERESİ · Farklılaşan satırlar**

Matris C'de sabit boyutlu 2B dizi (`int matris[MAKS_V][MAKS_V]`, AP-1 H11), C++'ta `std::vector<std::vector<int>>`, Python'da liste listesidir (`[[0] * V for _ in range(V)]`; `[[0] * V] * V` aynı satırı V kez paylaşır, kullanılmaz). Kenarlar C'de `int[][2]`, C++'ta `std::pair` vektörü (yapılandırılmış bağlama `auto& [u, v]`), Python'da demet listesidir. 10^10 C'de `long long`, Python'da sınırsız tamsayıdır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 16, 34, 35, 209; Python 26, 27, 141.)

## A.2 Örnek 2 · Komşuluk listesi ve dosyadan okuma

C++ **cpp/H11_02_komsuluk_listesi.cpp · satır 148–162**

```cpp
static Graf graf_kur(const KenarListesi& kl)
{
    Graf g;
    g.V = kl.V;
    g.E = kl.E;
    g.yonlu = kl.yonlu;
    g.komsu.assign(g.V, std::vector<int>());
    for (const Kenar& k : kl.kenar) {
        g.komsu[k.u].push_back(k.v);
        if (!g.yonlu) {
            g.komsu[k.v].push_back(k.u);
        }
    }
    return g;
}
```

Python **python/H11_02_komsuluk_listesi.py · satır 100–109**

```python
def graf_kur(kl):
    """Kenarlar dosya sırasıyla eklenir (yönsüzde iki yöne)"""
    g = Graf()
    g.V, g.E, g.yonlu = kl.V, kl.E, kl.yonlu
    g.komsu = [[] for _ in range(g.V)]
    for u, v, _ in kl.kenar:
        g.komsu[u].append(v)
        if not g.yonlu:
            g.komsu[v].append(u)
    return g
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Komşuluk listesi C'de dinamik dizilerden (H8) oluşan dizidir: `KomsuDizisi *komsu` (eşdeğeri `int **` + boyut dizisi), büyütme `realloc` ile elle; C++'ta `std::vector<std::vector<int>>`, Python'da liste listesi (`push_back`/`append` kendisi büyür). Dosya üç dilde bayt olarak okunur (`fgets`, `std::getline`, `open(..., "rb")`); sayı denetimi elle yapılır, çünkü Python `int()` "+3", " 3" ve "3_000"ü de kabul eder. Sıralama araçtır: `qsort` / `std::sort` / `list.sort()`. C++ ve Python'da listeler `==` ile karşılaştırılır; C'de döngüyle. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 50, 74, 144, 168, 236; Python 40, 47, 57, 115, 158.)

## A.3 Örnek 3 · BFS katmanları

C++ **cpp/H11_03_bfs_katmanlar.cpp · satır 158–165**

```cpp
158 static void kuyruk_yaz(std::queue<int> q)
159 {
160     std::cout << '[';
161     for (bool ilk = true; !q.empty(); q.pop(), ilk = false) {
162         std::cout << (ilk ? "" : " ") << q.front();
163     }
164     std::cout << ']';
165 }
```

Python **python/H11_03_bfs_katmanlar.py · satır 119–120**

```python
119 def kuyruk_metni(q):
120     return "[" + " ".join(str(x) for x in q) + "]"
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Kuyruk C'de H9'un dizi kuyruğudur (her köşe en çok bir kez girdiği için kapasite V yeter, sarma gerekmez); C++'ta `std::queue<int>` (dolaşılamadığı için içerik bir kopyadan çıkarılarak yazdırılır), Python'da `collections.deque` (`popleft` O(1); `list.pop(0)` O(n) olurdu, bkz. kavram K03). Algoritmanın kendisi üç dilde aynı satırlardır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 172, 185, 197; Python 126, 136, 143.)

## A.4 Örnek 4 · En kısa yol ve ebeveyn

C++ **cpp/H11_04_bfs_en_kisa_yol.cpp · satır 182–194**

```cpp
182 static std::vector<int> yol_cikar(const std::vector<int>& ebeveyn, int s,
183                                  int t)
184 {
185     std::vector<int> yol;
186     if (t != s && ebeveyn[t] == -1) {
187         return yol;                       // ulaşılamaz
188     }
189     for (int v = t; v != -1; v = ebeveyn[v]) {
190         yol.push_back(v);                 // t, ebeveyn[t], ..., s
191     }
192     std::reverse(yol.begin(), yol.end());   // s .. t
193     return yol;
194 }
```

Python **python/H11_04_bfs_en_kisa_yol.py · satır 132–143**

```python
def yol_cikar(ebeveyn, s, t):
    """s'den t'ye yolu ebeveyn zincirinden geri çıkarır: [s, .., t];
    yol yoksa boş liste."""
    yol = []
    if t != s and ebeveyn[t] == -1:
        return yol                      # ulaşılamaz
    v = t
    while v != -1:
        yol.append(v)                   # t, ebeveyn[t], ..., s
        v = ebeveyn[v]
    yol.reverse()                       # s .. t
    return yol
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Yol geri çıkarma üç dilde aynıdır: ebeveyn zinciri t'den s'ye izlenir, sonra ters çevrilir. C'de ters çevirme elle (iki indis), C++'ta `std::reverse`, Python'da `list.reverse()`; C fonksiyonu yolu çağıranın dizisine yazıp kenar sayısını (yoksa -1) döndürür, C++ ve Python boş vektör/liste döndürür. Ulaşılamayan köşe sözleşmesi üç dilde aynı: uzaklık = ebeveyn = -1. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 159, 227; Python 115, 170.)

## A.5 Örnek 5 · Özyinelemeli DFS

C++ **cpp/H11_05_dfs_ozyinelemeli.cpp · satır 229–242**

```cpp
static int dfs_orman(const Graf& g, Dfs& d)
{
    int agac = 0;
    std::cout << "DFS ormanı, kökler artan sırada:";
    for (int r = 0; r < g.V; r++) {
        if (d.renk[r] == BEYAZ) {
            std::cout << ' ' << r;
            dfs_ziyaret(g, r, d);
            agac++;
        }
    }
    std::cout << '\n';
    return agac;
}
```

Python **python/H11_05_dfs_ozyinelemeli.py · satır 172–183**

```python
def dfs_orman(g, d):
    """DFS ormanı: her beyaz köşeden (artan sırada) yeni bir ağaç. Kökleri
    yazar; ağaç sayısını döndürür."""
    agac = 0
    print("DFS ormanı, kökler artan sırada:", end="")
    for r in range(g.V):
        if d.renk[r] == BEYAZ:
            print(f" {r}", end="")
```

```
            dfs_ziyaret(g, r, d)
            agac += 1
    print()
    return agac
```

**DİL PENCERESİ · Farklılaşan satırlar**

DFS durumu C'de diziler içeren bir yapıdır (`Dfs`; diziler `malloc` ile ayrılıp `dfs_birak` ile bırakılır), C++'ta kurucusu dizileri dolduran bir `struct`, Python'da `__slots__`'lı bir sınıftır. Kenar kaydı C'de dört paralel dizi, C++'ta yapı vektörü, Python'da demet listesidir. Python'da özyineleme sınırı (≈ 1000) bu küçük graflarda sorun değildir; derin zincir deney (b) ve Hata Avcısı 7'dedir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 169, 175, 185; Python 117, 133, 204.)

## A.6 Örnek 6 · Açık yığınlı DFS

C++ **cpp/H11_06_dfs_acik_yigin.cpp · satır 276–311**

```
    while (!y.empty()) {
        int u = y.back();
        y.pop_back();
        if (ziyaret[u]) {
            atlanan++;
            if (iz) {
                std::cout << "   çıkar " << u
                          << ": zaten ziyaretli, atla;  ";
                yigin_yaz(y);
            }
            continue;
        }
        ziyaret[u] = 1;
        sira.push_back(u);
        if (iz) {
            std::cout << "   çıkar " << u << ": ziyaret (" << sira.size()
                      << "), it:";
        }
        std::size_t k = g.komsu[u].size();
        int itilen = 0;
        for (std::size_t j = 0; j < k; j++) {
            int v = g.komsu[u][ters ? k - 1 - j : j];
            if (!ziyaret[v]) {
                y.push_back(v);
                itme++;
                itilen++;
                if (iz) {
                    std::cout << ' ' << v;
                }
            }
        }
        if (iz) {
            std::cout << (itilen ? "" : " -") << ";  ";
            yigin_yaz(y);
        }
    }
```

Python **python/H11_06_dfs_acik_yigin.py · satır 204–225**

```python
    while y:
        u = y.pop()
        if ziyaret[u]:
            atlanan += 1
            if iz:
                print(f"   çıkar {u}: zaten ziyaretli, atla;  "
                      + yigin_metni(y))
            continue
        ziyaret[u] = True
        sira.append(u)
        k = len(g.komsu[u])
        itilen = []
        for j in range(k):
            v = g.komsu[u][k - 1 - j if ters else j]
            if not ziyaret[v]:
                y.append(v)
                itme += 1
                itilen.append(v)
        if iz:
            print(f"   çıkar {u}: ziyaret ({len(sira)}.), it:"
                  + ("".join(f" {v}" for v in itilen) if itilen else " -")
                  + ";  " + yigin_metni(y))
```

DİL PENCERESİ • **Farklılaşan satırlar**

Yığın C'de sabit kapasiteli dizidir (çerçeve yığınında kapasite V; itmeli sürümde V + 2E), C++'ta `std::vector` (`push_back`/`back`/`pop_back`; `std::stack` da olurdu), Python'da liste (`append`/`pop`). C++'ta `y.back()` başvurusu `push_back` sonrası geçersiz olabilir; kod onu itmeden sonra kullanmaz. Python'da çerçeve değiştirilebilir olsun diye `[u, i]` listesidir. Karşılaştırma C'de `memcmp`, C++ ve Python'da `==`. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 220, 228, 236, 339; Python 167, 171, 244.)

## A.7 Örnek 7 · Bağlı bileşenler

C++ **cpp/H11_07_bagli_bilesenler.cpp · satır 162–192**

```cpp
static std::vector<int> bileseni_etiketle(const Graf& g, int s, int no,
                                          std::vector<int>& etiket,
                                          std::vector<int>& ebeveyn,
                                          std::pair<int, int>& ilk_dongu,
                                          Sayac& say)
{
    std::queue<int> q;
    std::vector<int> uyeler;
    etiket[s] = no;
    ebeveyn[s] = -1;
    q.push(s);
    say.ekleme++;
    say.bfs_sayisi++;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
```

```
        uyeler.push_back(u);
        for (int v : g.komsu[u]) {
            say.inceleme++;
            if (etiket[v] == -1) {
                etiket[v] = no;
                ebeveyn[v] = u;
                q.push(v);
                say.ekleme++;
            } else if (v != ebeveyn[u] && ilk_dongu.first == -1) {
                ilk_dongu = {u, v};      // ağaç dışı kenar: döngü
            }
        }
    }
    return uyeler;
}
```

Python **python/H11_07_bagli_bilesenler.py · satır 120–144**

```
def bileseni_etiketle(g, s, no, etiket, ebeveyn, say):
    """s'nin bileşenini BFS ile 'no' etiketiyle işaretler. Dönüş:
    (bileşenin köşeleri BFS sırasıyla, ağaç dışı ilk kenar ya da
    (-1, -1))"""
    q = deque()
    uyeler = []
    ilk_dongu = (-1, -1)
    etiket[s] = no
    ebeveyn[s] = -1
    q.append(s)
    say.ekleme += 1
    say.bfs_sayisi += 1
    while q:
        u = q.popleft()
        uyeler.append(u)
        for v in g.komsu[u]:
            say.inceleme += 1
            if etiket[v] == -1:
                etiket[v] = no
                ebeveyn[v] = u
                q.append(v)
                say.ekleme += 1
            elif v != ebeveyn[u] and ilk_dongu[0] == -1:
                ilk_dongu = (u, v)      # ağaç dışı kenar: döngü
    return uyeler, ilk_dongu
```

**DİL PENCERESİ · Farklılaşan satırlar**

C fonksiyonu bileşenin köşelerini çağıranın kuyruk dizisinde bırakır ve ilk ağaç dışı kenarı iki öğeli bir diziye yazar; C++ köşeleri vektörle döndürür, kenarı `std::pair` başvurusuyla verir; Python ikisini demet olarak döndürür. Bileşen başına kenar sayısı üç dilde liste uzunluklarının toplamının yarısıdır.

## A.8 Örnek 8 · İki parçalılık

C++ cpp/H11_08_iki_parcali.cpp · satır 161–194

```cpp
static bool bfs_boya(const Graf& g, int s, Boyama& b)
{
    std::queue<int> q;
    b.renk[s] = 0;
    b.uzak[s] = 0;
    b.ebeveyn[s] = -1;
    q.push(s);
    std::cout << "   kök " << s << " -> renk 0\n";
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        std::cout << "   çıkan " << u << " (renk " << b.renk[u] << "):";
        for (std::size_t i = 0; i < g.komsu[u].size(); i++) {
            int v = g.komsu[u][i];
            if (b.renk[v] == -1) {
                b.renk[v] = 1 - b.renk[u];        // karşı renk
                b.uzak[v] = b.uzak[u] + 1;
                b.ebeveyn[v] = u;
                q.push(v);
                std::cout << ' ' << v << " -> " << b.renk[v];
            } else if (b.renk[v] == b.renk[u]) {
                std::cout << ' ' << v << " ÇELİŞKİ\n";
                b.cu = u;
                b.cv = v;
                return false;
            } else {
                std::cout << ' ' << v << " uygun";
            }
            std::cout << (i + 1 < g.komsu[u].size() ? "," : "");
        }
        std::cout << '\n';
    }
    return true;
}
```

Python python/H11_08_iki_parcali.py · satır 121–149

```python
def bfs_boya(g, s, b):
    """s'nin bileşenini BFS ile boyar; her adımı yazar. Çelişkide durur:
    b.cu, b.cv kenarı kaydedilir, False döner."""
    q = deque()
    b.renk[s] = 0
    b.uzak[s] = 0
    b.ebeveyn[s] = -1
    q.append(s)
    print(f"   kök {s} -> renk 0")
    while q:
        u = q.popleft()
        parcalar = []
        for v in g.komsu[u]:
            if b.renk[v] == -1:
                b.renk[v] = 1 - b.renk[u]       # karşı renk
```

```
                b.uzak[v] = b.uzak[u] + 1
                b.ebeveyn[v] = u
                q.append(v)
                parcalar.append(f" {v} -> {b.renk[v]}")
            elif b.renk[v] == b.renk[u]:
                parcalar.append(f" {v} ÇELİŞKİ")
                print(f"   çıkan {u} (renk {b.renk[u]}):"
                      + ",".join(parcalar))
                b.cu, b.cv = u, v
                return False
            else:
                parcalar.append(f" {v} uygun")
        print(f"   çıkan {u} (renk {b.renk[u]}):" + ",".join(parcalar))
    return True
```

**DİL PENCERESİ · Farklılaşan satırlar**

Döngü iki yolun birleşimidir: u'dan ortak ataya ve v'den ortak ataya. C'de iki dizi ve indislerle, C++'ta iki vektörle yazdırılır; Python'da `ya + yb[-2::-1]` dilimi ikinci yolu atası hariç ters çevirip ekler. Renk -1 (boyanmadı) üç dilde aynı sözleşmedir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 201; Python 165, 168.)

## A.9 Örnek 9 · Taşkın doldurma ve ada sayma

C++ cpp/H11_09_tasma_doldurma.cpp · satır 38–69

```
static std::vector<int> doldur(std::string& iz, int i0, int j0, char eski,
                               char yeni, int yon, std::vector<int>* katman)
{
    std::queue<int> q;
    std::vector<int> sira;
    iz[i0 * Y + j0] = yeni;              // işaret kuyruğa EKLERKEN
    q.push(i0 * Y + j0);
    if (katman != nullptr) {
        (*katman)[i0 * Y + j0] = 0;
    }
    while (!q.empty()) {
        int h = q.front(), i = h / Y, j = h % Y;
        q.pop();
        sira.push_back(h);
        for (int k = 0; k < yon; k++) {
            int ni = i + di[k], nj = j + dj[k];
            deneme++;
            if (!icinde(ni, nj)) {
                sinir_disi++;             // erişilmeden reddedildi
                continue;
            }
            if (iz[ni * Y + nj] == eski) {
                iz[ni * Y + nj] = yeni;
                q.push(ni * Y + nj);
                if (katman != nullptr) {
                    (*katman)[ni * Y + nj] = (*katman)[h] + 1;
                }
```

```
            }
        }
    }
    return sira;
}
```

Python `python/H11_09_tasma_doldurma.py · satır 31–57`

```python
def doldur(iz, i0, j0, eski, yeni, yon, katman=None):
    """(i0, j0)'dan başlayıp 'eski' karakterli hücreleri 'yeni' yapar
    (BFS). iz: karakter listesi; yon: 4 ya da 8. Dönüş: değiştirilen
    hücreler (BFS sırasıyla). katman verilirse dalga numaraları yazılır."""
    global deneme, sinir_disi
    q = deque()
    sira = []
    iz[i0 * Y + j0] = yeni              # işaret kuyruğa EKLERKEN
    q.append(i0 * Y + j0)
    if katman is not None:
        katman[i0 * Y + j0] = 0
    while q:
        h = q.popleft()
        i, j = h // Y, h % Y
        sira.append(h)
        for k in range(yon):
            ni, nj = i + di[k], j + dj[k]
            deneme += 1
            if not icinde(ni, nj):
                sinir_disi += 1         # erişilmeden reddedildi
                continue
            if iz[ni * Y + nj] == eski:
                iz[ni * Y + nj] = yeni
                q.append(ni * Y + nj)
                if katman is not None:
                    katman[ni * Y + nj] = katman[h] + 1
    return sira
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Izgara C'de tek boyutlu `char` dizisidir (hücre (i, j) = `iz[i * Y + j]`, AP-1 H11 satır öncelikli düzen), C++'ta `std::string`, Python'da karakter listesi (dizgiler değiştirilemez). Sınır denetimi `icinde(i, j)` üç dilde erişimden ÖNCE yapılır; Python'da negatif indis hata vermez, listenin SONUNDAN okur: sınır dışı erişim C'deki gibi gürültülü değil, sessizce yanlıştır (Hata Avcısı 3). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 74, 122; Python 68, 93.)

## A.10 Örnek 10 · Labirentte BFS

C++ `cpp/H11_10_labirent_bfs.cpp · satır 53–92`

```cpp
static Sonuc bfs_labirent(Labirent& L)
{
    Sonuc s;
    int n = L.S * L.Y;
    std::vector<int> ebeveyn(n, -2), q;  // -2: görülmedi
```

```
    std::size_t bas = 0;
    int kaynak = L.sr * L.Y + L.sc, hedef = L.er * L.Y + L.ec;
    ebeveyn[kaynak] = -1;
    q.push_back(kaynak);
    while (bas < q.size()) {
        int h = q[bas++], r = h / L.Y, c = h % L.Y;
        if (h == hedef) {
            break;                      // E kuyruktan çıktı
        }
        for (int d = 0; d < 4; d++) {
            int yr = r + DR[d], yc = c + DC[d];
            if (acik_mi(L, yr, yc) && ebeveyn[yr * L.Y + yc] == -2) {
                ebeveyn[yr * L.Y + yc] = h;
                q.push_back(yr * L.Y + yc);
            }
        }
    }
    s.ziyaret = static_cast<long>(bas);
    s.eklenen = static_cast<long>(q.size());
    for (std::size_t i = 0; i < q.size(); i++) {    // işlenen ve sınır
        char& k = L.h[q[i] / L.Y][q[i] % L.Y];
        if (k == '.') {
            k = i < bas ? 'o' : '+';
        }
    }
    if (ebeveyn[hedef] != -2) {
        s.uzunluk = 0;
        for (int h = ebeveyn[hedef]; h != kaynak; h = ebeveyn[h]) {
            L.h[h / L.Y][h % L.Y] = '*'; // ebeveyn zinciri: yol
            s.uzunluk++;
        }
        s.uzunluk++;                    // son adım: E'ye giriş
    }
    return s;
}
```

Python **python/H11_10_labirent_bfs.py · satır 51–86**

```
def bfs_labirent(L):
    """Izgara BFS: S'den başlar, E kuyruktan çıkınca durur. Yol '*',
    işlenen öteki hücreler 'o', kuyrukta kalan (sınır) hücreler '+' ile
    işaretlenir (L değiştirilir). Kuyruk: liste + baş indisi (sınırı
    sonradan yazdırabilmek için deque yerine)."""
    s = Sonuc()
    ebeveyn = [-2] * (L.S * L.Y)        # -2: görülmedi
    q = []
    bas = 0
    kaynak, hedef = L.sr * L.Y + L.sc, L.er * L.Y + L.ec
    ebeveyn[kaynak] = -1
    q.append(kaynak)
    while bas < len(q):
        h = q[bas]
        bas += 1
        r, c = h // L.Y, h % L.Y
        if h == hedef:
```

```
            break                       # E kuyruktan çıktı
        for d in range(4):
            yr, yc = r + DR[d], c + DC[d]
            if acik_mi(L, yr, yc) and ebeveyn[yr * L.Y + yc] == -2:
                ebeveyn[yr * L.Y + yc] = h
                q.append(yr * L.Y + yc)
    s.ziyaret, s.eklenen = bas, len(q)
    for i, h in enumerate(q):           # işlenen ve sınır
        if L.h[h // L.Y][h % L.Y] == ".":
            L.h[h // L.Y][h % L.Y] = "o" if i < bas else "+"
    if ebeveyn[hedef] != -2:
        s.uzunluk = 0
        h = ebeveyn[hedef]
        while h != kaynak:
            L.h[h // L.Y][h % L.Y] = "*"    # ebeveyn zinciri: yol
            s.uzunluk += 1
            h = ebeveyn[h]
        s.uzunluk += 1                  # son adım: E'ye giriş
    return s
```

**DİL PENCERESİ · Farklılaşan satırlar**

Labirent C'de sabit boyutlu 2B `char` dizisi içeren bir yapıdır; yapı ataması (`a = *asil`) diziyi de kopyalar. C++'ta `std::vector<std::string>` kopyası derin kopyadır; Python'da satırlar listelerden oluştuğu için kopya açıkça (`kopya()`) yapılır — `L2 = L` yalnız başvuruyu kopyalardı. BFS kuyruğu üç dilde dizi + baş indisidir; böylece kuyrukta kalan sınır hücreleri sonradan işaretlenebilir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 24, 195; Python 27, 31, 112.)

## A.11 Örnek 11 · Kahn

C++ **cpp/H11_11_kahn.cpp · satır 247–264**

```
247 static bool gecerli_mi(const Graf& g, const std::vector<int>& sira)
248 {
249     if (static_cast<int>(sira.size()) != g.V) {
250         return false;
251     }
252     std::vector<int> konum(g.V);
253     for (int i = 0; i < g.V; i++) {
254         konum[sira[i]] = i;
255     }
256     for (int u = 0; u < g.V; u++) {
257         for (int v : g.komsu[u]) {
258             if (konum[u] >= konum[v]) {
259                 return false;
260             }
261         }
262     }
263     return true;
264 }
```

Python **python/H11_11_kahn.py · satır 163–174**

```python
def gecerli_mi(g, sira):
    """Kâhin: her kenar u -> v için konum[u] < konum[v] mi?"""
    if len(sira) != g.V:
        return False
    konum = [0] * g.V
    for i, v in enumerate(sira):
        konum[v] = i
    for u in range(g.V):
        for v in g.komsu[u]:
            if konum[u] >= konum[v]:
                return False
    return True
```

DİL PENCERESİ · **Farklılaşan satırlar**

En küçük öbek C'de elle (H7: yukarı taşı / aşağı süz), C++'ta `std::priority_queue<int, std::vector<int>, std::greater<int>>` (varsayılanı en BÜYÜK öbektir), Python'da `heapq` (liste üzerinde en küçük öbek). Öbek içeriği yalnız gösterim için sıralanır (`qsort`, kopyadan çıkarma, `sorted`); çıkarma sırası üç dilde aynıdır, çünkü anahtarlar tekildir. Geçerli sıra sayımı H3 geri izlemesidir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 155, 158, 197, 212; Python 12, 115, 139, 143.)

## A.12 Örnek 12 · DFS topolojik sırası

C++ **cpp/H11_12_dfs_topolojik.cpp · satır 228–232**

```cpp
    std::vector<int> sira(t.bitis_sirasi.rbegin(), t.bitis_sirasi.rend());
    std::vector<int> konum(g.V);
    for (int i = 0; i < g.V; i++) {
        konum[sira[i]] = i;
    }
```

Python **python/H11_12_dfs_topolojik.py · satır 175–178**

```python
    sira = t.bitis_sirasi[::-1]          # ters çevir
    konum = [0] * g.V
    for i, v in enumerate(sira):
        konum[v] = i
```

DİL PENCERESİ · **Farklılaşan satırlar**

Gri yol C'de dizi + sayaç (`gri_yol`, `ng`), C++'ta vektör (`push_back`/`pop_back`), Python'da listedir. Bitiş sırasının tersi C'de indisle, C++'ta ters yineleyicilerle (`rbegin`, `rend`), Python'da `[::-1]` dilimiyle alınır. Döngünün başı C'de döngüyle, C++'ta `std::find`, Python'da `list.index` ile bulunur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 168, 209; Python 134, 165.)

## A.13 Örnek 13 · Ortak yazarlık ağı

C++ **cpp/H11_13_ortak_yazarlik.cpp · satır 77–100**

```cpp
static int bfs(const Komsu& komsu, int s, std::vector<int>& uzak,
               std::vector<int>& ebeveyn, long& inceleme)
{
    std::queue<int> q;
    int ulasilan = 0;
    uzak.assign(komsu.size(), -1);
    ebeveyn.assign(komsu.size(), -1);
    uzak[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ulasilan++;
        for (int v : komsu[u]) {
            inceleme++;
            if (uzak[v] == -1) {
                uzak[v] = uzak[u] + 1;
                ebeveyn[v] = u;
                q.push(v);
            }
        }
    }
    return ulasilan;
}
```

Python **python/H11_13_ortak_yazarlik.py · satır 56–73**

```python
def bfs(komsu, s):
    """BFS: (uzak, ebeveyn, ulaşılan, inceleme); ulaşılamayan -1"""
    q = deque()
    uzak = [-1] * len(komsu)
    ebeveyn = [-1] * len(komsu)
    ulasilan = inceleme = 0
    uzak[s] = 0
    q.append(s)
    while q:
        u = q.popleft()
        ulasilan += 1
        for v in komsu[u]:
            inceleme += 1
            if uzak[v] == -1:
                uzak[v] = uzak[u] + 1
                ebeveyn[v] = u
                q.append(v)
    return uzak, ebeveyn, ulasilan, inceleme
```

DİL PENCERESİ · **Farklılaşan satırlar**

Adlar üç dilde BAYT dizisi olarak karşılaştırılır: C'de `strcmp`, C++'ta `std::string::compare` (ikisi de unsigned char sırası), Python'da `bytes`; böylece “Ömer” ve “İlker” (UTF-8 çok baytlı) üç dilde aynı yere sıralanır. C'de adlar `malloc` + `memcpy` ile kopyalanır (`strdup` C17'de yok) ve programın sonunda bırakılır. Ortalamalar kayan nokta yerine tamsayı aritmetiğiyle iki ondalığa yuvarlanır. Ad → köşe eşlemesi ikili aramayla elle (H4); kütüphane karşılığı `bsearch` / `std::lower_bound` / `bisect` ya da sözlüktür. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 43, 124, 196, 252; Python 18, 52, 140, 195.)